Eduvos
Your Education. Your Future.

Eduvos Assessment Coversheet

Academic Details

Campus:	**Online Campus (Pretoria Registered)**
Faculty:	**Information Technology**
Module Code:	**ITBDA4-14**
Lecturer's Name:	**Mr. G Lekula**
Group (if applicable)	**N/A**

Assessment Details

Assessment Type:	**Formative Assessment: Project 2**
Assignment/Brief Number:	
Due Date/ Submission Date:	**13 November 2026**

Student Details (Individual, applicable)

Student Full Name:	**Mikaylen Naidoo**
Student Number:	**CON-1459393-V9B6**

Academic Integrity

AI Disclosure Completed [X] **Yes** [ ] **No**

Declaration:
I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature **Mikaylen Naidoo MN**

Date **13 November 2026**

Lecturer Use Only

Lecturer Comments:	
Marks Awarded (%):	
Lecturer Signature:	
Date:	

Assessment:

Here is the GitHub Link to Access all of my code.
https://github.com/Mikaylen-Naidoo/CON-1459393-V9B6-ITBDA4-14-Project-2

I have also attached my code in segments for Question 1 and there are Appendixes at the end with the rest of my code.

Question 1: Keyword search with Hadoop Streaming MapReduce in R

1.1.) Logical layout and explanation

The figure below shows the logical layout of the job as I designed it. The corpus and the keyword enter the job, one mapper processes each document, the framework groups the mapper output by filename, a single reducer summarises each group, and the result is written to one output file.

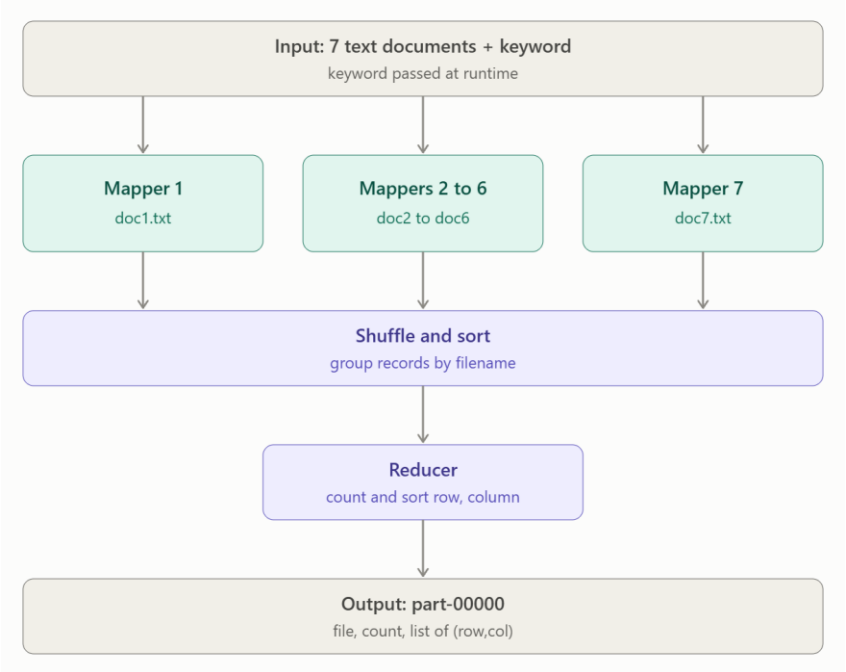


Figure 1: Logical layout of the keyword-search MapReduce job: seven documents are processed by parallel mappers, shuffled and sorted by filename, reduced to one line per document and written to `part-00000`.

Stage 1: Input and splitting. My corpus consists of seven text documents. Hadoop treats each document as one input split, so each is handled by its own mapper and the documents are processed in parallel. On a cluster I would store the files in the Hadoop Distributed File System (Shvachko et al., 2010), whose design follows the Google File System (Ghemawat, Gobioff and Leung, 2003). I pass the keyword to every mapper as a command-line argument at job submission, so the same program can search for any word. I configured the job so that a file is never divided across mappers (see 1.4). This matters because a mapper numbers rows from the start of its split, so a split that began part-way through a file would give me incorrect row numbers.

Stage 2: Map phase. Each mapper reads its document one line at a time and keeps a running row counter that increases for every line, including blank lines. On each line it performs a case-insensitive, whole-word search for the keyword. For every occurrence it emits one key-value record in the form `filename <TAB> row <TAB> column`, where the filename is the key. I take the filename from the `mapreduce_map_input_file` environment variable that Hadoop sets for each split. Lines without the keyword emit nothing.

Stage 3: Shuffle and sort. I rely on the Hadoop framework to collect the records from all mappers, group them by key (the filename) and sort them. This guarantees that all matches from one document reach the reducer together.

Stage 4: Reduce phase. The reducer receives the sorted records and processes one document's group at a time. For each document it counts the matches, sorts the positions numerically (by row, then by column) and writes a single line: `filename <TAB> total matches <TAB> (row,col) (row,col) ....`

Stage 5: Output. I write the result to `part-00000`. It lists each document that contains the keyword with its match count and the exact row and column of every occurrence. Documents without the keyword produce no records and therefore do not appear.

Worked example for the keyword "africa". In `doc1.txt` the first line contains the word at character 64, so my mapper emits `doc1.txt 1 64`. Line 3 contains only "African", which is not a whole-word match, so I get nothing for that line. The reducer receives the single record for `doc1.txt` and outputs `doc1.txt 1 (1,64)`.

1.2.) Mapper implementation

The mapper is `mapper.R`. It takes the keyword as its first argument, reads one document from standard input and emits one record per match. I use regular expressions for the matching (Thompson, 1968). This is how it works:

1. I read the keyword from the command-line argument and stop with an error message if none is supplied, which enforces the dynamic-input requirement.
2. I escape any special regular-expression characters in the keyword so that it is matched literally, then add word boundaries (`\b`) for whole-word matching.
3. I read the filename from the Hadoop environment variable.
4. I read the input in blocks of lines, increment the row counter for every line, and use `gregexpr` with the case-insensitive flag `(?i)` to find every match on the line.
5. I mark the text as UTF-8 so that column positions count characters rather than bytes. The documents contain curly quotation marks that would otherwise shift the column numbers.

Code for Q1.2.

```
#!/usr/bin/env Rscript
# ---------------------------------------------------------------------
# mapper.R  -  Hadoop Streaming mapper (keyword search with row/column)
# Usage : Rscript mapper.R <keyword>
# Input : text lines on stdin (one input split = one document)
# Output: filename <TAB> row <TAB> column      (one record per match)
# ---------------------------------------------------------------------

args <- commandArgs(trailingOnly = TRUE)
if (length(args) < 1 || !nzchar(args[1])) {
  write("ERROR: keyword argument missing. Usage: Rscript mapper.R <keyword>", stderr())
  quit(status = 1)
}
keyword <- args[1]

# Escape regex metacharacters so the keyword is matched literally,
# then add word boundaries so "africa" does not match "African".
escape_regex <- function(x) gsub("([][{}()+*^$|\\\\?.])", "\\\\\\1", x)
pattern <- paste0("\\b", escape_regex(keyword), "\\b")

# Hadoop exposes the current input file as an environment variable
# (Hadoop 2+/3: mapreduce_map_input_file, Hadoop 1: map_input_file)
input_file <- Sys.getenv("mapreduce_map_input_file", unset = Sys.getenv("map_input_file", unset =
"unknown"))
doc_name   <- basename(input_file)

con <- file("stdin", open = "r")
row <- 0L

repeat {
  lines <- readLines(con, n = 1000L, warn = FALSE)
```

```
  if (length(lines) == 0L) break
  Encoding(lines) <- "UTF-8"          # mark (do not re-encode) so columns count characters, not bytes

  for (line in lines) {
    row <- row + 1L                      # blank lines still count as rows
    if (!nzchar(line)) next

    # case-insensitive search; (?i) works with perl = TRUE
    m <- gregexpr(paste0("(?i)", pattern), line, perl = TRUE)[[1]]
    if (m[1] != -1L) {
      for (col in as.integer(m)) {       # one record per occurrence
        cat(doc_name, "\t", row, "\t", col, "\n", sep = "")
      }
    }
  }
}
close(con)
```

1.3.) Reducer implementation

The reducer is `reducer.R`. It reads the sorted mapper records from standard input and produces one summary line per document. Because the shuffle has already sorted the records by key, all records for one document are consecutive, so I can process the input as a stream: when the key changes, the reducer writes out the previous document's result and starts a new group, and it writes the final group when the input ends. I sort the positions numerically, because a text-based sort would place row 10 before row 2.

Code for 1.3.)

```
#!/usr/bin/env Rscript
# ---------------------------------------------------------------------
# reducer.R - Hadoop Streaming reducer
# Input : filename <TAB> row <TAB> column   (sorted by filename by the shuffle)
# Output: filename <TAB> total matches <TAB> (row,col) (row,col) ...
# ---------------------------------------------------------------------

con <- file("stdin", open = "r")

current <- NULL
rows <- integer(0); cols <- integer(0)

emit <- function(key, rows, cols) {
  if (is.null(key)) return(invisible())
  o <- order(rows, cols)                         # numeric order: row, then column
  locs <- paste0("(", rows[o], ",", cols[o], ")", collapse = " ")
  cat(key, "\t", length(rows), "\t", locs, "\n", sep = "")
}

repeat {
  lines <- readLines(con, n = 1000L, warn = FALSE)
  if (length(lines) == 0L) break
  for (line in lines) {
    f <- strsplit(line, "\t", fixed = TRUE)[[1]]
    if (length(f) < 3L) next                     # skip malformed records
    key <- f[1]
    if (is.null(current) || key != current) {    # new key -> flush previous group
      emit(current, rows, cols)
      current <- key
      rows <- integer(0); cols <- integer(0)
    }
    rows <- c(rows, as.integer(f[2]))
    cols <- c(cols, as.integer(f[3]))
  }
}
emit(current, rows, cols)                        # flush the last group
close(con)
```

1.4.) Hadoop MapReduce configuration settings

I submit the job with Hadoop Streaming through the script `run_job.sh`. I give the keyword to the script as an argument (for example `./run_job.sh africa`) and it passes it on to the mapper, so the keyword is never hardcoded.

Code for 1.4)

```
#!/bin/bash
# run_job.sh - Submit the keyword search as a Hadoop Streaming job
# Usage: ./run_job.sh <keyword>

KEYWORD="$1"
INPUT=/user/mikay/map_reduce_test_docs
OUTPUT=/user/mikay/keyword_output

hdfs dfs -rm -r -f $OUTPUT

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -D mapreduce.job.name="Keyword search - $KEYWORD" \
  -D mapreduce.job.reduces=1 \
  -D mapreduce.input.fileinputformat.split.minsize=1073741824 \
  -D stream.num.map.output.key.fields=1 \
  -files mapper.R,reducer.R \
  -mapper "Rscript mapper.R $KEYWORD" \
  -reducer "Rscript reducer.R" \
  -input $INPUT \
  -output $OUTPUT
```

Table 1: Hadoop Streaming settings used in `run_job.sh`

Setting	Purpose
`hdfs dfs -rm -r -f $OUTPUT`	Hadoop will not start a job if the output directory already exists, so any previous output is removed first.
`hadoop-streaming-*.jar`	The Hadoop Streaming library, which lets any executable (here R scripts) act as mapper and reducer by reading standard input and writing standard output.
`mapreduce.job.name`	Names the job, including the keyword, so that it can be identified in the job history.
`mapreduce.job.reduces=1`	Uses a single reducer, so all results are written to one sorted output file, `part-00000`.
`mapreduce.input.fileinputformat.split.minsize=1073741824`	Sets the minimum split size to 1 GB so that a file is never divided across mappers, which keeps row numbering correct.

Setting	Purpose
`stream.num.map.output.key.fields=1`	Defines the first tab-separated field (the filename) as the key; row and column form the value.
`-files mapper.R,reducer.R`	Distributes the two scripts to every node that runs a task.
`-mapper "Rscript mapper.R $KEYWORD"`	Runs the mapper and passes the keyword at run time.
`-reducer "Rscript reducer.R"`	Runs the reducer on the shuffled and sorted mapper output.
`-input` and `-output`	The HDFS directory that holds the documents and the directory where the results are written.

I place the generic `-D` options before the streaming-specific options, as Hadoop requires. In `mapred-site.xml` the property `mapreduce.framework.name` selects where the job runs: `yarn` for a cluster or `local` for a single machine.

Running the job and demonstration evidence

Hadoop is not installed on my development machine, so I demonstrated the complete pipeline locally with `run_local.R`. The script uses exactly the same `mapper.R` and `reducer.R`: it starts one mapper process for each document, sets the same `mapreduce_map_input_file` variable that Hadoop provides, sorts the mapper output by key and passes it to the reducer, then saves the result as `part-00000`. I enter the keyword at run time through a prompt in the R console.

Code for `run_local.R` (local demonstration of the job)

```
# ====================================================================
# run_local.R  -  Local demonstration of the Hadoop Streaming job
# Simulates: MAP (one mapper per document) -> SHUFFLE & SORT -> REDUCE
# using the SAME mapper.R and reducer.R that Hadoop would run.
#
# HOW TO RUN: open this file in RStudio and click "Source".
#             Type a keyword in the Console when asked, then press Enter.
# ====================================================================

# ---- Settings (no changes needed) --------------------------------
input_dir <- "data/map_reduce_test_docs"
rscript   <- file.path(R.home("bin"), "Rscript")

# ---- Dynamic keyword input (not hardcoded) -----------------------
if (interactive()) {
  keyword <- trimws(readline("Enter keyword to search: "))
} else {
  keyword <- "africa"        # only used if run outside RStudio
}
if (!nzchar(keyword)) stop("No keyword entered. Source the script again and type a keyword.")

# ---- Safety checks -----------------------------------------------
if (!file.exists("mapper.R") || !file.exists("reducer.R")) {
  stop("mapper.R / reducer.R not found in the working directory: ", getwd(),
```

```
       "\nSave both files in your RStudio project folder and try again.")
}
if (!dir.exists(input_dir)) {
  stop("Input folder not found: ", input_dir)
}

# ---- Collect the documents (ignore macOS junk files) --------------
files <- list.files(input_dir, pattern = "\\.txt$", full.names = TRUE, recursive = TRUE)
files <- files[!grepl("__MACOSX", files) & !grepl("^\\._", basename(files))]
files <- sort(files)
if (length(files) == 0) stop("No .txt documents found in: ", input_dir)

cat("Keyword  :", keyword, "\n")
cat("Documents:", length(files), "\n\n")

# ---- MAP: one mapper process per document (= one input split) -----
map_out <- character(0)
for (f in files) {
  Sys.setenv(mapreduce_map_input_file = f)       # what Hadoop sets for each split
  out <- system2(rscript, c("mapper.R", shQuote(keyword)),
                 stdin = f, stdout = TRUE)
  map_out <- c(map_out, out)
}
cat("---- MAP output (filename <TAB> row <TAB> column) ----\n")
if (length(map_out) == 0) {
  cat("(no matches found)\n")
  stop_quietly <- TRUE
} else {
  writeLines(map_out)
}

# ---- SHUFFLE & SORT: group records by key (filename) ---------------
if (length(map_out) > 0) {
  key      <- sub("\t.*", "", map_out)
  shuffled <- map_out[order(key, method = "radix")]
  tmp      <- tempfile()
  writeLines(shuffled, tmp)

  cat("\n---- SHUFFLE & SORT output (grouped by filename) ----\n")
  writeLines(shuffled)

  # ---- REDUCE ------------------------------------------------------
  red_out <- system2(rscript, "reducer.R", stdin = tmp, stdout = TRUE)

  cat("\n---- REDUCE output (filename <TAB> total <TAB> (row,col) list) ----\n")
  writeLines(red_out)

  writeLines(red_out, "part-00000")             # mimics Hadoop's output file
  cat("\nResult saved to:", file.path(getwd(), "part-00000"), "\n")
}
```

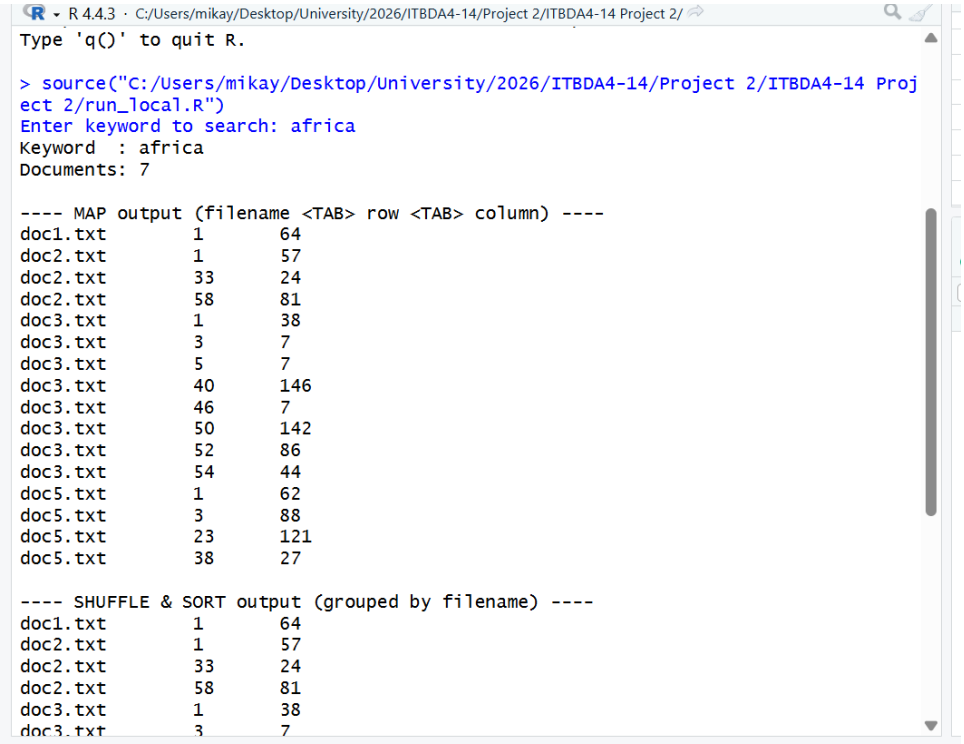


Figure 2: Console output for the keyword "africa": the keyword is typed in at the prompt, the seven documents are found and the mapper records (filename, row, column) are printed, followed by the start of the shuffle-and-sort output.

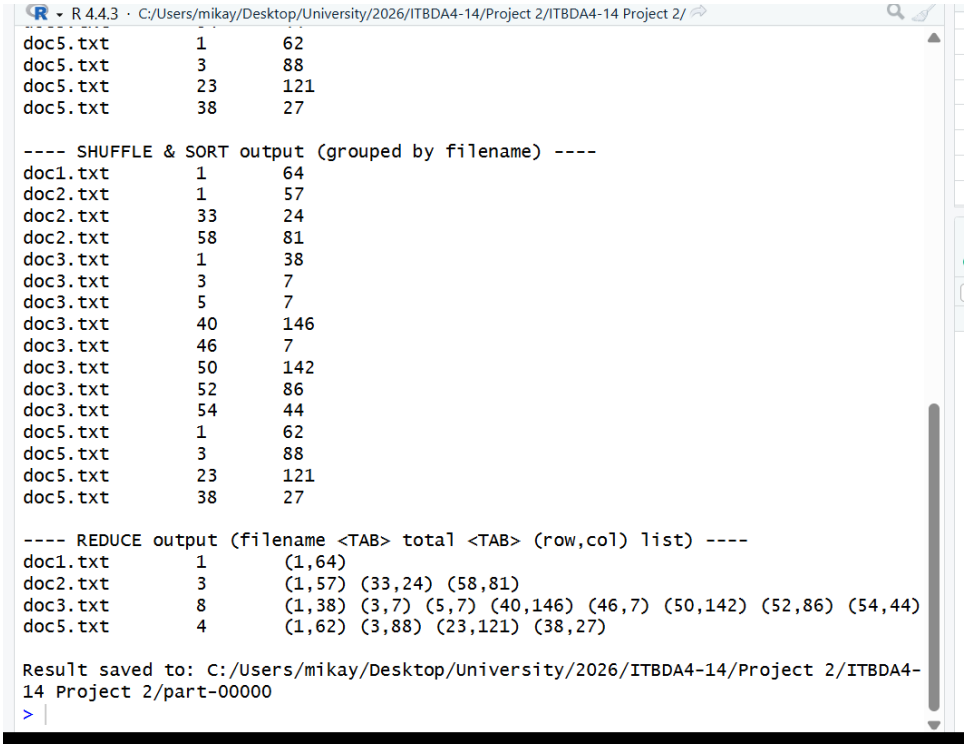


Figure 3: Console output for the keyword "africa": the grouped shuffle-and-sort output and the reducer output, one line per document with the match count and the (row,col) positions. The result is saved to `part-00000`.

Table 2: Final result for the keyword "africa" (reducer output)

Document	Matches	Positions (row,col)
doc1.txt	1	(1,64)
doc2.txt	3	(1,57) (33,24) (58,81)
doc3.txt	8	(1,38) (3,7) (5,7) (40,146) (46,7) (50,142) (52,86) (54,44)
doc5.txt	4	(1,62) (3,88) (23,121) (38,27)

`doc4.txt`, `doc6.txt` and `doc7.txt` do not contain the keyword as a whole word. In total the mappers emitted 16 records, which equals the sum of the match counts reported by the reducer. I also checked the result against an independent regular-expression search over the same files, which I wrote separately from the mapper; both approaches found 16 matches at identical row and column positions.

To show that the keyword is dynamic, I ran the same code a second time with a different keyword, "visa".

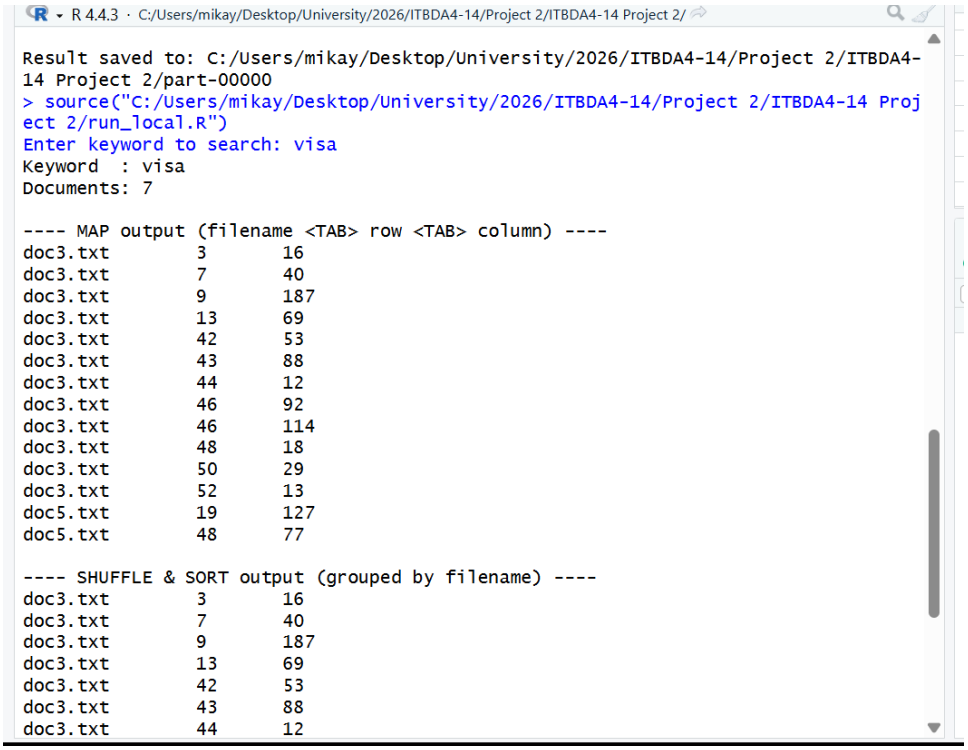


Figure 4: Console output for the second keyword, "visa": the keyword is entered at the prompt and the mapper records are printed.

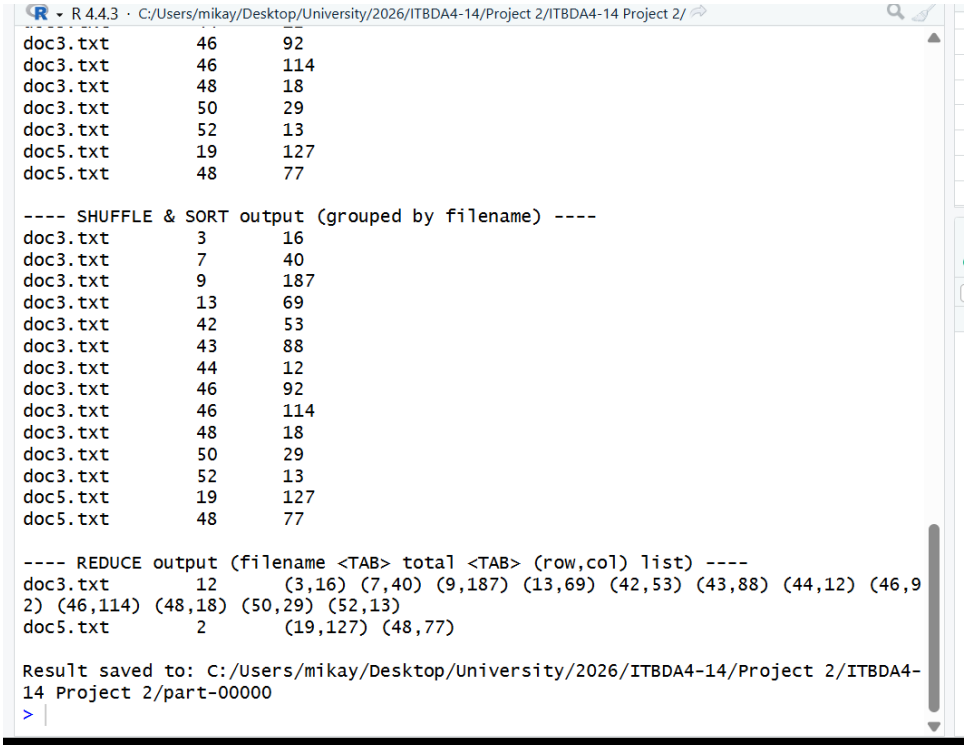


Figure 5: Console output for the keyword "visa": the reducer output, one line per document, saved to `part-00000`.

Table 3: Final result for the keyword "visa"

Document	Matches	Positions (row,col)
doc3.txt	12	(3,16) (7,40) (9,187) (13,69) (42,53) (43,88) (44,12) (46,92) (46,114) (48,18) (50,29) (52,13)
doc5.txt	2	(19,127) (48,77)

Because I saved the first result before the second run, I renamed the output files `part-00000_africa` and `part-00000_visa` so that neither overwrote the other (see the figure below).

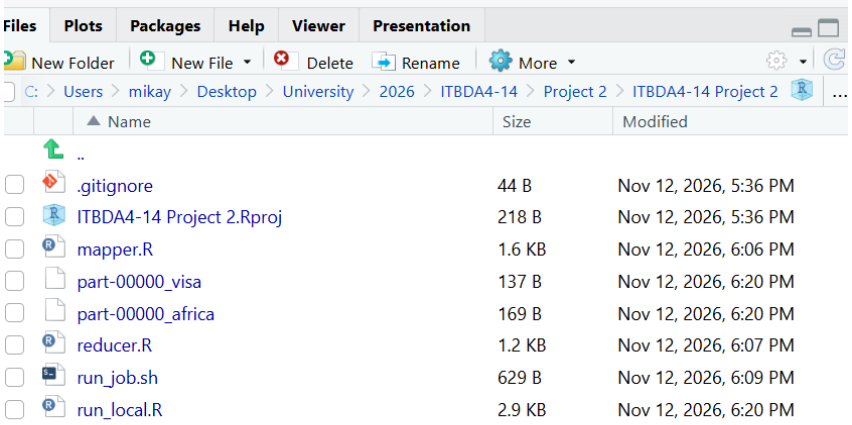


Figure 6: Project folder after both runs: `mapper.R`, `reducer.R`, `run_local.R`, `run_job.sh` and the two saved result files `part-00000_africa` and `part-00000_visa`.

Conclusion

My mapper turns every keyword occurrence into a `(file, row, column)` record, the shuffle groups the records by file, and my reducer produces a sorted list of positions for each document. The solution meets the brief: it is case-insensitive, reports row and column for every match, takes the keyword as a run-time input, and is configured for Hadoop Streaming. The main design choice I made, whole-word matching, avoids counting words such as "African" as the keyword.

Question 2: Topic modelling of news articles with Latent Dirichlet Allocation

In this question I apply topic modelling to a collection of ten South African government news articles dated 8 and 9 July 2024, supplied with the brief as `news.zip` (files `news_1.txt` to `news_10.txt`). I follow the usual text-mining workflow: build a data frame, denoise the text, fit a Latent Dirichlet Allocation (LDA) model, report the top words of every topic with their probabilities, and visualise the result. I wrote the analysis in R, using the `tm` package for text handling (Feinerer, Hornik and Meyer, 2008) and the `topicmodels` package for LDA (Grün and Hornik, 2011). The complete script, `q2_lda.R`, is given in Appendix A.

My script reads the unzipped files directly from the dataset folder, so I do no unzipping in code. It searches sub-folders, which also makes it work when the files sit in a nested folder called `sa_news`.

2.1.) Data frame and pre-processing

What I did: I read each of the ten files as UTF-8 text and collapsed it into a single string. I stored the strings in a data frame with the two columns the brief requires: `file_name` (column 1) and `content` (column 2). I added a third column, `clean_content`, which holds the pre-processed text. The first figure below shows the raw data frame as my script prints it and the second shows the same data frame in the RStudio viewer, with its three columns, ten rows and no missing values.

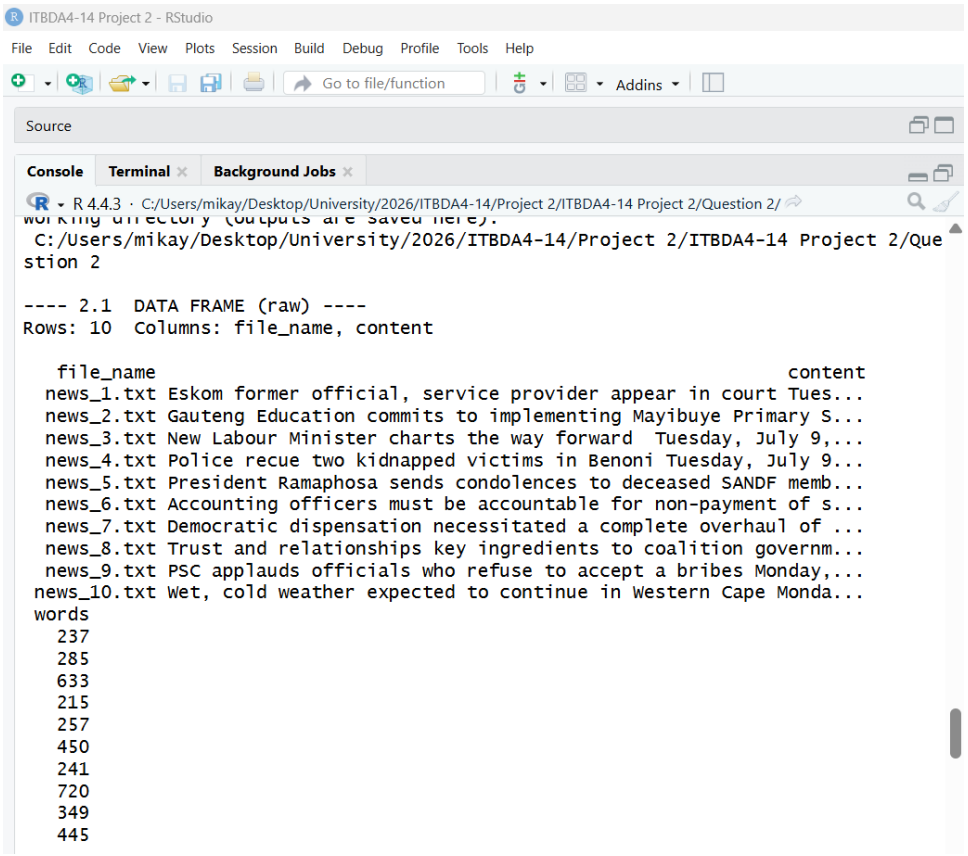


Figure 7: Section 2.1 of the console output: the raw data frame with ten rows and the two required columns, `file_name` and `content`, together with the word count of each article.

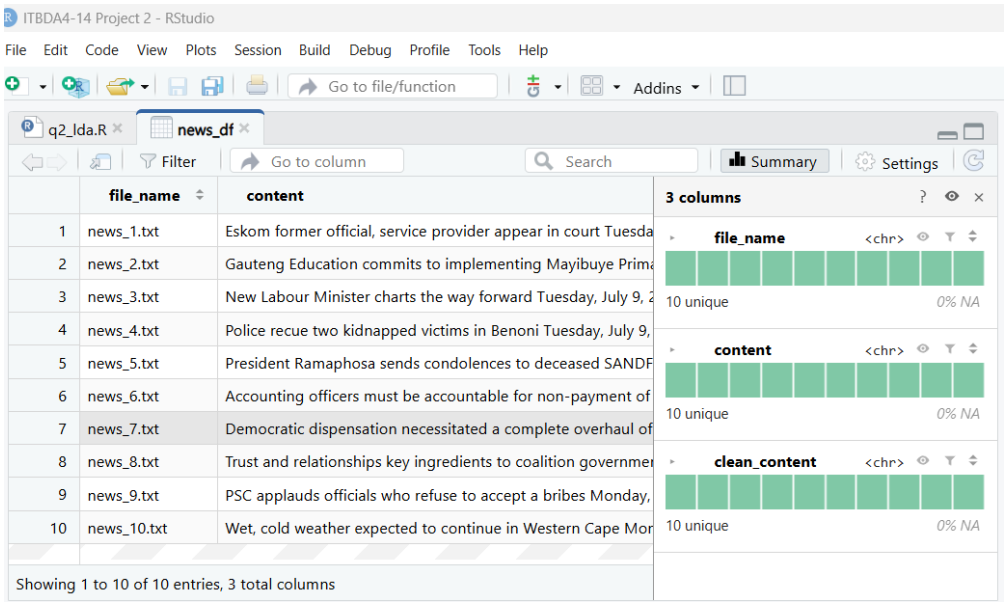


Figure 8: The data frame `news_df` opened in the RStudio viewer: ten unique file names, ten articles and the added `clean_content` column, with 0% missing values.

Data denoising

Raw web text contains elements that carry no topical meaning but would dominate the word counts. I applied the following denoising steps to each article, in this order. They follow common practice in text mining, where the choice of pre-processing steps can change the topics that a model finds (Uysal and Gunal, 2014; Denny and Spirling, 2018):

6. I replaced line breaks and tabs with spaces, and converted typographic (curly) quotation marks and apostrophes to plain characters.
7. I converted the text to lower case so that "Eskom" and "eskom" count as the same word.

8. I removed the date line under each headline (for example "Tuesday, July 9, 2024"), web addresses, e-mail addresses and the news agency's site name, because they appear in every article and say nothing about its topic.
9. I removed possessive endings ("Eskom's" becomes "Eskom"), then replaced every character that is not a letter with a space, which removes punctuation, numbers and symbols.
10. I removed stop words using the standard English list plus corpus-specific words such as "said", "will", weekday and month names, "sanews" and "gov".
11. I removed words shorter than three letters.

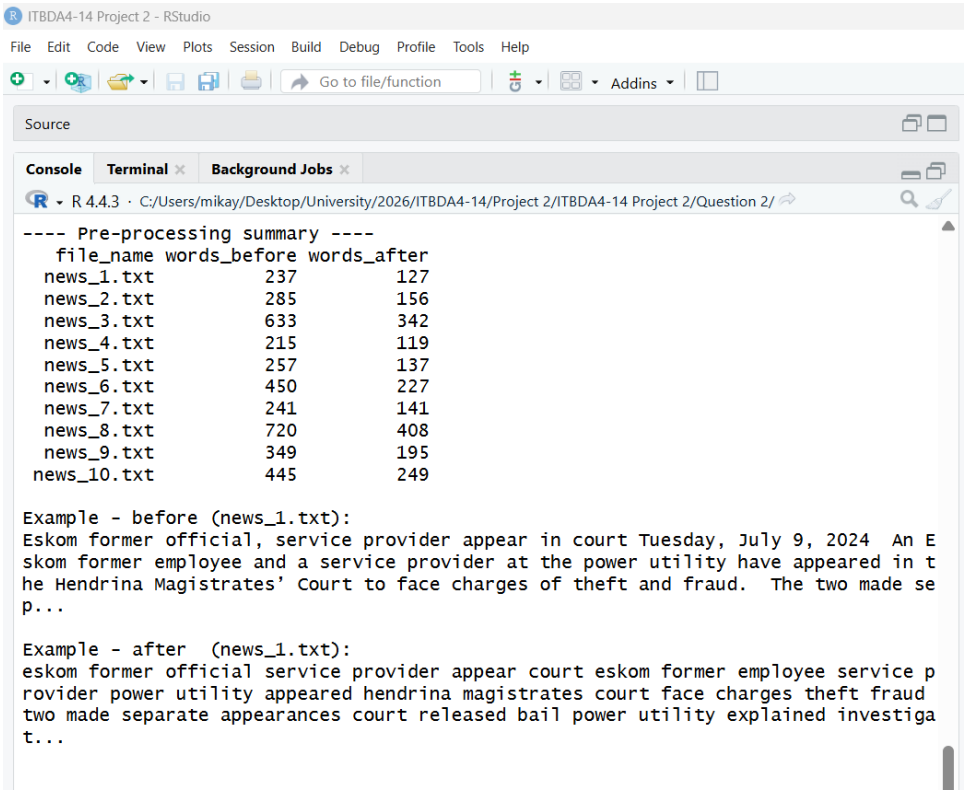


Figure 9: The pre-processing summary: the number of words in each article before and after denoising, with an example article shown before and after cleaning.

Table 4: Words per article before and after pre-processing

File	Words before	Words after	File	Words before	Words after
news_1.txt	237	127	news_6.txt	450	227
news_2.txt	285	156	news_7.txt	241	141
news_3.txt	633	342	news_8.txt	720	408
news_4.txt	215	119	news_9.txt	349	195
news_5.txt	257	137	news_10.txt	445	249

Across the ten articles my pre-processing reduced 3,832 words to 2,101 meaningful words, a reduction of 45%. The example in the figure shows how the headline and first sentence of `news_1.txt` change: the date, capital letters, punctuation and function words are gone, and the content words ("eskom", "employee", "service provider", "power utility", "court") remain.

2.2.) Latent Dirichlet Allocation

What I did: LDA is a probabilistic model that treats every document as a mixture of topics and every topic as a probability distribution over words (Blei, 2012). I fitted the model with Gibbs sampling (Griffiths and Steyvers, 2004), which is available in `topicmodels` (Grün and Hornik, 2011). I turned the cleaned articles into a document-term matrix that keeps only terms appearing in at least two documents, since a word found in a single article cannot link articles together; my matrix has 10 documents and 187 terms. I used a fixed random seed (1234) so that I, or anyone else, can reproduce the result.

Choosing n and k: The brief leaves the number of topics (n) and the number of words reported per topic (k) open. With only ten short articles, a larger n would give unstable, overlapping topics, so I chose **n = 3 topics** and **k = 10 words**. Both are set at the top of my script and can be changed in one place.

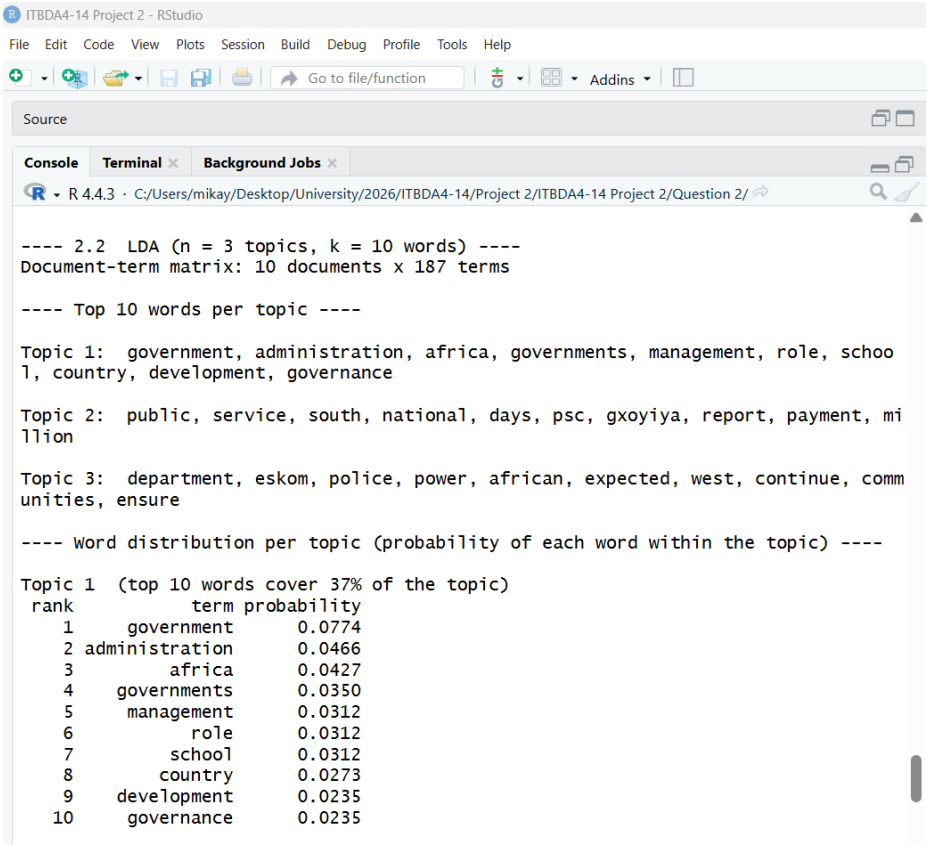


Figure 10: Section 2.2 of the console output: the document-term matrix (10 documents x 187 terms), the top 10 words of each topic and the start of the word-distribution report.

The word distribution of each topic is the probability that each word is generated by that topic. In the table below I list the top ten words for each topic with their probabilities; the final row shows how much of the topic's probability mass the ten words cover.

Table 5: Top 10 words per topic with their probabilities (word distribution per topic)

Rank	Topic 1	P	Topic 2	P	Topic 3	P
1	government	0.0774	public	0.0857	department	0.0868
2	administration	0.0466	service	0.0692	eskom	0.0436
3	africa	0.0427	south	0.0594	police	0.0400
4	governments	0.0350	national	0.0561	power	0.0292
5	management	0.0312	days	0.0430	african	0.0256
6	role	0.0312	psc	0.0397	expected	0.0256
7	school	0.0312	gxoyiya	0.0331	west	0.0256
8	country	0.0273	report	0.0299	continue	0.0220

Rank	Topic 1	P	Topic 2	P	Topic 3	P
9	development	0.0235	payment	0.0233	communities	0.0184
10	governance	0.0235	million	0.0200	ensure	0.0184
Total		**0.370**		**0.459**		**0.335**

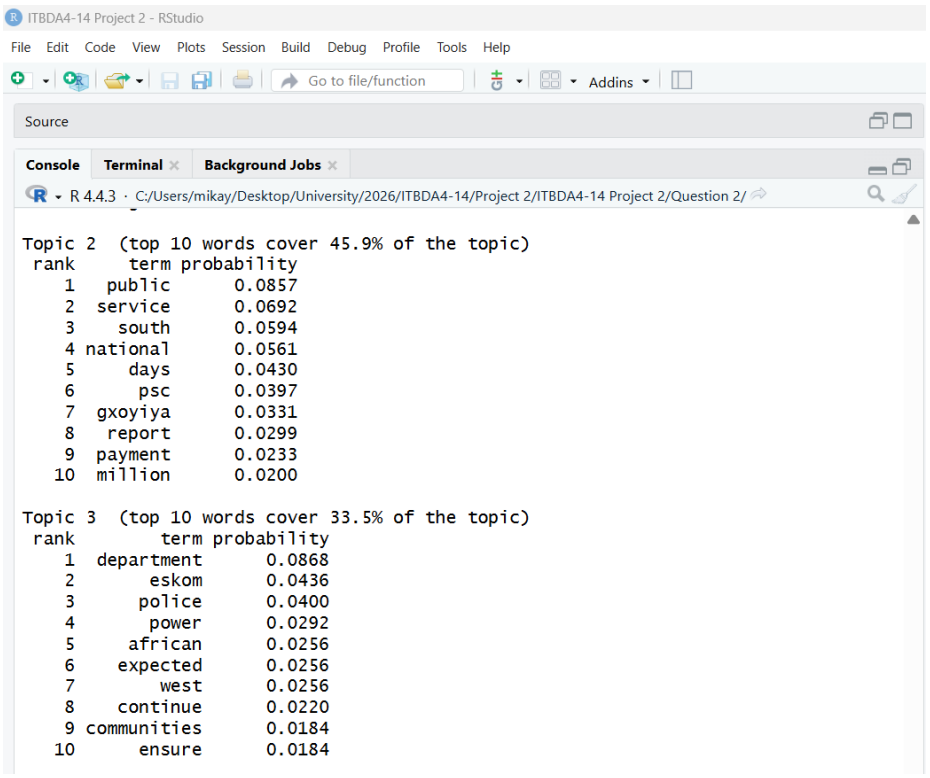


Figure 11: The word distributions of Topic 2 (the top 10 words cover 45.9% of the topic) and Topic 3 (33.5%), as printed by the script.

The topic mix of each document is the probability that the document belongs to each topic. I assign each article to the topic with the highest probability (its dominant topic). Three articles are dominated by Topic 1, three by Topic 2 and four by Topic 3.

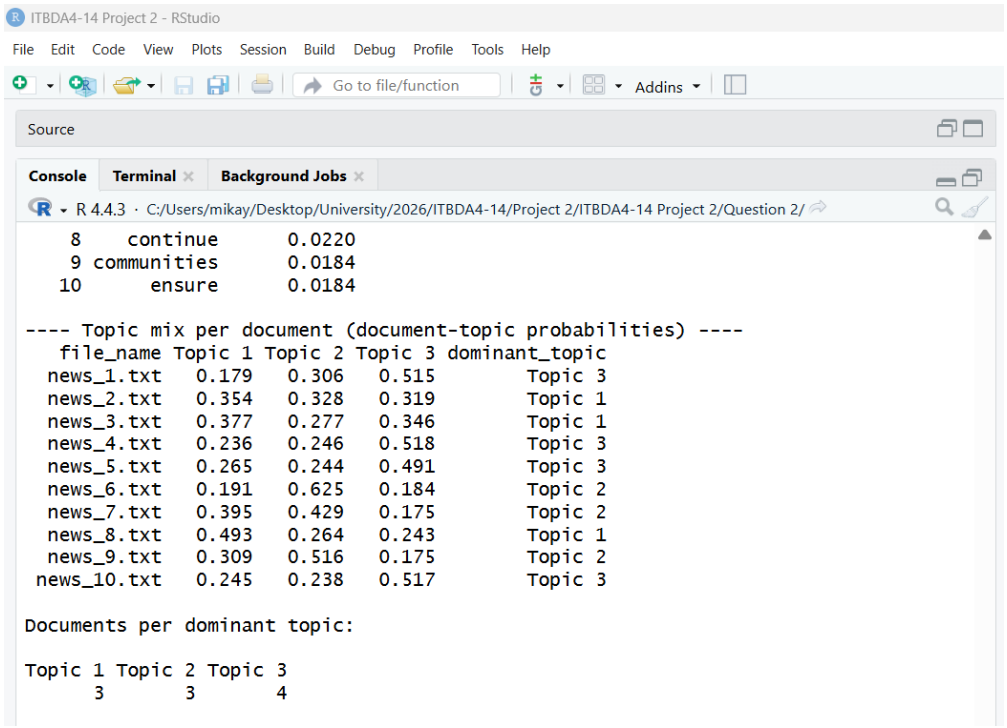


Figure 12: The topic mix per document (document-topic probabilities) and the dominant topic of each article, followed by the number of documents per dominant topic (Topic 1: 3, Topic 2: 3, Topic 3: 4).

Table 6: Topic mix per document

File	Headline (as published)	T1	T2	T3	Dominant
news_1	Eskom former official, service provider appear in court	0.179	0.306	**0.515**	Topic 3
news_2	Gauteng Education commits to implementing Mayibuye Primary School remedial actions	**0.354**	0.328	0.319	Topic 1
news_3	New Labour Minister charts the way forward	**0.377**	0.277	0.346	Topic 1
news_4	Police recue [sic] two kidnapped victims in Benoni	0.236	0.246	**0.518**	Topic 3
news_5	President Ramaphosa sends condolences to deceased SANDF members	0.265	0.244	**0.491**	Topic 3
news_6	Accounting officers must be accountable for non-payment of service providers	0.191	**0.625**	0.184	Topic 2
news_7	Democratic dispensation necessitated a complete overhaul of state bureaucracy	0.395	**0.429**	0.175	Topic 2
news_8	Trust and relationships key ingredients to coalition governments	**0.493**	0.264	0.243	Topic 1
news_9	PSC applauds officials who refuse to accept a bribes [sic]	0.309	**0.516**	0.175	Topic 2
news_10	Wet, cold weather expected to continue in Western Cape	0.245	0.238	**0.517**	Topic 3

Interpretation of the topics

- **Topic 1 – governance and public administration** (government, administration, governments, management, role, governance, development). The articles dominated by this topic are `news_8` (coalition governments), `news_3` (a new Labour Minister) and `news_2` (a school remedial plan), so I read this topic as government structures and how they are run. It is the least secure of my three assignments, because `news_2` and `news_3` are close to an even split between the topics (Table 6).
- **Topic 2 – public service and accountability** (public, service, national, PSC, payment, report, days, million, gxoyiya). The Public Service Commission (PSC) and a named official, Gxoyiya, appear in these articles (the word cloud also shows "commissioner" and "officers"). I see them as concerning non-payment of service providers (`news_6`, 0.63), bribery (`news_9`, 0.52) and reform of the state bureaucracy (`news_7`, 0.43, only just ahead of Topic 1 at 0.40).
- **Topic 3 – mixed operational news (departments, Eskom, police, weather)** (department, Eskom, police, power, expected, communities, continue, west). The four articles dominated by this topic are unrelated stories: the Eskom court case (`news_1`), the rescue of kidnapped victims by police in Benoni (`news_4`), a condolence message on deceased SANDF members (`news_5`) and the weather in the Western Cape (`news_10`). I therefore regard Topic 3 as the least coherent topic. It is held together by the words "department", "Eskom" and "police" and by generic words such as "ensure" and "communities"; "expected" and "continue" come from the headline of the weather article. Its articles do have the highest probabilities for their dominant topic (0.49 to 0.52), but this reflects a shared operational vocabulary more than a single theme, which is why I did not give it a narrower label.

The topics are interpretable but not sharply separated: the highest probability in each document lies between 0.35 and 0.63; `news_2` and `news_3` are close to evenly split between all three topics and `news_7` is split between Topics 1 and 2. I expected this with ten short articles, and I return to it at the end of this question. The country name is also spread across the topics: "africa" is a top word of Topic 1, "south" of Topic 2 and "african" of Topic 3, so it does not help me to separate them.

2.3.) Visualisation of the top keywords per topic

I used three visualisation methods. I produced all figures with `ggplot2` (Wickham, 2016) or base graphics, and my script saves them as PNG files.

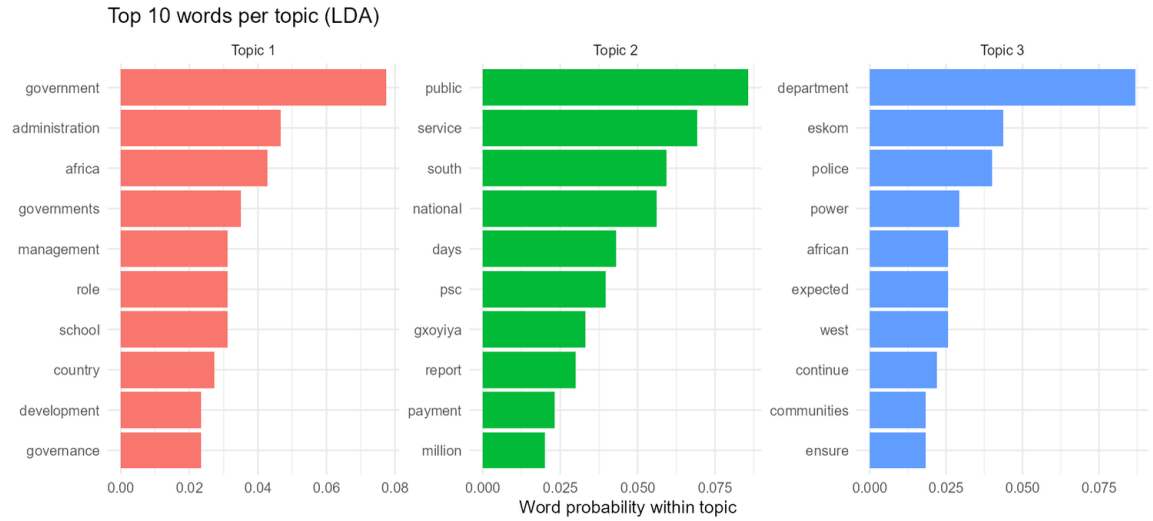


Figure 13: Method 1 – bar charts of the top 10 words per topic. The bar length is the probability of the word within its topic, so the charts show both which words define each topic and how strongly.

In the bar charts I can see that Topic 3 is led by a single dominant word ("department" at 0.087, about twice the next word, "eskom" at 0.044). Topic 1 is led by "government" (0.077) with a smaller gap to "administration" (0.047), whereas Topic 2 spreads its probability more evenly over "public" (0.086), "service" (0.069), "south" (0.059) and "national" (0.056).

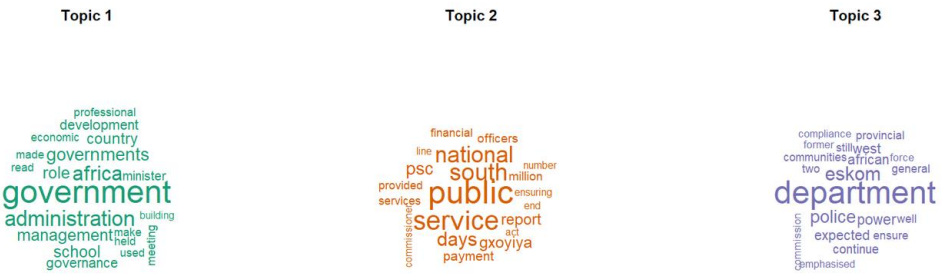


Figure 14: Method 2 – word clouds of the top 20 words of each topic. The size of a word is proportional to its probability within the topic.

The word clouds, which show the top 20 words of each topic, give me an immediate impression of each topic's vocabulary: administration language in Topic 1 (with "minister", "economic" and "professional" among the smaller words), public-service and commission language in Topic 2 ("public", "service", "national", "psc", plus "commissioner", "officers" and "financial") and department, utility and policing language in Topic 3 ("department", "eskom", "police", "power"). The smaller words of Topic 3 ("commission", "force", "provincial") reflect the mix of stories I described above.

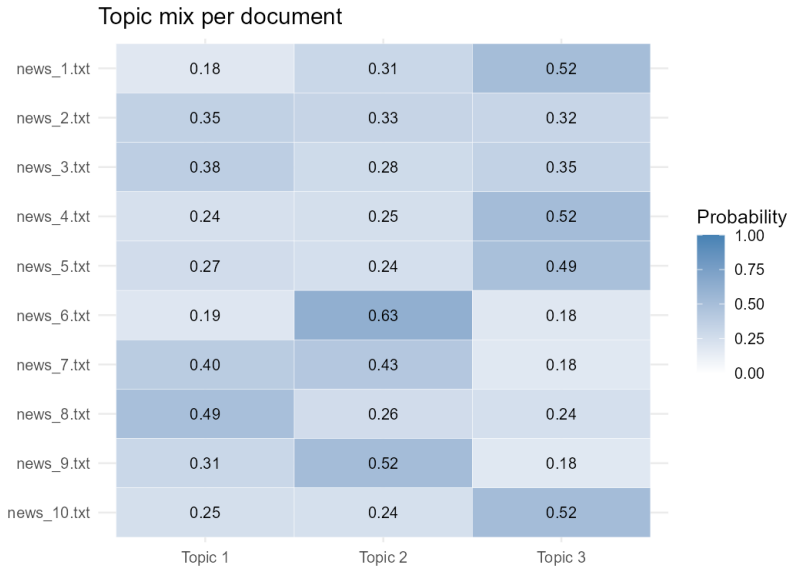


Figure 15: Method 3 – heatmap of the topic mix per document. Darker cells mean a higher probability that the article belongs to the topic; the number in each cell is that probability.

The heatmap shows which articles define each topic: `news_6` (0.63) is the clearest Topic 2 article, followed by `news_9` (0.52); Topic 3 is anchored by `news_1`, `news_4`, `news_5` and `news_10` (0.49 to 0.52); and the strongest Topic 1 article is `news_8` (0.49). Articles such as `news_2` and `news_3` have pale, similar colours across the row, which tells me that they mix several topics.

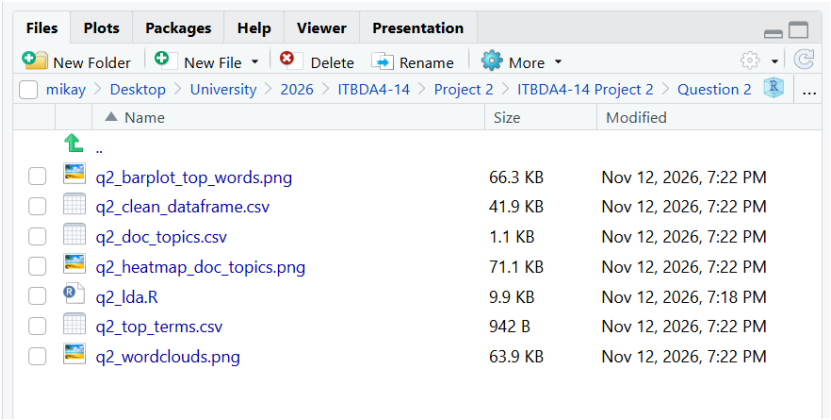


Figure 16: The files created by the script in the `Question 2` folder: the cleaned data frame, the top-term and document-topic tables, the three plots and the script itself.

Conclusion and limitations

My pipeline turned ten raw articles into three topics: governance and administration, public service and accountability, and a mixed group of operational news (Eskom, police, the defence force and the weather). I would read the results with care. Ten short articles give little evidence for each topic, so the three topics overlap and the document-topic probabilities are fairly diffuse; the third topic in particular groups unrelated stories. My choice of three topics is a judgement, and a different seed or number of topics would give somewhat different words. I would need a larger corpus and a measure such as topic coherence to choose n more rigorously. Nevertheless the model recovers the main themes of the collection, and I think the three visualisations present them clearly.

Question 3: Building a WebText API - web text mining proof of concept

The scenario asks for a proof of concept for an analytics company that wants a web-search text-mining API: a program that reads data from a web link and generates insights from its content. Natural language processing (NLP) is advancing quickly (Hirschberg and Manning, 2015), and my proof of concept follows the typical sequence of such a service: collect a page legitimately, clean its text, and summarise it. I wrote the code in R, and the complete script, `q3_webtext.R`, is given in Appendix B.

3.1.) Web scraping and formation of an R data structure

Choice of website: I chose the English Wikipedia article *Natural language processing* (https://en.wikipedia.org/wiki/Natural_language_processing) for three reasons. First, Wikipedia's text is released under the Creative Commons Attribution-ShareAlike licence (CC BY-SA 4.0), which explicitly permits re-use with attribution, so using it is clearly within fair use. Second, the topic matches the scenario, so my proof of concept summarises a page about the very field that the API belongs to. Third, Wikipedia is a widely studied source of text for research (Mesgari et al., 2015).

Compliance with fair use of web scraping: Legal and ethical guidance on web scraping stresses respect for access rules, restraint in the number of requests and recognition of the source (Krotov, Johnson and Silva, 2020; Krotov and Johnson, 2023; Landers et al., 2016). I wrote my scraper to follow these principles:

Table 7: How the scraping complies with fair use

Principle	How I met it
Public content only	The page is freely available; I use no login, paywall or private data.
Respect for `robots.txt`	My script downloads https://en.wikipedia.org/robots.txt, parses the rules for all bots and checks that the article path is allowed before it requests the page. The check returned: "Automated access to this page allowed: TRUE".

Principle	How I met it
Honest identification	I send a descriptive User-Agent that names the project, states that it is student research and gives a contact e-mail address, as Wikimedia's policy requires. I type the e-mail address in when the script starts and do not store it in the code.
Minimal load	I request the page once and save it locally as q3_raw_page.html; all later analysis uses the saved copy, so I never request the page twice.
Attribution and licence	I store the source, licence (CC BY-SA 4.0), access date (12 November 2026) and the page's last-edited date (1 November 2026, 04:08 UTC) with the data and quote them in the figure captions.

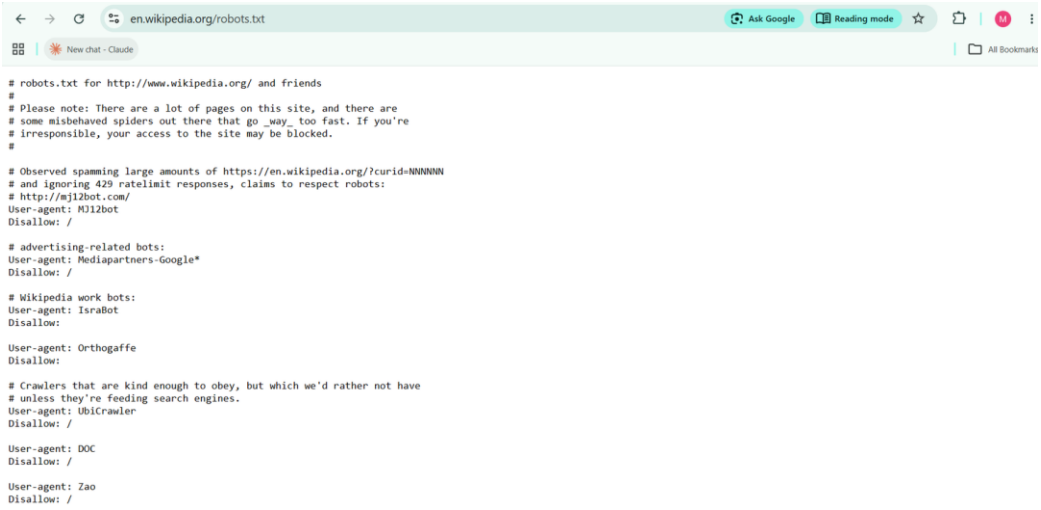


Figure 17: The `robots.txt` file of Wikipedia opened in a web browser, used as evidence for the access check.

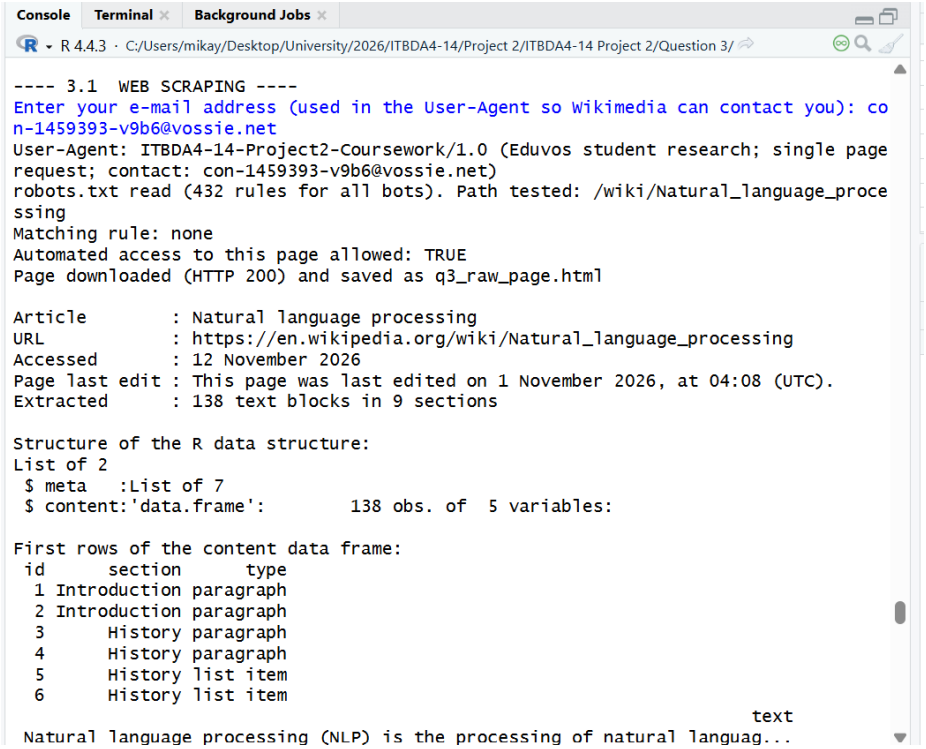


Figure 18: Section 3.1 of the console output: the User-Agent, the `robots.txt` check, the successful download (HTTP 200), the article metadata and the structure of the R data structure.

The R data structure: I parse the HTML page with `rvest` into an R list with two parts. `meta` holds seven fields (source, title, URL, access date, last-edited date, licence and the User-Agent used). `content` is a data frame with 138 rows and five variables: an identifier, the section heading, the sub-section, the type of block (paragraph or list item) and the text. I removed navigation menus, the infobox, tables, figures, reference superscripts and edit links during parsing, so only the readable article text remains. The article is split into nine sections; the figure below shows how many text blocks each contains.

```
                                                                        text
Natural language processing (NLP) is the processing of natural languag...
Major processing tasks in an NLP system include: speech recognition, t...
Natural language processing has its roots in the 1950s. Already in 195...
The premise of symbolic NLP is often illustrated using John Searle's C...
1950s: The Georgetown experiment in 1954 involved fully automatic tran...
1960s: Some notably successful natural language processing systems dev...

Text blocks per section:

                                       Introduction
                                                  2
                                            History
                                                 11
 Approaches: Symbolic, statistical, neural networks
                                                 14
                                   Common NLP tasks
                                                  4
General tendencies and (possible) future directions
                                                 10
                                           See also
                                                 23
                                         References
                                                 64
                                     Further reading
                                                  9
                                      External links
                                                  1
```

Figure 19: The number of text blocks in each section of the scraped article: 138 blocks in nine sections, of which `References` (64) and `See also` (23) are the largest.

3.2.) Text pre-processing

What I did: The scraped text still contains material that is not pertinent to the article's content. I applied pre-processing in five steps, and I recorded the number of words after each step so that the effect of every step can be seen. Cleaning decisions of this kind can change the result of a text analysis, so I made each step explicit and reproducible (Uysal and Gunal, 2014; Denny and Spirling, 2018).

12. **Non-pertinent sections removed.** `References`, `See also`, `Further reading` and `External links` list sources and links rather than content; removing them took out 1,916 words.
13. **Markup noise removed.** I removed or standardised citation markers such as `[12]`, web addresses, HTML entities and typographic symbols. Replacing en and em dashes by spaces splits some words in two, so the count rises slightly at this step.
14. **Case and symbols.** I converted the text to lower case and removed punctuation, numbers and symbols. Hyphenated words are split in two, which is why the count rises slightly at this step.
15. **Stop words and short words.** I removed English stop words, a list of corpus-specific filler words (such as "also", "used" and "may") and words shorter than three letters, which took out 984 words.
16. **Stemming with stem completion.** I reduced words to their stems with the Snowball English stemmer, an improved version of the Porter algorithm (Porter, 1980), so that "language" and "languages" are counted together, and I mapped each stem back to its most common real word so that the charts stay readable.

Table 8: Number of words after each pre-processing step

Step	Words	Change
0. Raw extracted text (all sections)	4,135	-
1. Remove non-pertinent sections (references, see also, ...)	2,219	-1,916

Step	Words	Change
2. Remove citation markers, URLs, entities and odd symbols	2,229	+10
3. Lower case, remove punctuation and numbers	2,266	+37
4. Remove stop words and words shorter than 3 letters	1,282	-984

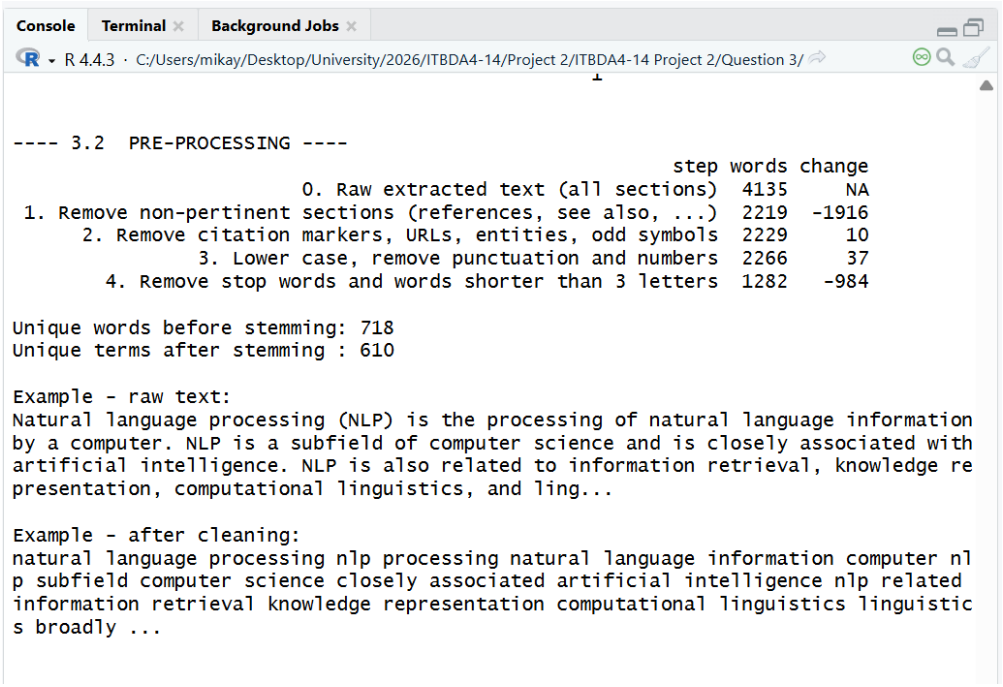


Figure 20: Section 3.2 of the console output: the words remaining after each step, the vocabulary before stemming (718 unique words) and after stemming (610 unique terms), and an example paragraph before and after cleaning.

The raw article contained 4,135 words. After cleaning, 1,282 meaningful tokens remained (31% of the original), of which 610 are unique terms after stemming. The example paragraph shows the effect: "Natural language processing (NLP) is the processing of natural language information by a computer" becomes "natural language processing nlp processing natural language information computer", which keeps the content words and drops the function words and punctuation.

3.3.) Text summarisation and visualisation

I applied three summarisation techniques to the cleaned data. The first is a **keyword summary** using term frequency and TF-IDF (Salton and Buckley, 1988; Spärck Jones, 1972). The second and third are **extractive summaries**, which select the most important sentences of the article rather than writing new text (Nenkova and McKeown, 2012): one scores sentences by word frequency (Luhn, 1958) and the other uses the graph-based TextRank method, in which sentences are ranked with the PageRank algorithm (Brin and Page, 1998), an approach closely related to LexRank (Erkan and Radev, 2004). I produced four fully labelled figures; every figure has a title, a subtitle where useful, axis labels and a source caption.

Technique 1: term frequency and TF-IDF keywords

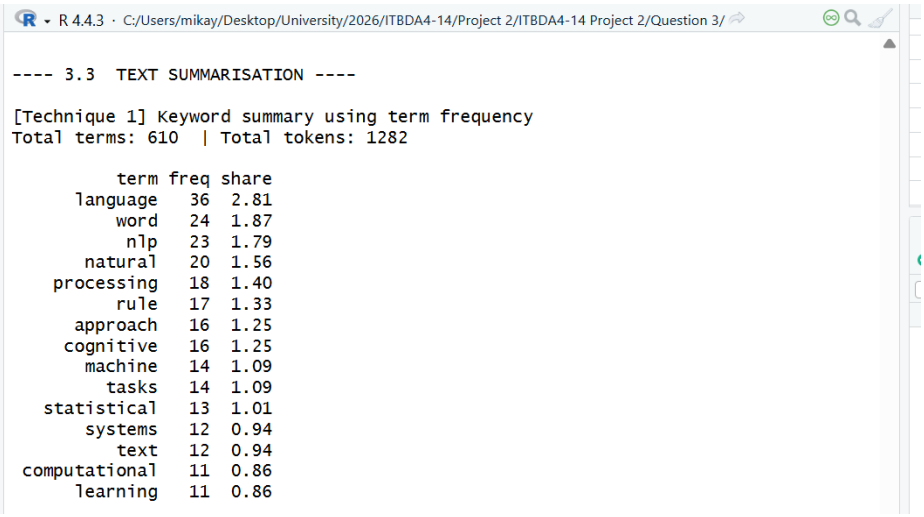


Figure 21: Section 3.3 of the console output: the keyword summary of the article by term frequency, with the total of 610 terms and 1,282 tokens.

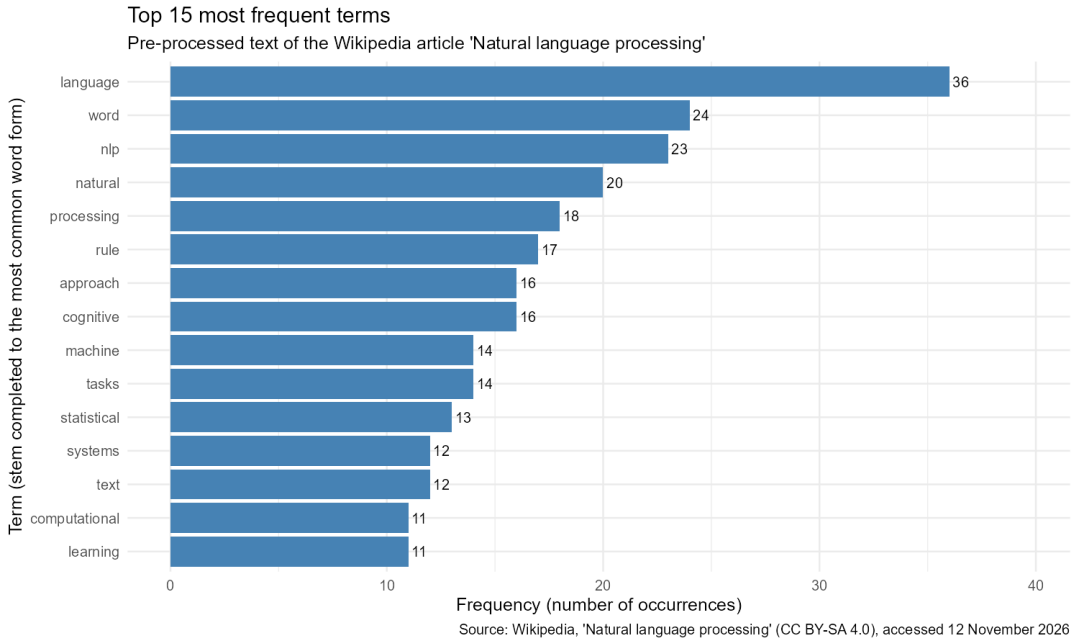


Figure 22: Bar chart of the 15 most frequent terms in the pre-processed article. "language" is the most frequent term (36 occurrences), followed by "word" (24), "nlp" (23), "natural" (20) and "processing" (18).

The frequency chart confirms to me that the article is about language processing. The top five terms are the field's own vocabulary ("language", "word", "nlp", "natural" and "processing"). The next terms ("rule", "approach", "machine", "statistical", "systems" and "learning") reflect the article's main storyline: the move from rule-based to statistical and machine-learning approaches. The term "cognitive" (16) is frequent because the section on future directions discusses cognition and NLP at length; the TF-IDF analysis below confirms that it is concentrated in that section.

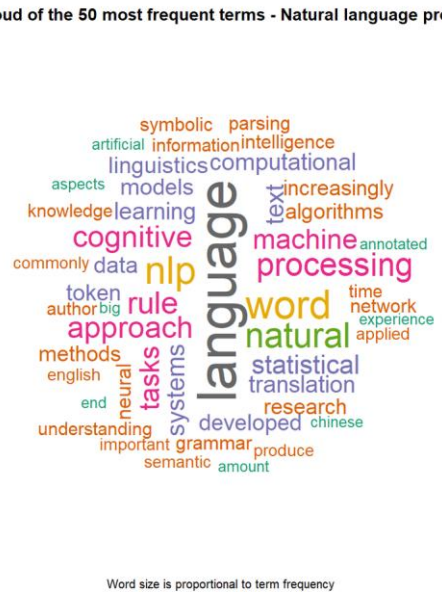


Figure 23: Word cloud of the 50 most frequent terms of the article. The size of a word is proportional to its frequency.

Term frequency alone favours words that are common everywhere in the article. To find the words that **characterise** each section, I treated every section as a document and weighted the terms with TF-IDF, which rewards words that are frequent in one section but rare in the others.

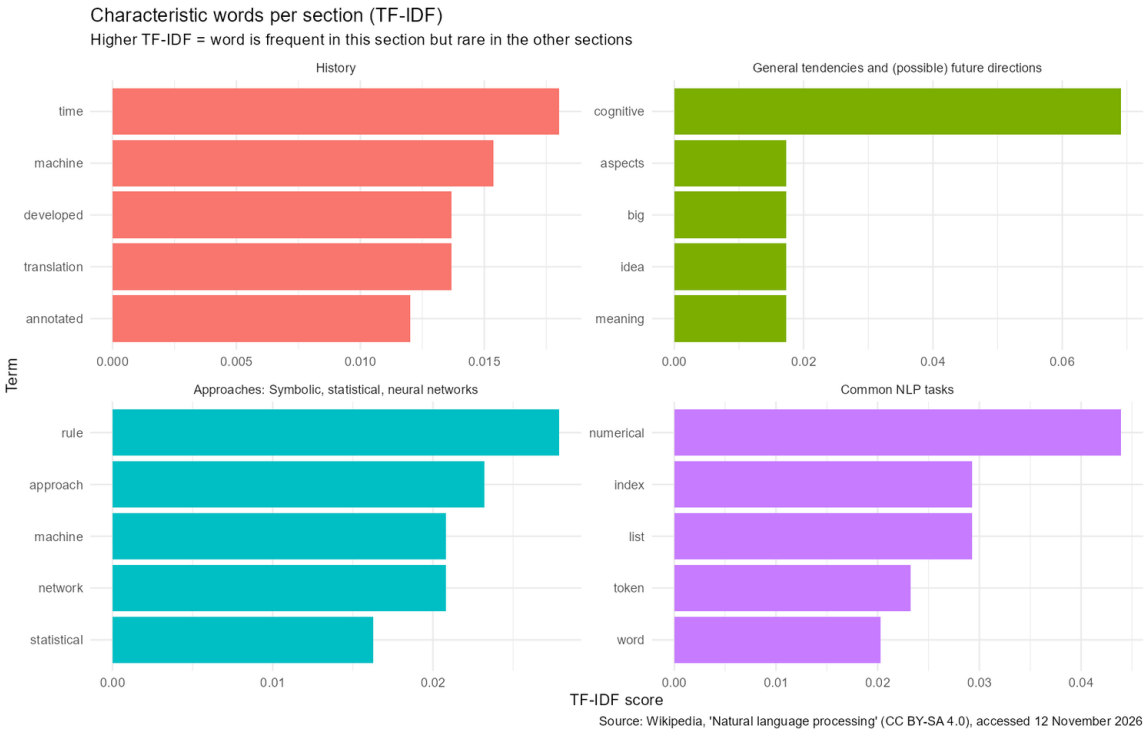


Figure 24: TF-IDF keywords for the four largest sections. Higher TF-IDF means that the word is frequent in this section but rare in the other sections.

TF-IDF separates the sections reasonably well. **History** is characterised by *time*, *machine*, *developed*, *translation* and *annotated*; **Approaches** by *rule*, *approach*, *machine*, *network* and *statistical*; **Common NLP tasks** by task vocabulary such as *numerical*, *index*, *list* and *token*; and **General tendencies and (possible) future directions** by *cognitive*, *aspects*, *big*, *idea* and *meaning*. The separation is not perfect: "machine" appears in the lists of both History and Approaches, and four of the five words for General tendencies and (possible) future directions have the same small score (about 0.017), which suggests to me that they occur only a few times, so only "cognitive" (0.069) stands out clearly. With four short sections I read the lower-ranked words with caution.

Techniques 2 and 3: extractive summaries

I scored the 39 paragraph sentences of the article (those with at least six words after cleaning) in two ways and kept the five best sentences of each method in their original order. The first method scores a sentence by the sum of the normalised frequencies of its words divided by the square root of its length. For TextRank I build a graph in which every sentence is a node, join sentences by the TF-IDF cosine similarity of their words, and rank the nodes with PageRank, so a sentence scores highly if it is similar to many other important sentences.

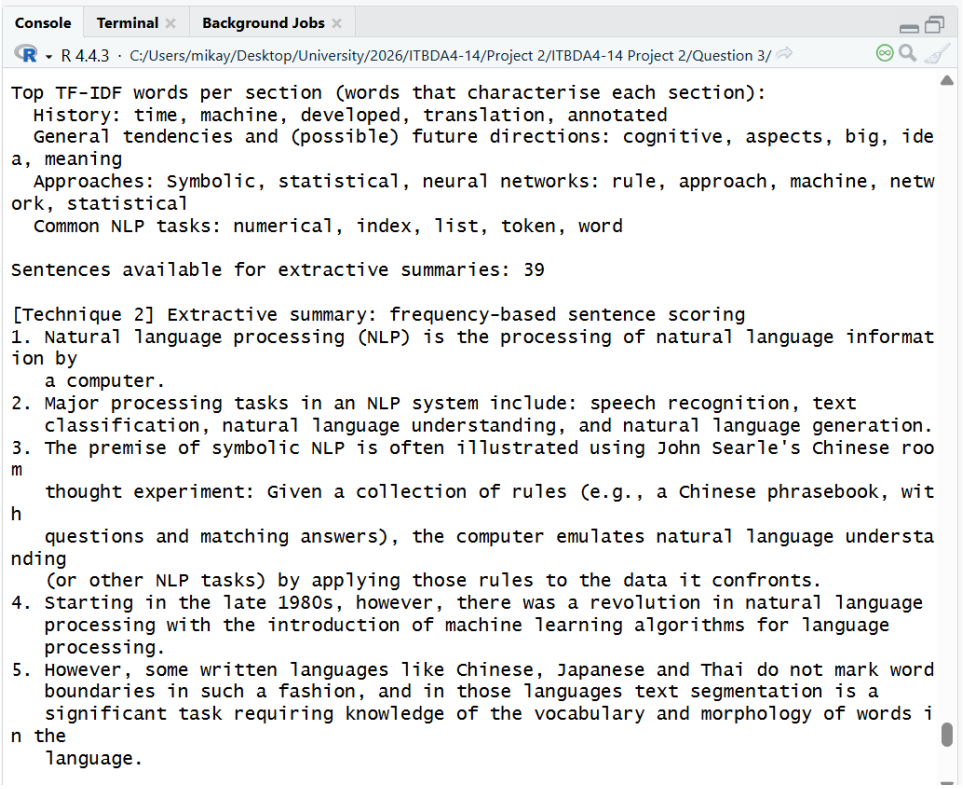


Figure 25: Console output showing the TF-IDF words per section, the number of sentences available (39) and the five sentences chosen by the frequency-based summary (Technique 2).

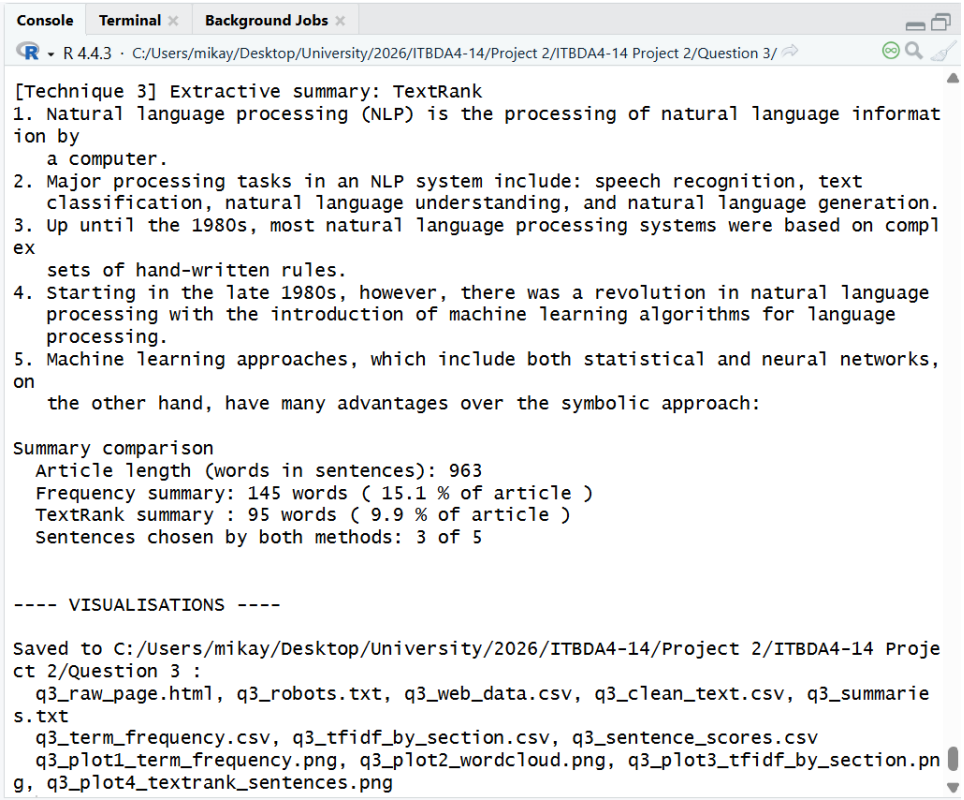


Figure 26: Console output showing the five sentences chosen by TextRank (Technique 3) and the comparison of the two summaries: the frequency summary has 145 words (15.1% of the article), the TextRank summary 95 words (9.9%), and 3 of 5 sentences are chosen by both methods.

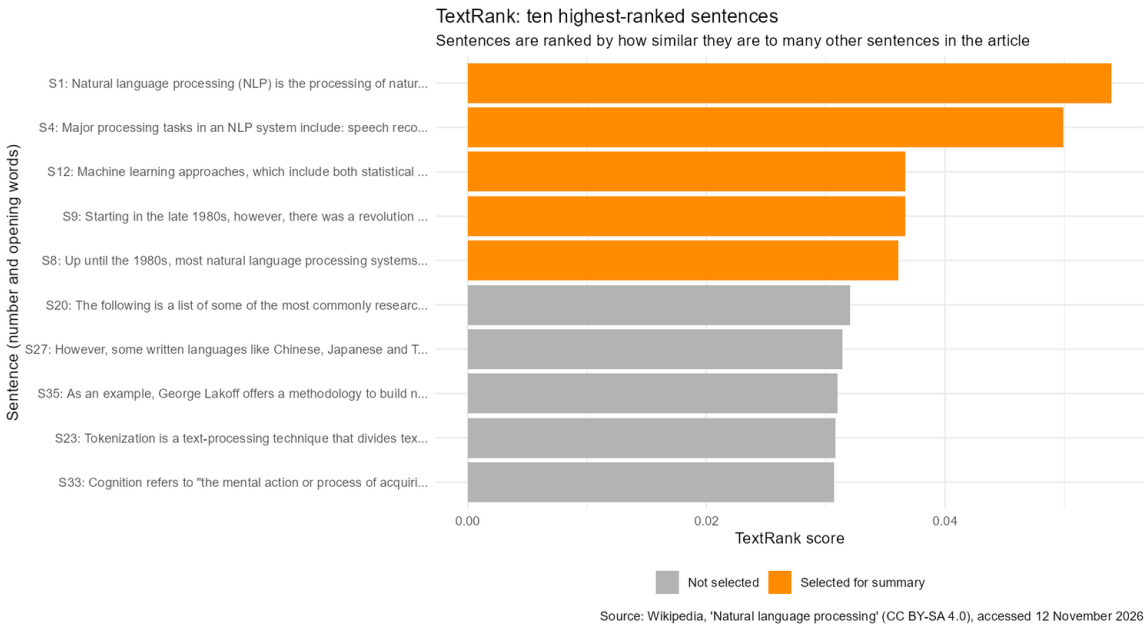


Figure 27: TextRank scores of the ten highest-ranked sentences. Orange bars are the five sentences selected for the summary.

Table 9: The two five-sentence summaries of the article (sentences from Wikipedia, CC BY-SA 4.0)

Frequency-based summary (Technique 2)	**TextRank summary (Technique 3)**
1. Natural language processing (NLP) is the processing of natural language information by a computer.	1. Natural language processing (NLP) is the processing of natural language information by a computer.

Frequency-based summary (Technique 2)	TextRank summary (Technique 3)
2. Major processing tasks in an NLP system include: speech recognition, text classification, natural language understanding, and natural language generation.	2. Major processing tasks in an NLP system include: speech recognition, text classification, natural language understanding, and natural language generation.
3. The premise of symbolic NLP is often illustrated using John Searle's Chinese room thought experiment: Given a collection of rules (e.g., a Chinese phrasebook, with questions and matching answers), the computer emulates natural language understanding (or other NLP tasks) by applying those rules to the data it confronts.	3. Up until the 1980s, most natural language processing systems were based on complex sets of hand-written rules.
4. Starting in the late 1980s, however, there was a revolution in natural language processing with the introduction of machine learning algorithms for language processing.	4. Starting in the late 1980s, however, there was a revolution in natural language processing with the introduction of machine learning algorithms for language processing.
5. However, some written languages like Chinese, Japanese and Thai do not mark word boundaries in such a fashion, and in those languages text segmentation is a significant task requiring knowledge of the vocabulary and morphology of words in the language.	5. Machine learning approaches, which include both statistical and neural networks, on the other hand, have many advantages over the symbolic approach:

Findings

- **What the article is about.** My keyword summary shows that the article concerns the processing of natural language by computers and the shift from rule-based (symbolic) systems to statistical and neural machine-learning systems. The two extractive summaries tell the same story: a definition, the main tasks, the symbolic era and the revolution brought by machine learning.
- **The two summaries agree on the core.** Both choose the definition (S1), the list of major tasks (S4) and the late-1980s shift to machine learning (S9). They differ in the remaining two sentences. The frequency method prefers long, detailed sentences that are dense with the most common words (the Chinese room experiment and the text-segmentation sentence, S27). TextRank scores a sentence by how similar it is to many other sentences, and in my results it selects the sentence on hand-written rules (S8) and the sentence on the advantages of machine learning (S12), which gives a shorter summary (95 against 145 words).
- **Compression.** My TextRank summary is 9.9% and my frequency summary 15.1% of the 963 words in the article's sentences, so a reader gets the main ideas in about five sentences.
- **Section structure.** TF-IDF shows me that the article is organised around history, approaches, tasks and future directions, each with its own vocabulary, which is the kind of insight a web-text API could return for any link.

Limitations

Wikipedia articles change over time, so my results apply to the version I saved on 12 November 2026. Extractive summaries copy whole sentences and cannot paraphrase, TextRank in particular can select a sentence that ends mid-thought (my fifth TextRank sentence ends with a colon), and stemming can merge words with different meanings. A production API would also need rate limiting, error handling for pages that block scrapers and more robust parsing for pages that are not Wikipedia articles.

Question 4: ETL manipulation using R and PostgreSQL

Extract, Transform and Load (ETL) is the methodology of converting raw, unstructured data into structured datasets suitable for analysis (Vassiliadis, 2009). In this question I demonstrate a complete ETL workflow. I **extract** data from an unstructured, web-hosted source, **transform** it by cleaning and imputing missing values, and **load** it into a relational database for analytical reporting.

Use case: I built a profile dataset of South Africa's public universities, for example to compare institution types, sizes and the spread of enrolment across provinces. This suits the brief because each university has an article with free text and an information box whose values are written in inconsistent ways, and for several universities some facts are missing. My R scripts are `q4_scrape_clean.R` (4.1 and 4.2) and `q4_load_postgres.R` (4.3); both are given in Appendices C and D.

4.1.) Web scraping and regular-expression analysis

Source and compliance: The articles come from English Wikipedia (CC BY-SA 4.0). I followed the same scraping principles as in Question 3 (Krotov, Johnson and Silva, 2020): my script reads `robots.txt` and checks every article path (26 of 26 pages were allowed), identifies itself with a descriptive `User-Agent` and contact e-mail, waits 1.5 seconds between requests, makes one request per page and saves each page in the folder `q4_raw_pages`. When I re-run the script it uses the saved copies, so no page is requested twice.

Records: I scraped one article for each of the 26 public universities, which is above the 20 records required. Each article is an unstructured record: a free-text lead paragraph and an information box (the infobox) whose values are strings such as "35,000+ (2026)[2]" rather than numbers.

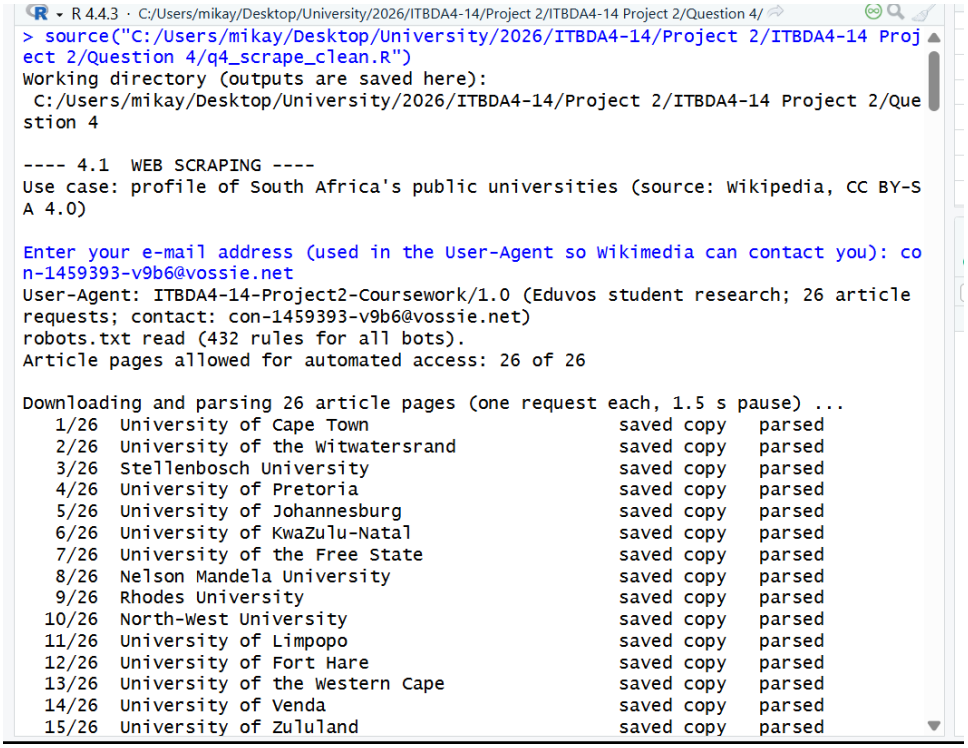


Figure 28: Section 4.1 of the console output: the use case, the User-Agent, the `robots.txt` check (26 of 26 article pages allowed) and the start of the download list.

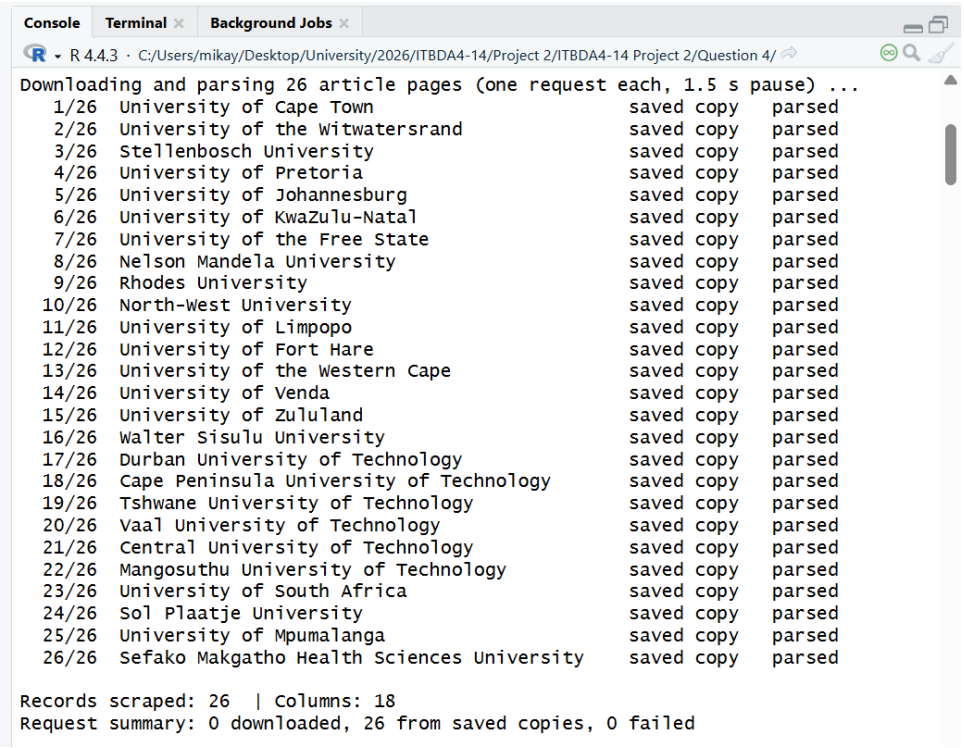


Figure 29: All 26 pages were parsed (all from saved copies, none failed): 26 records and 18 columns were created.

The data frame: textual and numerical attributes

I parsed each article into a record with the following attributes. The numerical attributes include both existing values (taken from the page) and inferred values (calculated from the page).

Table 10: Attributes of the raw data frame

Type	Attributes
Textual	`university` (name), `source_url`, `lead_text` (first two paragraphs), `established_raw`, `type_raw`, `students_raw`, `undergraduates_raw`, `postgraduates_raw`, `staff_raw`, `location_raw` (infobox strings)
Numerical – existing	`founded_first_pass`, `students_first_pass` and `staff_first_pass`: the first number found in each infobox string with a deliberately simple regular expression
Numerical – inferred	`article_words` (words in the article), `n_citations` (number of citation markers) and `n_sections` (number of section headings)
Identifiers	`id` and `scraped_on` (date of the saved page)

```
Structure of the raw data frame:
'data.frame':   26 obs. of  18 variables:
 $ id                : int  1 2 3 4 5 ...
 $ university        : chr  "University of Cape Town" "University of the Witwaters
rand" ...
 $ source_url        : chr  "https://en.wikipedia.org/wiki/University_of_Cape_Tow
n" "https://en.wikipedia.org/wiki/University_of_the_Witwatersrand" ...
 $ lead_text         : chr  "The University of Cape Town (UCT) (Afrikaans: Univers
iteit van Kaapstad, Xhosa: iYunivesithi yaseKapa) is a pub"| __truncated__ "The Uni
versity of the Witwatersrand, Johannesburg (/vətˈvɑːtəsrɑːnt/), commonly known as W
its University or Wit"| __truncated__ ...
 $ established_raw   : chr  "1 October 1829; 197 years ago (1829-10-01)" "1 March
1922; 104 years ago (1922-03-01)[1]" ...
 $ type_raw          : chr  "Public" "Public university" ...
 $ students_raw      : chr  "28,233[4]" "40,259[3]" ...
 $ undergraduates_raw : chr  "16,530[4]" "25,352[3]" ...
 $ postgraduates_raw  : chr  "11,193[4]" "14,025[3]" ...
 $ staff_raw         : chr  "1,176[3]" "1,112 (Full-time, Permanent)[3]" ...
 $ location_raw      : chr  "Cape Town\n,\nWestern Cape\n,\nSouth Africa\n\n33°57′
27″S 18°27′38″E / 33.95750°S 18.46056°E / -33.95750; 18.46056" "Johannesburg\n,\nGa
uteng\n,\nSouth Africa\n\n26°11′27″S 28°1′49″E / 26.19083°S 28.03028°E / -26.19083;
28.03028" ...
 $ founded_first_pass : num  1829 1922 ...
 $ students_first_pass: num  28233 40259 ...
 $ staff_first_pass   : num  1176 1112 ...
 $ article_words      : int  4364 3929 2011 5743 1980 ...
 $ n_citations        : int  152 134 85 217 121 ...
 $ n_sections         : int  16 11 12 14 11 ...
 $ scraped_on         : chr  "2026-11-12" "2026-11-12" ...
```

Figure 30: Structure of the raw data frame (`str`): 26 observations and 18 variables. The infobox columns are text strings such as `"28,233[4]"` and the location column contains GPS coordinates.

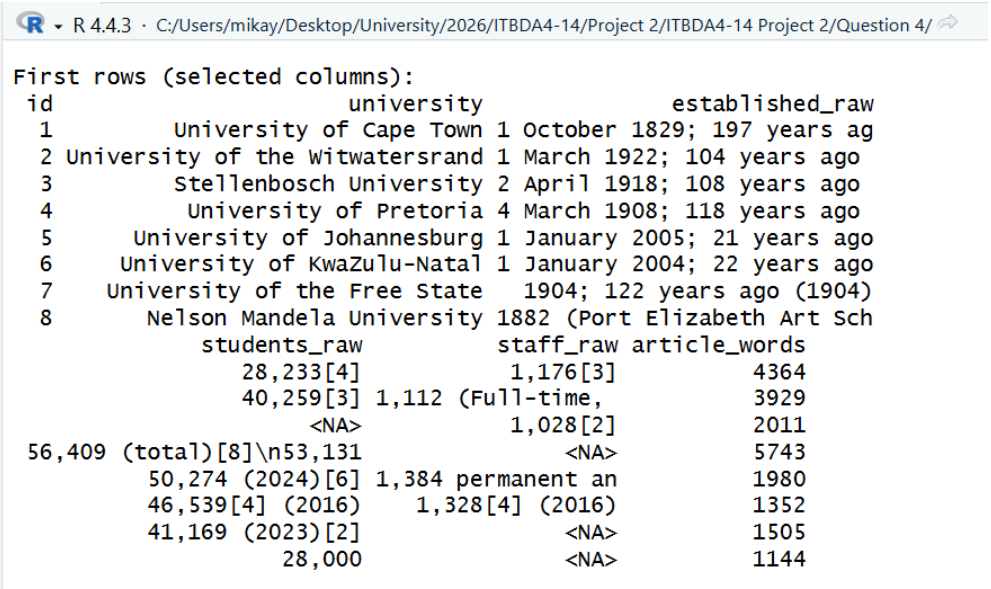


Figure 31: The first records of the raw data frame. The values are inconsistent and some are missing: dates with extra text ("1 October 1829; 197 years ago"), counts with citation markers ("28,233[4]") or notes ("56,409 (total)[8]\n53,131"), and `<NA>` for missing staff counts.

Regular-expression analysis of the raw data

I used regular expressions (Thompson, 1968) to measure how inconsistent the scraped data are, running thirteen checks over the 26 records. The result shows that the data initially contain the spurious, inconsistent and incomplete information that is typical of real web data (Wickham, 2014).

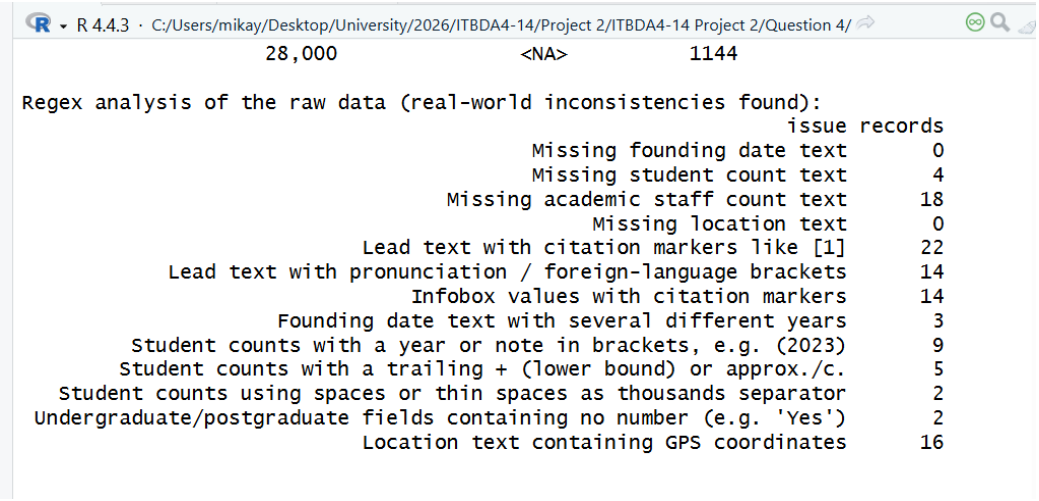


Figure 32: Regex analysis of the raw data: the number of records affected by each type of inconsistency.

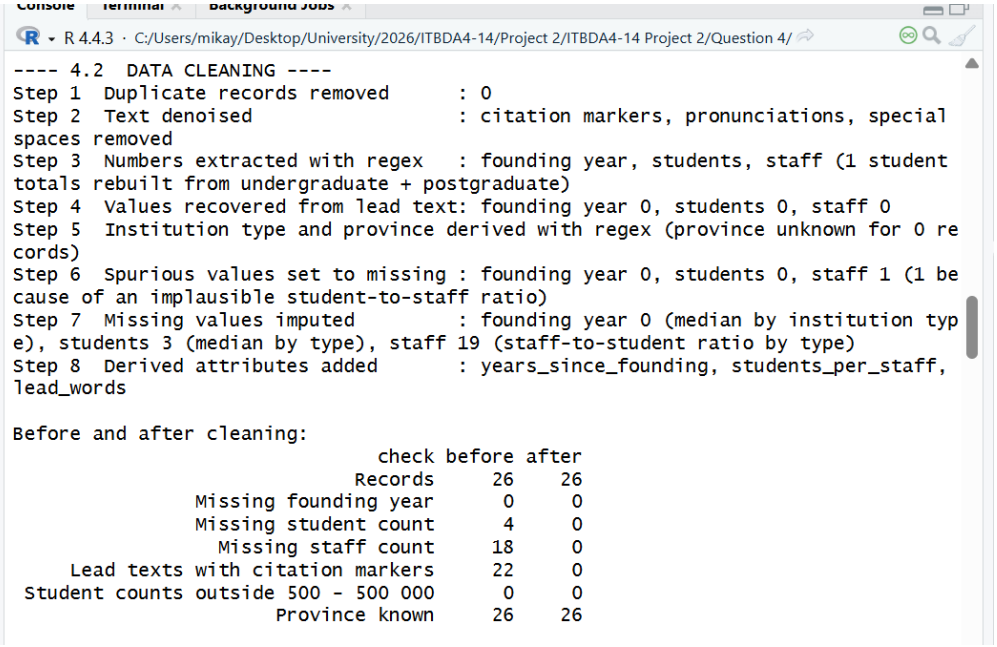
Table 11: Real-world inconsistencies found in the raw data (records affected, out of 26)

Inconsistency	Records
Missing founding date text	0
Missing student count text	4
Missing academic staff count text	18
Missing location text	0
Lead text with citation markers like `[1]`	22
Lead text with pronunciation or foreign-language brackets	14
Infobox values with citation markers	14
Founding date text with several different years	3
Student counts with a year or note in brackets, e.g. (2023)	9
Student counts with a trailing + (lower bound) or approx./c.	5
Student counts using spaces or thin spaces as thousands separator	2
Undergraduate or postgraduate fields containing no number (e.g. 'Yes')	2
Location text containing GPS coordinates	16

The most important gap is academic staff: 18 of 26 pages (69%) report no staff count. Four pages report no student count. In addition, the numbers that are present are written in many ways: with brackets and years, with a "+" for a lower bound, with spaces instead of commas, and (for the University of Pretoria) with two different totals in one cell. I saved the raw data as `q4_universities_raw.csv`.

4.2.) Data cleaning, denoising and missing-value imputation

What I did: I carried out the transformation in eight steps, each of which prints a summary line to the console. Together they follow standard data-cleaning practice for tidy, analysis-ready data (Wickham, 2014).



```
R 4.4.3 · C:/Users/mikay/Desktop/University/2026/ITBDA4-14/Project 2/ITBDA4-14 Project 2/Question 4/

---- 4.2  DATA CLEANING ----
Step 1  Duplicate records removed      : 0
Step 2  Text denoised                  : citation markers, pronunciations, special
spaces removed
Step 3  Numbers extracted with regex   : founding year, students, staff (1 student
totals rebuilt from undergraduate + postgraduate)
Step 4  Values recovered from lead text: founding year 0, students 0, staff 0
Step 5  Institution type and province derived with regex (province unknown for 0 re
cords)
Step 6  Spurious values set to missing : founding year 0, students 0, staff 1 (1 be
cause of an implausible student-to-staff ratio)
Step 7  Missing values imputed         : founding year 0 (median by institution typ
e), students 3 (median by type), staff 19 (staff-to-student ratio by type)
Step 8  Derived attributes added       : years_since_founding, students_per_staff,
lead_words

Before and after cleaning:
                               check before after
                             Records     26    26
               Missing founding year      0     0
               Missing student count      4     0
                 Missing staff count     18     0
     Lead texts with citation markers     22     0
 Student counts outside 500 - 500 000      0     0
                       Province known     26    26
```

Figure 33: Section 4.2 of the console output: the eight cleaning steps and the before-and-after quality table.

Table 12: The eight cleaning steps and what each did

Step	Technique	Result on this dataset
1	Duplicate removal (case-insensitive on university name)	0 duplicates
2	Denoising with regular expressions: citation markers `[n]`, pronunciation and foreign-language brackets, non-breaking and thin spaces, HTML entities, space before punctuation	Citation markers in lead texts: 22 to 0
3	Regex number extraction: removes years in brackets, joins digit groups ("21 993", "53,000"), understands "million" and "k", takes the midpoint of a range, ignores stray years; founding year is the earliest year stated	1 student total rebuilt from undergraduate + postgraduate (Stellenbosch)
4	Recovery of missing values from the lead paragraph with regular expressions	0 additional values found
5	Derivation of categories: institution type (traditional, comprehensive, university of technology) and province (the first province or city named)	Province identified for all 26 records
6	Removal of spurious values: founding year outside 1800-2026, students outside 500-500 000, staff outside 20-20 000, or a ratio outside 5-60 students per staff member	1 staff value removed (ratio too high)
7	Missing-value imputation (flagged)	3 student counts, 19 staff counts
8	Derived attributes: years since founding, students per staff member, lead-paragraph word count	3 new columns

Regex extraction in practice

The figures below show the regular expressions at work. Strings such as "46,539[4] (2016)", "23,000+ (2023)[1]" and "56,409 (total)[8] 53,131 (contact)" are all reduced to a single clean number (46,539; 23,000; 56,409). For the founding year, the first four-digit number in the text would give me 2004 for the University of KwaZulu-Natal and for the Durban University of Technology, but my rule "earliest year stated" finds 1910 and 1907, the years in which their predecessor institutions were established.

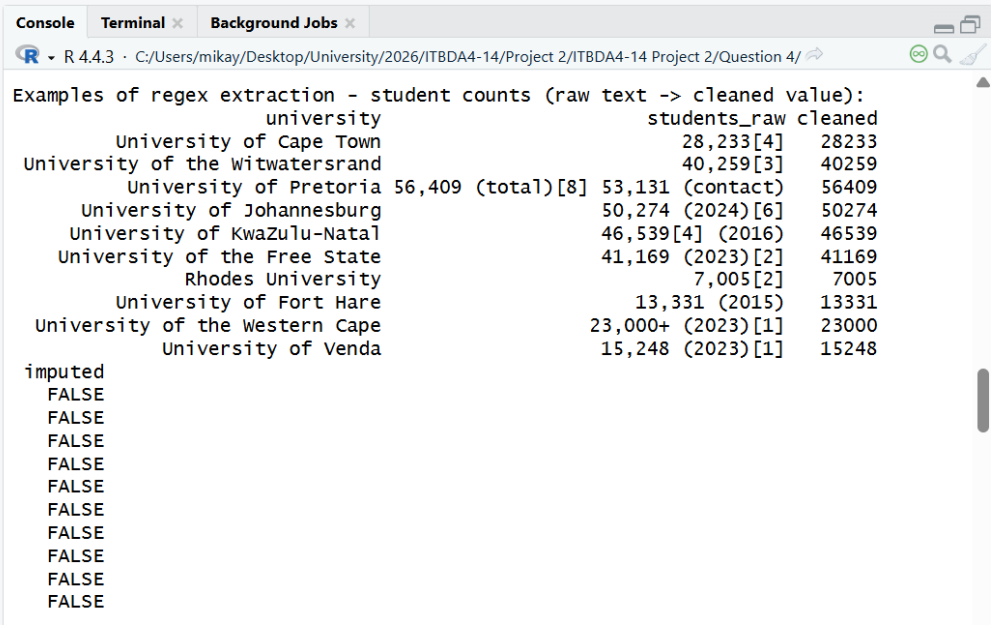


Figure 34: Regex extraction of student counts: the raw text on the left and the cleaned number on the right.

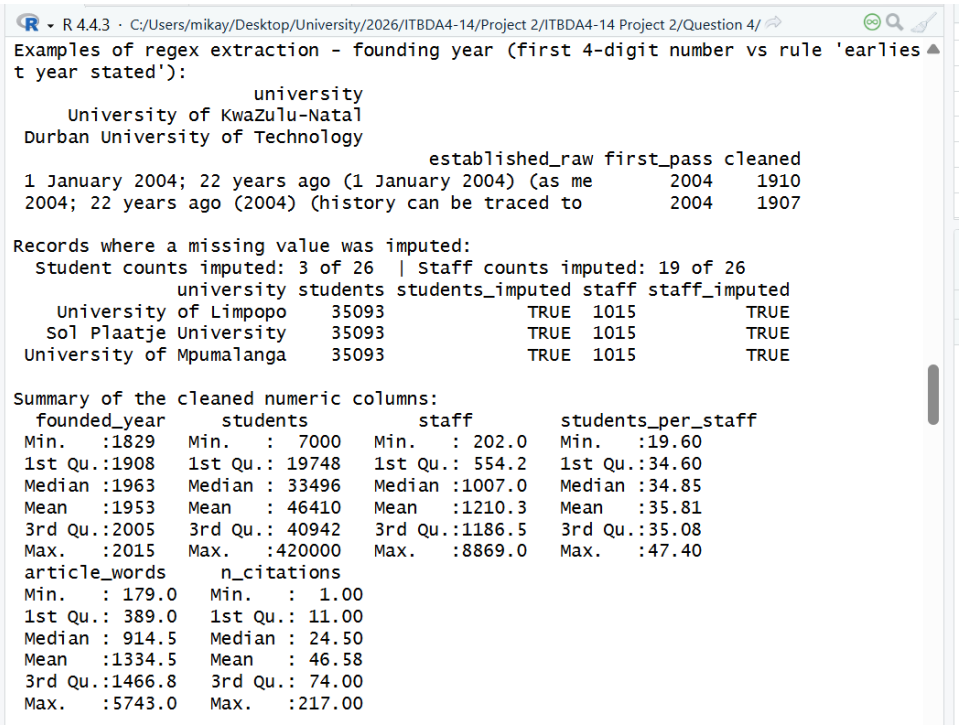


Figure 35: Regex extraction of founding years (first four-digit number against the rule "earliest year stated"), the three records where student counts were imputed, and the summary of the cleaned numeric columns.

Missing-value imputation

After extraction, three student counts (University of Limpopo, Sol Plaatje University and University of Mpumalanga) and 19 staff counts (18 missing plus 1 removed as implausible: the Tshwane University of Technology reported 65,000 students against 755 academic staff, 86 students per staff member) were still missing. Statistically, I cannot ignore such gaps, because the chance that a value is missing may depend on the institution itself (Rubin, 1976), and dropping those records would discard most of the dataset. I therefore used two simple imputation methods, in the spirit of multiple-imputation practice (van Buuren and Groothuis-Oudshoorn, 2011) but without simulating uncertainty:

- I replaced **student counts and founding year** with the **median of the institution type** (traditional university, comprehensive university or university of technology). The three imputed student counts all equal 35,093, the median of the traditional universities.
- I replaced **staff counts** with the number of students multiplied by the **median staff-to-student ratio** of the institution type, calculated from the institutions that did report staff.
- I **flagged** every imputed value in a Boolean column (`students_imputed`, `staff_imputed`, `founded_year_imputed`), so that later analysis can exclude or separately treat imputed values.

The imputation is a pragmatic choice: median imputation preserves the centre of each group but understates its variability, and with 19 of 26 staff counts imputed the staff variable is largely modelled rather than observed. This is why my SQL analysis in 4.3 uses only reported staff counts for staffing comparisons.

Table 13: Data quality before and after cleaning

Check	Before	After
Records	26	26
Missing founding year	0	0
Missing student count	4	0
Missing staff count	18	0
Lead texts with citation markers	22	0
Student counts outside 500 to 500 000	0	0
Province known	26	26

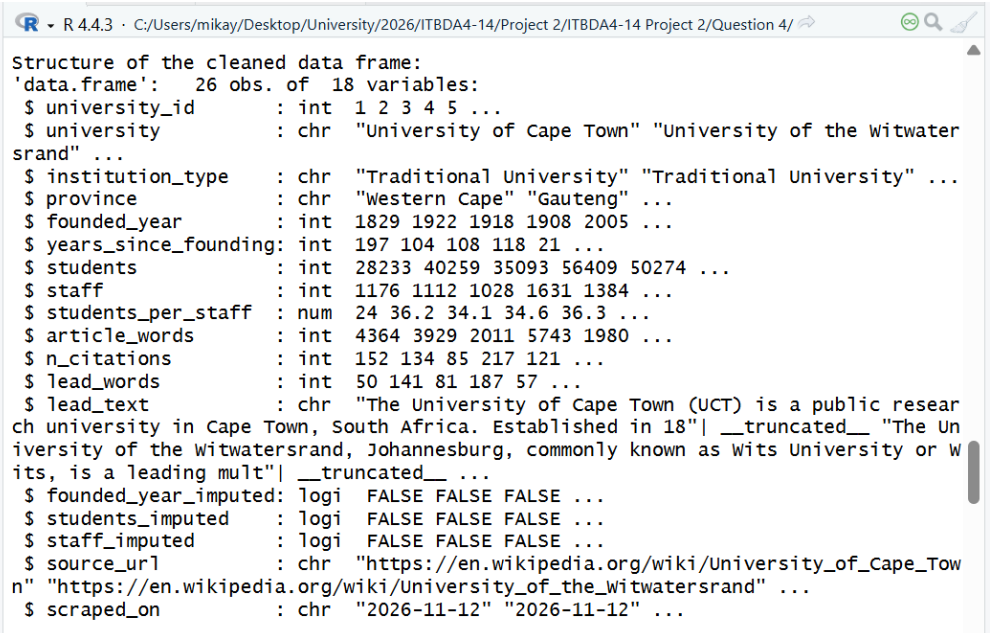


Figure 36: Structure of the cleaned data frame: 26 observations and 18 variables, with integer counts, a numeric ratio, a text lead paragraph, three imputation flags and the source URL.

```
Cleaned data (first rows):
 university_id                     university        institution_type
             1         University of Cape Town   Traditional University
             2 University of the Witwatersrand   Traditional University
             3         Stellenbosch University   Traditional University
             4          University of Pretoria   Traditional University
             5      University of Johannesburg   Traditional University
             6     University of KwaZulu-Natal   Traditional University
             7    University of the Free State   Traditional University
             8       Nelson Mandela University Comprehensive University
             9                 Rhodes University   Traditional University
            10           North-West University   Traditional University
     province founded_year students staff students_per_staff
 Western Cape         1829    28233  1176               24.0
      Gauteng         1922    40259  1112               36.2
 Western Cape         1918    35093  1028               34.1
      Gauteng         1908    56409  1631               34.6
      Gauteng         2005    50274  1384               36.3
 KwaZulu-Natal         1910    46539  1328               35.0
    Free State         1904    41169  1190               34.6
 Eastern Cape         1882    28000   591               47.4
 Eastern Cape         1904     7005   357               19.6
   North West         2004    56000  1619               34.6

Saved to C:/Users/mikay/Desktop/University/2026/ITBDA4-14/Project 2/ITBDA4-14 Proje
ct 2/Question 4 :
  q4_universities_raw.csv (4.1), q4_universities_clean.csv (4.2), q4_quality_repor
t.csv, q4_robots.txt, folder q4_raw_pages/
> source("C:/Users/mikay/Desktop/University/2026/ITBDA4-14/Project 2/ITBDA4-14 Proj
ect 2/Question 4/q4_load_postgres.R")
```

Figure 37: The first cleaned records (university, institution type, province, founding year, students, staff and students per staff) and the confirmation that the files were saved, including `q4_universities_clean.csv`.

Export. I exported the cleaned data frame as `q4_universities_clean.csv` (UTF-8, comma-separated, empty cell for a missing value, snake_case column names, ISO dates), so that it can be loaded into a database or used directly for further analysis. After cleaning, the founding years run from 1829 to 2015, student counts from 7,000 to 420,000 (median 33,496) and the median number of students per staff member is 34.9.

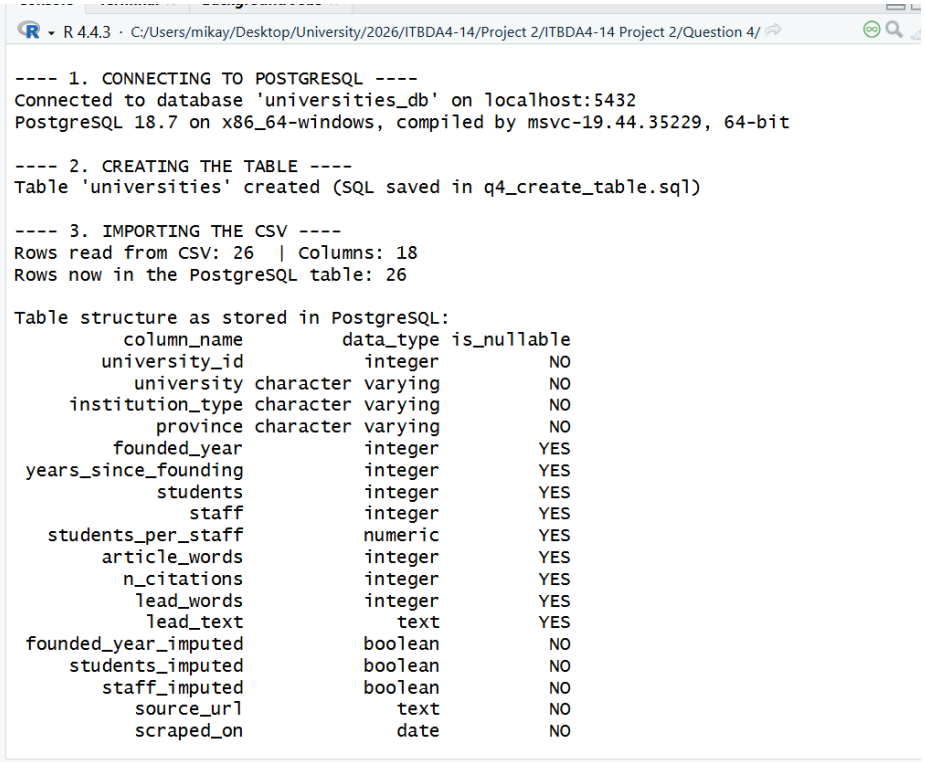
4.3.) Importing the data into PostgreSQL and SQL analysis

Import: I imported the cleaned CSV into a relational table named `universities` in a PostgreSQL database called `universities_db` (PostgreSQL is a relational database management system; the relational model itself goes back to Codd, 1970). My script `q4_load_postgres.R` connects with the `DBI` and `RPostgres` packages, asks for the database password when it runs (I never store the password in the code), creates the table with explicit data types and constraints, imports the CSV and reports the number of rows. The constraints protect the quality of the data: a primary key, a unique university name, `NOT NULL` where a value is always known, and `CHECK` rules for valid institution types, plausible years and positive counts.

Table 14: Structure of the `universities` table

Column	Data type	Constraint
`university_id`	INTEGER	PRIMARY KEY
`university`	VARCHAR(150)	NOT NULL, UNIQUE
`institution_type`	VARCHAR(40)	NOT NULL, CHECK in ('Traditional University', 'Comprehensive University', 'University of Technology')
`province`	VARCHAR(30)	NOT NULL
`founded_year`, `years_since_founding`	INTEGER	CHECK founded_year between 1800 and 2026
`students`, `staff`	INTEGER	CHECK greater than 0
`students_per_staff`	NUMERIC(6,1)	-

Column	Data type	Constraint
`article_words`, `n_citations`, `lead_words`	INTEGER	-
`lead_text`	TEXT	-
`founded_year_imputed`, `students_imputed`, `staff_imputed`	BOOLEAN	NOT NULL
`source_url`	TEXT	NOT NULL
`scraped_on`	DATE	NOT NULL



```
R 4.4.3 · C:/Users/mikay/Desktop/University/2026/ITBDA4-14/Project 2/ITBDA4-14 Project 2/Question 4/

---- 1. CONNECTING TO POSTGRESQL ----
Connected to database 'universities_db' on localhost:5432
PostgreSQL 18.7 on x86_64-windows, compiled by msvc-19.44.35229, 64-bit

---- 2. CREATING THE TABLE ----
Table 'universities' created (SQL saved in q4_create_table.sql)

---- 3. IMPORTING THE CSV ----
Rows read from CSV: 26  | Columns: 18
Rows now in the PostgreSQL table: 26

Table structure as stored in PostgreSQL:
          column_name         data_type is_nullable
        university_id           integer          NO
           university character varying          NO
     institution_type character varying          NO
             province character varying          NO
         founded_year           integer         YES
 years_since_founding           integer         YES
              students           integer         YES
                 staff           integer         YES
   students_per_staff           numeric         YES
        article_words           integer         YES
          n_citations           integer         YES
           lead_words           integer         YES
            lead_text              text         YES
 founded_year_imputed           boolean          NO
     students_imputed           boolean          NO
        staff_imputed           boolean          NO
           source_url              text          NO
           scraped_on              date          NO
```

Figure 38: Sections 1 to 3 of the loader's console output: the connection to `universities_db` on PostgreSQL 18.7, the creation of the table, the import of 26 rows from the CSV and the stored table structure.

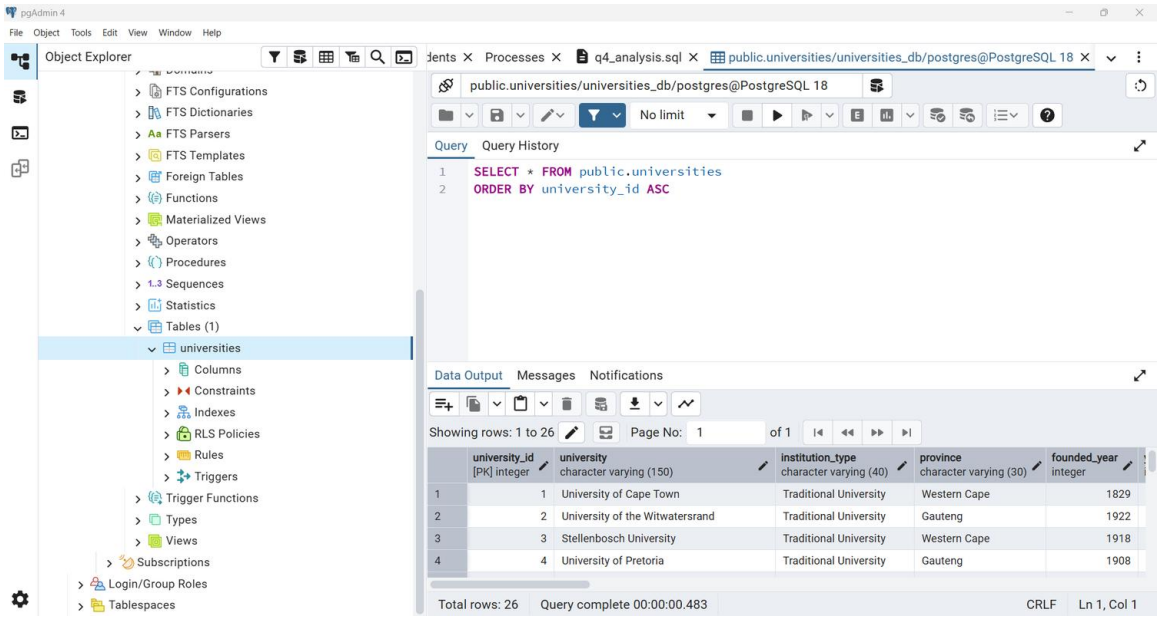


Figure 39: pgAdmin 4: the table `universities` under `universities_db`, `public`, `Tables`, and its first rows retrieved with `SELECT * FROM public.universities ORDER BY university_id` (26 rows, primary key `university_id`).

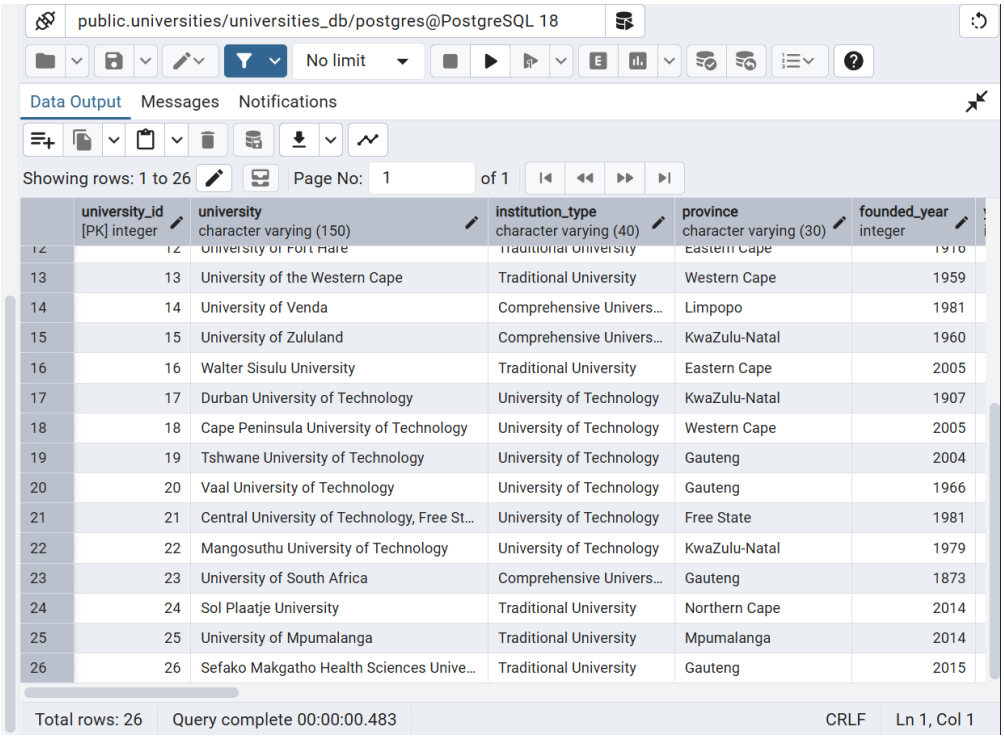


Figure 40: pgAdmin 4: the last rows of the imported table (rows 12 to 26), confirming that all 26 universities were loaded with their institution type, province and founding year.

I then ran two SQL investigations on the table. Because 19 of 26 staff counts and 3 of 26 student counts are imputed, both of my queries report how many values are imputed, and the staffing comparison uses only reported values.

SQL investigation 1: how do the three institution types compare?

Question. Which institution types enrol the most students, how large is a typical institution of each type, how many students are there per academic staff member, and how old are the institutions? My query groups the table by institution type and uses aggregate functions, the median (`PERCENTILE_CONT`) and the `FILTER` clause to calculate the staffing ratio only for universities whose staff and student counts were both reported.

Listing 5: SQL investigation 1

```
SELECT institution_type,
       COUNT(*)                                                                    AS universities,
       SUM(students)                                                               AS total_students,
       ROUND(AVG(students))                                                        AS avg_students,
       ROUND((PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY students))::numeric) AS median_students,
       ROUND(AVG(students_per_staff)
             FILTER (WHERE NOT staff_imputed AND NOT students_imputed), 1)   AS
avg_students_per_staff_reported,
       COUNT(*) FILTER (WHERE NOT staff_imputed AND NOT students_imputed)    AS unis_with_reported_ratio,
       ROUND(AVG(years_since_founding))                                            AS avg_age_years,
       COUNT(*) FILTER (WHERE students_imputed)                                    AS imputed_enrolments
FROM universities
GROUP BY institution_type
ORDER BY total_students DESC;
```

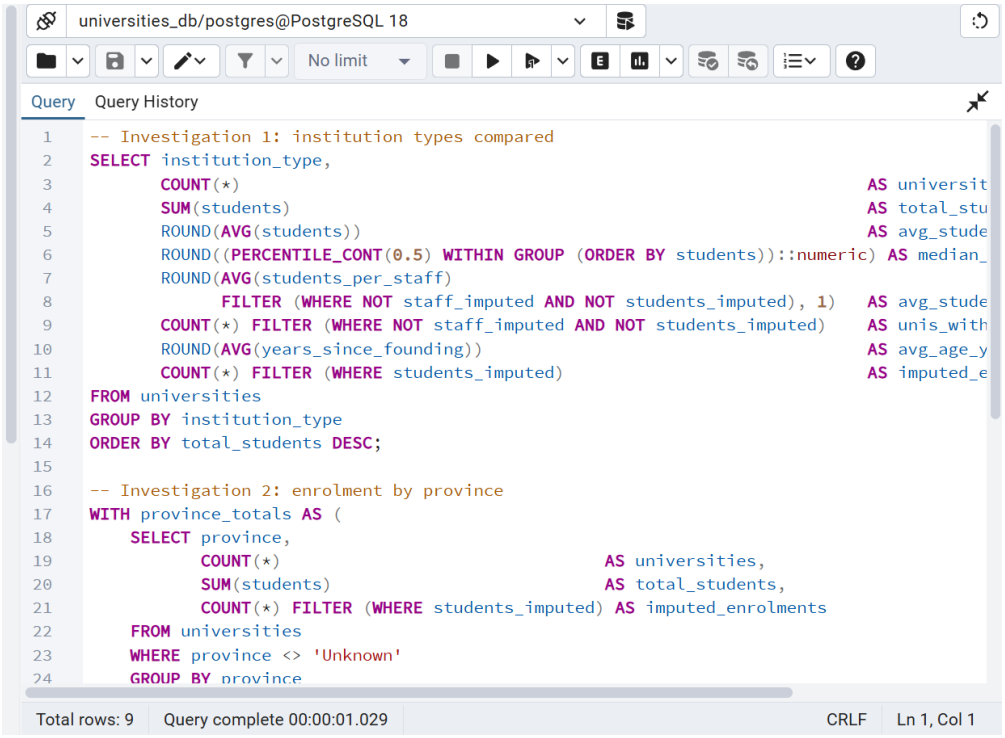
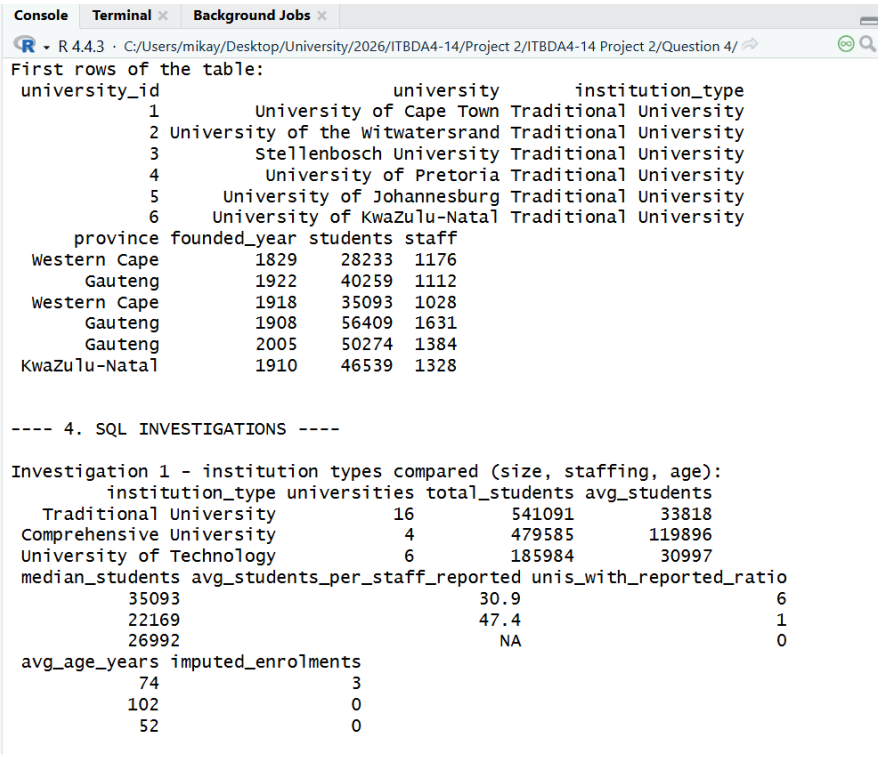


Figure 41: pgAdmin 4 Query Tool: the SQL for investigation 1 and the start of investigation 2, executed against `universities_db`.



```
Console   Terminal ×   Background Jobs ×
R 4.4.3 · C:/Users/mikay/Desktop/University/2026/ITBDA4-14/Project 2/ITBDA4-14 Project 2/Question 4/
First rows of the table:
 university_id                      university       institution_type
             1         University of Cape Town Traditional University
             2 University of the Witwatersrand Traditional University
             3         Stellenbosch University Traditional University
             4          University of Pretoria Traditional University
             5      University of Johannesburg Traditional University
             6     University of KwaZulu-Natal Traditional University
      province founded_year students staff
  Western Cape         1829    28233  1176
       Gauteng         1922    40259  1112
  Western Cape         1918    35093  1028
       Gauteng         1908    56409  1631
       Gauteng         2005    50274  1384
 KwaZulu-Natal         1910    46539  1328


---- 4. SQL INVESTIGATIONS ----

Investigation 1 - institution types compared (size, staffing, age):
         institution_type universities total_students avg_students
   Traditional University           16         541091        33818
 Comprehensive University            4         479585       119896
 University of Technology            6         185984        30997
 median_students avg_students_per_staff_reported unis_with_reported_ratio
           35093                           30.9                        6
           22169                           47.4                        1
           26992                             NA                        0
 avg_age_years imputed_enrolments
            74                   3
           102                   0
            52                   0
```

Figure 42: Console output of investigation 1 (institution types compared). The remaining columns wrap onto the following lines.

Table 15: Result of investigation 1: institution types compared

Institution type	Univ.	Total students	Average	Median	Students per staff (reported)	Univ. with reported ratio	Avg age (years)	Imputed
Traditional	16	541,091	33,818	35,093	30.9	6	74	3
Comprehensive	4	479,585	119,896	22,169	47.4	1	102	0
University of Technology	6	185,984	30,997	26,992	NA	0	52	0

Interpretation. In my results, traditional universities are the largest group, with 16 of the 26 institutions and the most students overall (541,091); their average (33,818) and median (35,093) are close together, so enrolment is fairly evenly spread. Comprehensive universities look biggest per institution (average 119,896), but this is driven by one case: the University of South Africa (UNISA), a distance-learning university with about 420,000 students, lifts the average far above the median of 22,169. Without UNISA the other three comprehensive universities average about 20,000 students. Universities of Technology are the youngest group (average age 52 years against 74 and 102), which fits the mergers and conversions of the technikons in the 2000s, and they have the smallest total enrolment (185,984) with a typical size close to that of traditional universities. I treat the staffing comparisons as weak evidence: after excluding imputed values, the staff ratio is available for only six traditional universities (about 31 students per academic staff member), one comprehensive university and no university of technology.

SQL investigation 2: how is enrolment spread across provinces?

Question. How concentrated is higher-education enrolment geographically, which province enrols the most students, what share of the national total does each province hold, and which university is the largest in each province? My query combines two common table expressions (CTEs), a window function (`SUM(...) OVER ()` for the national total and `RANK() OVER` for the ranking) and `DISTINCT ON` to find the largest university per province.

Listing 6: SQL investigation 2

```
WITH province_totals AS (
    SELECT province,
           COUNT(*)                                  AS universities,
           SUM(students)                             AS total_students,
           COUNT(*) FILTER (WHERE students_imputed) AS imputed_enrolments
    FROM universities
    WHERE province <> 'Unknown'
    GROUP BY province
),
largest AS (
    SELECT DISTINCT ON (province)
           province, university AS largest_university, students AS largest_enrolment
    FROM universities
    WHERE province <> 'Unknown'
    ORDER BY province, students DESC
)
SELECT p.province,
       p.universities,
       p.total_students,
       ROUND(100.0 * p.total_students / SUM(p.total_students) OVER (), 1) AS pct_of_national_enrolment,
       RANK() OVER (ORDER BY p.total_students DESC)                       AS enrolment_rank,
       p.imputed_enrolments,
       l.largest_university,
       l.largest_enrolment
FROM province_totals p
JOIN largest l USING (province)
ORDER BY enrolment_rank;
```

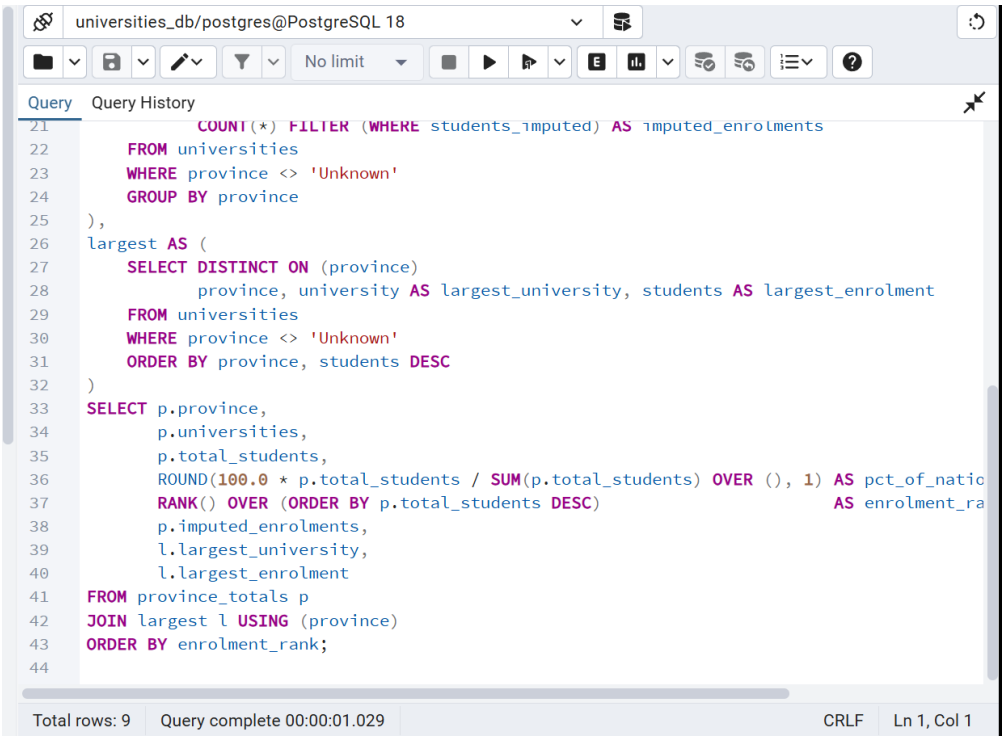
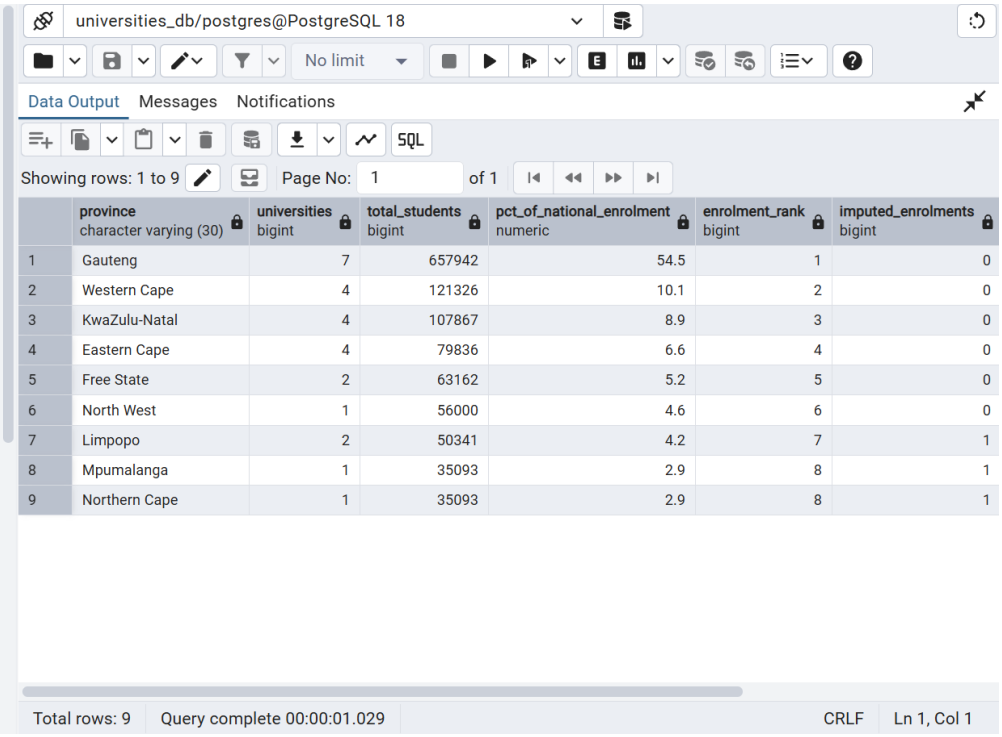


Figure 43: pgAdmin 4 Query Tool: the SQL for investigation 2 (two CTEs, a join, a window function and `RANK`).



	province character varying (30)	universities bigint	total_students bigint	pct_of_national_enrolment numeric	enrolment_rank bigint	imputed_enrolments bigint
1	Gauteng	7	657942	54.5	1	0
2	Western Cape	4	121326	10.1	2	0
3	KwaZulu-Natal	4	107867	8.9	3	0
4	Eastern Cape	4	79836	6.6	4	0
5	Free State	2	63162	5.2	5	0
6	North West	1	56000	4.6	6	0
7	Limpopo	2	50341	4.2	7	1
8	Mpumalanga	1	35093	2.9	8	1
9	Northern Cape	1	35093	2.9	8	1

Figure 44: pgAdmin 4: the result of investigation 2, nine provinces ranked by enrolment (`Showing rows: 1 to 9`, query complete in about one second).

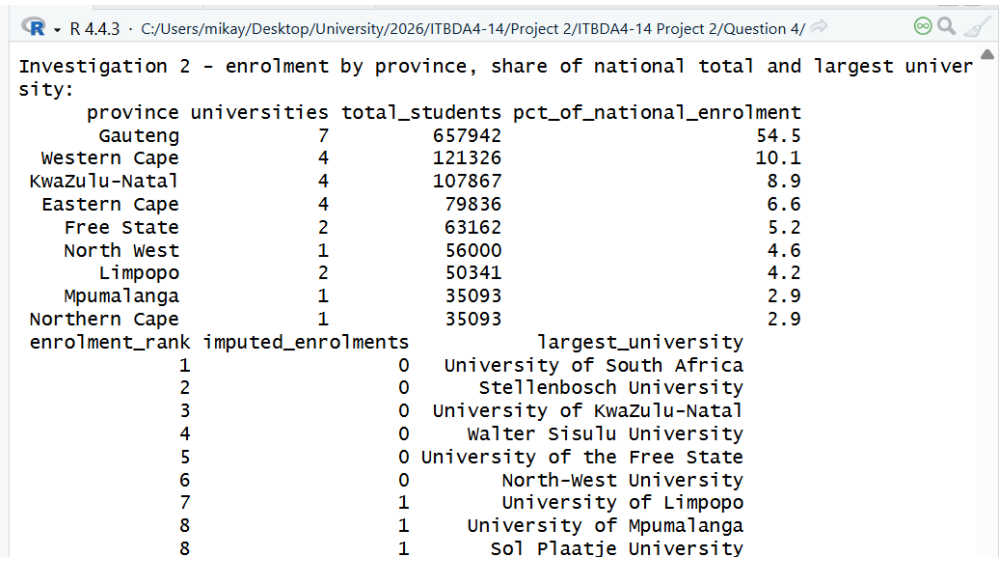


Figure 45: Console output of investigation 2: provinces, number of universities, total students, share of national enrolment, rank and largest university.

Table 16: Result of investigation 2: enrolment by province

Rank	Province	Univ.	Total students	% of national	Imputed	Largest university (students)
1	Gauteng	7	657,942	54.5	0	University of South Africa (420,000)
2	Western Cape	4	121,326	10.1	0	Stellenbosch University (35,093)
3	KwaZulu-Natal	4	107,867	8.9	0	University of KwaZulu-Natal (46,539)
4	Eastern Cape	4	79,836	6.6	0	Walter Sisulu University (31,500)

Rank	Province	Univ.	Total students	% of national	Imputed	Largest university (students)
5	Free State	2	63,162	5.2	0	University of the Free State (41,169)
6	North West	1	56,000	4.6	0	North-West University (56,000)
7	Limpopo	2	50,341	4.2	1	University of Limpopo (35,093)
8	Mpumalanga	1	35,093	2.9	1	University of Mpumalanga (35,093)
8	Northern Cape	1	35,093	2.9	1	Sol Plaatje University (35,093)

Interpretation. In my results, enrolment is highly concentrated: Gauteng alone holds 54.5% of all students, and the top three provinces (Gauteng, Western Cape and KwaZulu-Natal) together hold about 73.5%. Gauteng's share is inflated by UNISA, which is headquartered in Gauteng but teaches students nationwide through distance learning. Without UNISA's 420,000 students, Gauteng would have about 238,000 students, roughly 30% of the remaining total, so it would still be the largest province but far less dominant. Provinces with several universities, such as the Eastern Cape (four universities, 6.6%), enrol relatively few students each. I do not treat the bottom of the ranking as reliable evidence: Mpumalanga and the Northern Cape tie at rank 8 only because each has one university whose enrolment I imputed with the same median (35,093), and the largest university of Limpopo is also an imputed value. This is why my query reports the number of imputed enrolments alongside the totals.

Limitations of the analysis

- The data come from Wikipedia infoboxes that cite different years (roughly 2015 to 2026), so my totals are approximate and are not an official enrolment statistic.
- Three of 26 enrolment figures (11.5%) and 19 of 26 academic staff figures (73%) are imputed. The flags in the database make this visible, and I regard the findings as a demonstration of the ETL pipeline rather than definitive statistics.
- I derive institution type by a regular expression from the university name and the lead paragraph, so a few classifications (for example Walter Sisulu University, which is labelled traditional) may differ from the official categories.

4.4.) GitHub repository

I published the project in a GitHub repository so that the documentation, datasets and analytics notebooks are version-controlled, traceable and reproducible. Reproducible computational work depends on exactly this combination of code, data and documentation under version control (Sandve et al., 2013; Ram, 2013; Perez-Riverol et al., 2016). Repository: https://github.com/Mikaylen-Naidoo/CON-1459393-V9B6-ITBDA4-14-Project-2

Contents of the repository

Table 17: Repository contents against the requirements of question 4.4

Requirement	Where it is in the repository
Project documentation (Questions 1 to 4)	`README.md` (overview, structure, how to reproduce, key results, data sources and limitations) and the folder `Final Report (Project 2)` containing this report

Requirement	Where it is in the repository
The project datasets	Question 1: `data/map_reduce_test_docs/` (seven articles) and the saved results `part-00000_africa` and `part-00000_visa`. Question 2: `data/news/` (ten articles) and the output CSV files. Question 3: `q3_raw_page.html` and its CSV files. Question 4: `q4_raw_pages/` (26 saved articles), `q4_universities_raw.csv`, `q4_universities_clean.csv` and the SQL result files
Notebooks showing the flow of analytics for Questions 1 to 4	One R Markdown notebook in each question folder (`Q1_MapReduce_Notebook.Rmd`, `Q2_LDA_Notebook.Rmd`, `Q3_WebText_Notebook.Rmd`, `Q4_ETL_Notebook.Rmd`), each knitted to a `.md` file that GitHub displays with its tables and plots
R scripts	`mapper.R`, `reducer.R`, `run_local.R`, `run_job.sh`, `q2_lda.R`, `q3_webtext.R`, `q4_scrape_clean.R`, `q4_load_postgres.R`

I wrote the notebooks as literate programs, in which explanatory text, code and results are woven together (Knuth, 1984), so that a reader can follow my analysis from raw data to conclusion. They use the saved datasets, so they run without an internet connection; the Question 4 notebook runs its SQL against PostgreSQL when the database password is supplied in the session and otherwise shows the saved results. By documenting the source and licence of my data clearly, I also make it findable and reusable, in line with the FAIR principles for research data (Wilkinson et al., 2016).

Good practice applied

- **Credentials are never committed.** My scripts ask for the e-mail address and the database password when they run, and a `.gitignore` file excludes R history and workspace files, which could contain secrets.
- **Licensing and attribution.** I credit the Wikipedia-derived data to Wikipedia under CC BY-SA 4.0 in the README, the data and the figure captions; the Question 1 and 2 datasets were supplied with the brief and are included for assessment only.
- **Reproducibility.** Fixed random seeds, saved source pages and a documented run order allow another person to reproduce every result I report.

Figure 46 shows the top level of my repository on GitHub, with the folders for Questions 1 to 4, the `.gitignore` file and `README.md`.

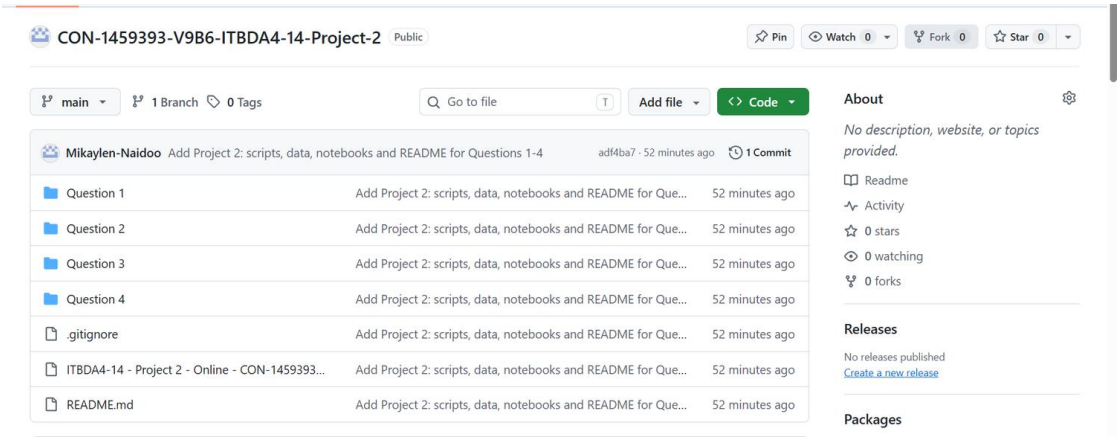


Figure 46: The GitHub repository `CON-1459393-V9B6-ITBDA4-14-Project-2` after the first commit: the folders for Questions 1 to 4, the `.gitignore` file and `README.md`.

References

Blei, D.M. (2012) 'Probabilistic topic models', *Communications of the ACM*, 55(4), pp. 77–84. https://doi.org/10.1145/2133806.2133826

Brin, S. and Page, L. (1998) 'The anatomy of a large-scale hypertextual Web search engine', *Computer Networks and ISDN Systems*, 30(1–7), pp. 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X

Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377–387. https://doi.org/10.1145/362384.362685

Dean, J. and Ghemawat, S. (2008) 'MapReduce: simplified data processing on large clusters', *Communications of the ACM*, 51(1), pp. 107–113. https://doi.org/10.1145/1327452.1327492

Denny, M.J. and Spirling, A. (2018) 'Text preprocessing for unsupervised learning: why it matters, when it misleads, and what to do about it', *Political Analysis*, 26(2), pp. 168–189. https://doi.org/10.1017/pan.2017.44

Erkan, G. and Radev, D.R. (2004) 'LexRank: graph-based lexical centrality as salience in text summarization', *Journal of Artificial Intelligence Research*, 22, pp. 457–479. https://doi.org/10.1613/jair.1523

Feinerer, I., Hornik, K. and Meyer, D. (2008) 'Text mining infrastructure in R', *Journal of Statistical Software*, 25(5), pp. 1–54. https://doi.org/10.18637/jss.v025.i05

Ghemawat, S., Gobioff, H. and Leung, S.-T. (2003) 'The Google file system', in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*. New York: ACM, pp. 29–43. https://doi.org/10.1145/945445.945450

Griffiths, T.L. and Steyvers, M. (2004) 'Finding scientific topics', *Proceedings of the National Academy of Sciences*, 101(Suppl. 1), pp. 5228–5235. https://doi.org/10.1073/pnas.0307752101

Grün, B. and Hornik, K. (2011) 'topicmodels: an R package for fitting topic models', *Journal of Statistical Software*, 40(13), pp. 1–30. https://doi.org/10.18637/jss.v040.i13

Hirschberg, J. and Manning, C.D. (2015) 'Advances in natural language processing', *Science*, 349(6245), pp. 261–266. https://doi.org/10.1126/science.aaa8685

Knuth, D.E. (1984) 'Literate programming', *The Computer Journal*, 27(2), pp. 97–111. https://doi.org/10.1093/comjnl/27.2.97

Krotov, V. and Johnson, L. (2023) 'Big web data: challenges related to data, technology, legality, and ethics', *Business Horizons*, 66(4), pp. 481–491. https://doi.org/10.1016/j.bushor.2022.10.001

Krotov, V., Johnson, L. and Silva, L. (2020) 'Tutorial: legality and ethics of web scraping', *Communications of the Association for Information Systems*, 47, pp. 539–563. https://doi.org/10.17705/1CAIS.04724

Landers, R.N. et al. (2016) 'A primer on theory-driven web scraping: automatic extraction of big data from the Internet for use in psychological research', *Psychological Methods*, 21(4), pp. 475–492. https://doi.org/10.1037/met0000081

Luhn, H.P. (1958) 'The automatic creation of literature abstracts', *IBM Journal of Research and Development*, 2(2), pp. 159–165. https://doi.org/10.1147/rd.22.0159

Mesgari, M. et al. (2015) '"The sum of all human knowledge": a systematic review of scholarly research on the content of Wikipedia', *Journal of the Association for Information Science and Technology*, 66(2), pp. 219–245. https://doi.org/10.1002/asi.23172

Nenkova, A. and McKeown, K. (2012) 'A survey of text summarization techniques', in Aggarwal, C.C. and Zhai, C. (eds.) *Mining text data*. Boston, MA: Springer, pp. 43–76. https://doi.org/10.1007/978-1-4614-3223-4_3

Perez-Riverol, Y. et al. (2016) 'Ten simple rules for taking advantage of Git and GitHub', *PLOS Computational Biology*, 12(7), e1004947. https://doi.org/10.1371/journal.pcbi.1004947

Porter, M.F. (1980) 'An algorithm for suffix stripping', *Program*, 14(3), pp. 130–137. https://doi.org/10.1108/eb046814

Ram, K. (2013) 'Git can facilitate greater reproducibility and increased transparency in science', *Source Code for Biology and Medicine*, 8, article 7. https://doi.org/10.1186/1751-0473-8-7

Rubin, D.B. (1976) 'Inference and missing data', *Biometrika*, 63(3), pp. 581–592. https://doi.org/10.1093/biomet/63.3.581

Salton, G. and Buckley, C. (1988) 'Term-weighting approaches in automatic text retrieval', *Information Processing & Management*, 24(5), pp. 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Sandve, G.K. et al. (2013) 'Ten simple rules for reproducible computational research', *PLOS Computational Biology*, 9(10), e1003285. https://doi.org/10.1371/journal.pcbi.1003285

Shvachko, K. et al. (2010) 'The Hadoop distributed file system', in *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*. Incline Village, NV: IEEE, pp. 1–10. https://doi.org/10.1109/MSST.2010.5496972

Spärck Jones, K. (1972) 'A statistical interpretation of term specificity and its application in retrieval', *Journal of Documentation*, 28(1), pp. 11–21. https://doi.org/10.1108/eb026526

Thompson, K. (1968) 'Programming techniques: regular expression search algorithm', *Communications of the ACM*, 11(6), pp. 419–422. https://doi.org/10.1145/363347.363387

Uysal, A.K. and Gunal, S. (2014) 'The impact of preprocessing on text classification', *Information Processing & Management*, 50(1), pp. 104–112. https://doi.org/10.1016/j.ipm.2013.08.006

van Buuren, S. and Groothuis-Oudshoorn, K. (2011) 'mice: multivariate imputation by chained equations in R', *Journal of Statistical Software*, 45(3), pp. 1–67. https://doi.org/10.18637/jss.v045.i03

Vassiliadis, P. (2009) 'A survey of extract-transform-load technology', *International Journal of Data Warehousing and Mining*, 5(3), pp. 1–27. https://doi.org/10.4018/jdwm.2009070101

Wickham, H. (2014) 'Tidy data', *Journal of Statistical Software*, 59(10), pp. 1–23. https://doi.org/10.18637/jss.v059.i10

Wickham, H. (2016) *ggplot2: elegant graphics for data analysis*. 2nd edn. Cham: Springer. https://doi.org/10.1007/978-3-319-24277-4

Wilkinson, M.D. et al. (2016) 'The FAIR guiding principles for scientific data management and stewardship', *Scientific Data*, 3, article 160018. https://doi.org/10.1038/sdata.2016.18

Appendix A: R script for Question 2 (q2_lda.R)

The complete script for Question 2: data frame, pre-processing, LDA and the three visualisations.

Listing 7: `q2_lda.R`

```
# q2_lda.R  -  ITBDA4-14 Project 2, Question 2
# Topic modelling of news articles with Latent Dirichlet Allocation
#   2.1 Build data frame (file_name, content) and pre-process the text
#   2.2 LDA: top n topics, top k words per topic, word distributions
#   2.3 Visualise the top words per topic (3 methods)
#
# HOW TO RUN: set the working directory to your Q2 folder, open this
#             file in RStudio and click "Source". No edits are needed.
#             (The first run installs any missing packages.)

# ---- Settings ----------------------------------------------------
input_dir <- "data/news"
n_topics  <- 3      # n : number of topics
k_words   <- 10     # k : top words shown per topic
seed      <- 1234   # makes the LDA result reproducible

# ---- Packages (installed automatically if missing) ----------------
pkgs <- c("tm", "topicmodels", "ggplot2", "wordcloud", "RColorBrewer")
missing_pkgs <- pkgs[!(pkgs %in% rownames(installed.packages()))]
if (length(missing_pkgs) > 0) {
  message("Installing missing packages: ", paste(missing_pkgs, collapse = ", "))
  install.packages(missing_pkgs, repos = "https://cloud.r-project.org")
}
suppressPackageStartupMessages(invisible(lapply(pkgs, library, character.only = TRUE)))

cat("Working directory (outputs are saved here):\n", getwd(), "\n\n")

# 2.1  DATA FRAME + PRE-PROCESSING
if (!dir.exists(input_dir)) stop("Input folder not found: ", input_dir)

files <- list.files(input_dir, pattern = "\\.txt$", full.names = TRUE, recursive = TRUE)
files <- files[!grepl("__MACOSX", files) & !grepl("^\\._", basename(files))]
if (length(files) == 0) stop("No .txt files found in: ", input_dir)
files <- files[order(as.numeric(gsub("\\D", "", basename(files))))]   # news_1, news_2, ... news_10

read_doc <- function(f) {
  txt <- readLines(f, warn = FALSE, encoding = "UTF-8")
  Encoding(txt) <- "UTF-8"
  paste(txt, collapse = " ")
}

# Data frame: col_1 = file_name, col_2 = content (raw text of each file)
news_df <- data.frame(
  file_name = basename(files),
  content   = vapply(files, read_doc, character(1), USE.NAMES = FALSE),
  stringsAsFactors = FALSE
)
headlines <- vapply(files, function(f) trimws(readLines(f, n = 1, warn = FALSE, encoding = "UTF-8")),
                    character(1), USE.NAMES = FALSE)

cat("---- 2.1  DATA FRAME (raw) ----\n")
cat("Rows:", nrow(news_df), " Columns:", paste(names(news_df), collapse = ", "), "\n\n")
print(data.frame(file_name = news_df$file_name,
                 content   = paste0(substr(news_df$content, 1, 60), "..."),
                 words     = vapply(strsplit(news_df$content, "\\s+"), length, integer(1))),
      row.names = FALSE)

# ---- Data denoising ----------------------------------------------
custom_stop <- c("said", "says", "say", "will", "also", "can", "may", "must", "would",
                 "could", "should", "one", "per", "cent", "added", "according",
                 "however", "including", "among", "since", "new", "year", "years",
                 "sanews", "gov", "monday", "tuesday", "wednesday", "thursday", "friday",
                 "saturday", "sunday", "january", "february", "march", "april", "june",
                 "july", "august", "september", "october", "november", "december")
stop_all <- unique(c(tm::stopwords("english"), custom_stop))

weekdays_rx <- "(monday|tuesday|wednesday|thursday|friday|saturday|sunday)"
months_rx   <- "(january|february|march|april|may|june|july|august|september|october|november|december)"

clean_text <- function(x) {
  x <- gsub("[\r\n\t]", " ", x)                                  # line breaks, tabs
  x <- gsub("[\u2018\u2019\u201B\u00B4`]", "'", x, perl = TRUE)     # curly apostrophes -> '
  x <- gsub("[\u201C\u201D]", " ", x, perl = TRUE)                  # curly quotes
  x <- tolower(x)                                                # lower case
  x <- gsub(paste0(weekdays_rx, ",? +", months_rx, " +[0-9]{1,2},? +[0-9]{4}"), " ", x)  # date lines
  x <- gsub("https?://[^ ]+|www\\.[^ ]+|[a-z0-9.]+\\.(gov|co|org|com)\\.za", " ", x)  # URLs / site names
  x <- gsub("[^ ]+@[^ ]+", " ", x)                                # e-mail addresses
  x <- gsub("'s\\b", "", x, perl = TRUE)                         # possessives (Eskom's -> eskom)
```

```
  x <- gsub("[^a-z ]", " ", x)                                    # punctuation, numbers, symbols
  words <- strsplit(x, " +")[[1]]
  words <- words[nchar(words) >= 3 & !(words %in% stop_all)]      # short words + stop words
  paste(words, collapse = " ")
}

news_df$clean_content <- vapply(news_df$content, clean_text, character(1), USE.NAMES = FALSE)

cat("\n---- Pre-processing summary ----\n")
summary_df <- data.frame(
  file_name    = news_df$file_name,
  words_before = vapply(strsplit(news_df$content, "\\s+"), length, integer(1)),
  words_after  = vapply(strsplit(news_df$clean_content, " "), length, integer(1))
)
print(summary_df, row.names = FALSE)
cat("\nExample - before (", news_df$file_name[1], "):\n", substr(news_df$content[1], 1, 250), "...\n", sep
= "")
cat("\nExample - after  (", news_df$file_name[1], "):\n", substr(news_df$clean_content[1], 1, 250),
"...\n", sep = "")

write.csv(news_df, "q2_clean_dataframe.csv", row.names = FALSE)

# 2.2  LATENT DIRICHLET ALLOCATION
cat("\n\n---- 2.2  LDA (n =", n_topics, "topics, k =", k_words, "words) ----\n")

corpus <- VCorpus(VectorSource(news_df$clean_content))
dtm    <- DocumentTermMatrix(corpus, control = list(wordLengths = c(3, Inf)))
dtm    <- dtm[, slam::col_sums(dtm > 0) >= 2]          # keep terms found in at least 2 documents
keep   <- slam::row_sums(dtm) > 0                      # drop any document left empty
dtm    <- dtm[keep, ]
doc_names <- news_df$file_name[keep]
doc_heads <- headlines[keep]
cat("Document-term matrix:", nrow(dtm), "documents x", ncol(dtm), "terms\n\n")

lda_model <- LDA(dtm, k = n_topics, method = "Gibbs",
                 control = list(seed = seed, burnin = 1000, iter = 2000, thin = 100))

post  <- posterior(lda_model)
beta  <- post$terms      # topic x term  : word distribution per topic
gamma <- post$topics     # document x topic : topic mixture per document

# Top k words per topic, with their probability (word distribution)
top_terms <- do.call(rbind, lapply(seq_len(n_topics), function(t) {
  ord <- order(beta[t, ], decreasing = TRUE)[seq_len(k_words)]
  data.frame(topic = paste("Topic", t), rank = seq_len(k_words),
             term = colnames(beta)[ord], probability = round(beta[t, ord], 4),
             stringsAsFactors = FALSE)
}))

cat("---- Top", k_words, "words per topic ----\n")
for (t in seq_len(n_topics)) {
  sub <- top_terms[top_terms$topic == paste("Topic", t), ]
  cat("\nTopic ", t, ":  ", paste(sub$term, collapse = ", "), "\n", sep = "")
}

cat("\n---- Word distribution per topic (probability of each word within the topic) ----\n")
for (t in seq_len(n_topics)) {
  sub <- top_terms[top_terms$topic == paste("Topic", t), c("rank", "term", "probability")]
  cat("\nTopic ", t, "  (top ", k_words, " words cover ",
      round(100 * sum(sub$probability), 1), "% of the topic)\n", sep = "")
  print(sub, row.names = FALSE)
}

cat("\n---- Topic mix per document (document-topic probabilities) ----\n")
doc_topics <- data.frame(file_name = doc_names,
                         round(gamma, 3), check.names = FALSE, stringsAsFactors = FALSE)
colnames(doc_topics)[2:(n_topics + 1)] <- paste("Topic", seq_len(n_topics))
doc_topics$dominant_topic <- paste("Topic", apply(gamma, 1, which.max))
doc_topics$headline <- doc_heads
print(doc_topics[, c("file_name", paste("Topic", seq_len(n_topics)), "dominant_topic")], row.names =
FALSE)
cat("\nDocuments per dominant topic:\n"); print(table(doc_topics$dominant_topic))

write.csv(top_terms,  "q2_top_terms.csv",  row.names = FALSE)
write.csv(doc_topics, "q2_doc_topics.csv", row.names = FALSE)

# 2.3  VISUALISATION  (use the arrows in the Plots pane to move between plots)
cat("\n\n---- 2.3  VISUALISATIONS ----\n")

# (1) Bar charts: top k words per topic, bar length = word probability
bar_df <- top_terms
bar_df$id <- factor(seq_len(nrow(bar_df)), levels = rev(seq_len(nrow(bar_df))))
p_bar <- ggplot(bar_df, aes(x = id, y = probability, fill = topic)) +
  geom_col(show.legend = FALSE) +
```

```
  coord_flip() +
  facet_wrap(~ topic, scales = "free") +
  scale_x_discrete(labels = setNames(bar_df$term, bar_df$id)) +
  labs(title = paste("Top", k_words, "words per topic (LDA)"),
       x = NULL, y = "Word probability within topic") +
  theme_minimal(base_size = 12)
print(p_bar)
ggsave("q2_barplot_top_words.png", p_bar, width = 11, height = 5, dpi = 150)

# (2) Word clouds: one per topic, word size = word probability
draw_clouds <- function() {
  op <- par(mfrow = c(1, n_topics), mar = c(0, 0, 2, 0))
  on.exit(par(op))
  for (t in seq_len(n_topics)) {
    top <- sort(beta[t, ], decreasing = TRUE)[1:20]
    set.seed(seed)
    wordcloud(names(top), top, scale = c(3, 0.6), min.freq = 0, random.order = FALSE,
              colors = brewer.pal(8, "Dark2")[t])
    title(main = paste("Topic", t), cex.main = 1.4)
  }
}
draw_clouds()
png("q2_wordclouds.png", width = 1800, height = 600, res = 150); draw_clouds(); invisible(dev.off())

# (3) Heatmap: how strongly each document belongs to each topic
heat_df <- data.frame(
  file_name   = rep(doc_names, times = n_topics),
  topic       = rep(paste("Topic", seq_len(n_topics)), each = length(doc_names)),
  probability = as.vector(gamma)
)
heat_df$file_name <- factor(heat_df$file_name, levels = rev(doc_names))
p_heat <- ggplot(heat_df, aes(x = topic, y = file_name, fill = probability)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.2f", probability)), size = 3.5) +
  scale_fill_gradient(low = "white", high = "steelblue", limits = c(0, 1)) +
  labs(title = "Topic mix per document", x = NULL, y = NULL, fill = "Probability") +
  theme_minimal(base_size = 12)
print(p_heat)
ggsave("q2_heatmap_doc_topics.png", p_heat, width = 7, height = 5, dpi = 150)

cat("\nSaved to", getwd(), ":\n",
    " q2_clean_dataframe.csv, q2_top_terms.csv, q2_doc_topics.csv\n",
    " q2_barplot_top_words.png, q2_wordclouds.png, q2_heatmap_doc_topics.png\n")
```

Appendix B: R script for Question 3 (q3_webtext.R)

The complete script for Question 3: compliant scraping, pre-processing, three summarisation techniques and four plots.

Listing 8: `q3_webtext.R`

```
# q3 webtext.R  -  ITBDA4-14 Project 2, Question 3
# WebText mining proof of concept: Wikipedia article "Natural language processing"
#   3.1 Compliant web scraping -> R data structure
#   3.2 Text pre-processing (denoising)
#   3.3 Text summarisation (3 techniques) + 4 visualisations
#
# HOW TO RUN: set the working directory to your Question 3 folder, open this
#             file in RStudio and click "Source". Type your e-mail address when
#             asked (it is placed in the User-Agent, as Wikimedia's policy requires).

# Settings
page_url <- "https://en.wikipedia.org/wiki/Natural_language_processing"
robots_url <- "https://en.wikipedia.org/robots.txt"
snapshot <- "q3_raw_page.html"      # local copy of the downloaded page
top_n_terms <- 15                   # words shown in frequency chart
n_summary_sentences <- 5            # length of extractive summaries
seed <- 1234

# Packages (installed automatically if missing)
pkgs <- c("httr", "rvest", "xml2", "tm", "SnowballC", "ggplot2", "wordcloud", "RColorBrewer")
missing_pkgs <- pkgs[!(pkgs %in% rownames(installed.packages()))]
if (length(missing_pkgs) > 0) {
  message("Installing missing packages: ", paste(missing_pkgs, collapse = ", "))
  install.packages(missing_pkgs, repos = "https://cloud.r-project.org")
}
suppressPackageStartupMessages(invisible(lapply(pkgs, library, character.only = TRUE)))
cat("Working directory (outputs are saved here):\n", getwd(), "\n\n")


# 3.1  COMPLIANT WEB SCRAPING

cat("---- 3.1  WEB SCRAPING ----\n")

# (a) Identify ourselves: Wikimedia requires a descriptive User-Agent with contact details
if (interactive()) {
  contact <- trimws(readline("Enter your e-mail address (used in the User-Agent so Wikimedia can contact
you): "))
} else {
  contact <- "student@example.com"
}
if (!nzchar(contact)) stop("An e-mail address is required for the User-Agent. Source the script again and
type one.")
ua <- paste0("ITBDA4-14-Project2-Coursework/1.0 (Eduvos student research; single page request; contact: ",
contact, ")")
cat("User-Agent:", ua, "\n")

# (b) robots.txt check: is this page allowed for automated access?
robots_allows <- function(txt, path) {
  lines <- trimws(sub("#.*$", "", strsplit(txt, "\r?\n")[[1]]))
  in_star <- FALSE; prev_ua <- FALSE; rules <- list()
  for (ln in lines) {
    if (!nzchar(ln)) next
    kv <- regmatches(ln, regexec("^([A-Za-z-]+)\\s*:\\s*(.*)$", ln))[[1]]
    if (length(kv) != 3) next
    key <- tolower(kv[2]); val <- kv[3]
    if (key == "user-agent") {
      if (!prev_ua) in_star <- FALSE
      if (val == "*") in_star <- TRUE
      prev_ua <- TRUE
    } else {
      prev_ua <- FALSE
      if (in_star && key %in% c("allow", "disallow") && nzchar(val)) rules[[length(rules) + 1]] <- c(key,
val)
    }
  }
  allowed <- TRUE; best <- -1; best_rule <- "none"
  for (r in rules) {
    if (startsWith(path, r[2]) && nchar(r[2]) > best) {
      best <- nchar(r[2]); allowed <- (r[1] == "allow"); best_rule <- paste(r[1], r[2])
    }
  }
  list(allowed = allowed, n_rules = length(rules), rule = best_rule)
}

page_path <- sub("^https?://[^/]+", "", page_url)
online <- TRUE
```

```
robots_resp <- tryCatch(GET(robots_url, user_agent(ua), timeout(30)), error = function(e) NULL)
if (!is.null(robots_resp) && status_code(robots_resp) == 200) {
  robots_txt <- content(robots_resp, as = "text", encoding = "UTF-8")
  writeLines(robots_txt, "q3_robots.txt")
  chk <- robots_allows(robots_txt, page_path)
  cat("robots.txt read (", chk$n_rules, " rules for all bots). Path tested: ", page_path, "\n", sep = "")
  cat("Matching rule:", chk$rule, "\n")
  cat("Automated access to this page allowed:", chk$allowed, "\n")
  if (!chk$allowed) stop("robots.txt does not allow access to this page. Scraping stopped.")
} else {
  online <- FALSE
  cat("Could not read robots.txt (no connection or access refused).\n")
}

# (c) Download ONE page, politely, and keep a local copy
html_text <- NULL
if (online) {
  Sys.sleep(1)
  resp <- tryCatch(GET(page_url, user_agent(ua), timeout(60)), error = function(e) NULL)
  if (!is.null(resp) && status_code(resp) == 200) {
    writeBin(content(resp, as = "raw"), snapshot)
    html_text <- content(resp, as = "text", encoding = "UTF-8")
    cat("Page downloaded (HTTP 200) and saved as", snapshot, "\n")
  } else {
    cat("Download failed (status:", if (is.null(resp)) "no response" else status_code(resp), ").\n")
  }
}
if (is.null(html_text)) {
  if (file.exists(snapshot)) {
    cat("Using the saved local copy:", snapshot, "\n")
    html_text <- paste(readLines(snapshot, warn = FALSE, encoding = "UTF-8"), collapse = "\n")
  } else {
    stop("Could not download the page and no saved copy exists. Check your internet connection and try
again.")
  }
}
accessed <- format(Sys.time(), "%d %B %Y")

# (d) Parse the HTML into an R data structure
page <- read_html(html_text)
title_node <- html_element(page, "h1")
page_title <- if (inherits(title_node, "xml_missing")) "Natural language processing" else
html_text2(title_node)
lastmod_node <- html_element(page, "#footer-info-lastmod")
last_edited <- if (inherits(lastmod_node, "xml_missing")) "not stated" else
trimws(html_text2(lastmod_node))

body <- html_element(page, "div.mw-parser-output")
if (inherits(body, "xml_missing")) stop("Could not find the article text in the page (the page layout may
have changed).")

# remove non-prose elements: citation markers, edit links, tables/infoboxes, figures, maths, navigation
boxes
remove_sel <- c("sup", "style", "script", "table", "figure", ".mw-editsection", ".reference", ".noprint",
                ".navbox", ".hatnote", ".mwe-math-element", ".thumb", ".ambox", ".mw-empty-elt",
                ".metadata", ".sidebar", ".shortdescription")
for (sel in remove_sel) {
  nodes <- html_elements(body, sel)
  if (length(nodes) > 0) xml_remove(nodes)
}

# the article is split into nested <section> blocks, so collect headings, paragraphs and lists in document
order
nodes <- html_elements(body, xpath = paste0(
  ".//*[self::h2 or self::h3 or self::h4 or ",
  "(self::p and not(ancestor::li)) or ",
  "((self::ul or self::ol) and not(ancestor::li) and not(ancestor::ul) and not(ancestor::ol))]"))
sec <- "Introduction"; subsec <- NA_character_
rows <- list()
add_row <- function(type, txt) {
  txt <- trimws(txt)
  if (nchar(txt) >= 20) rows[[length(rows) + 1]] <<- data.frame(section = sec, subsection = subsec,
                                                                type = type, text = txt, stringsAsFactors
= FALSE)
}
for (i in seq_along(nodes)) {
  node <- nodes[[i]]
  nm   <- xml_name(node)
  if (nm %in% c("h2", "h3", "h4")) {
    htxt <- trimws(html_text2(node))
    if (nm == "h2") { sec <- htxt; subsec <- NA_character_ } else subsec <- htxt
  } else if (nm == "p") {
    add_row("paragraph", html_text2(node))
  } else {
```

```
    for (it in html_elements(node, xpath = "./li")) add_row("list item", html_text2(it))
  }
}
if (length(rows) == 0) stop("No article text could be extracted.")
web_df <- do.call(rbind, rows)
web_df <- data.frame(id = seq_len(nrow(web_df)), web_df, stringsAsFactors = FALSE)

# R data structure: a list holding the metadata and the content data frame
wiki_data <- list(
  meta = list(source = "Wikipedia (English)", title = page_title, url = page_url,
              accessed = accessed, last_edited = last_edited,
              licence = "CC BY-SA 4.0", user_agent = ua),
  content = web_df
)

cat("\nArticle        :", wiki_data$meta$title, "\n")
cat("URL            :", wiki_data$meta$url, "\n")
cat("Accessed       :", wiki_data$meta$accessed, "\n")
cat("Page last edit :", wiki_data$meta$last_edited, "\n")
cat("Extracted      :", nrow(web_df), "text blocks in", length(unique(web_df$section)), "sections\n\n")
cat("Structure of the R data structure:\n"); str(wiki_data, max.level = 1, give.attr = FALSE)
cat("\nFirst rows of the content data frame:\n")
print(data.frame(web_df[1:min(6, nrow(web_df)), c("id", "section", "type")],
                 text = paste0(substr(web_df$text[1:min(6, nrow(web_df))], 1, 70), "...")), row.names =
FALSE)
cat("\nText blocks per section:\n"); print(table(factor(web_df$section, levels = unique(web_df$section))))

write.csv(web_df, "q3_web_data.csv", row.names = FALSE)


# 3.2  TEXT PRE-PROCESSING (DENOISING)

cat("\n\n---- 3.2  PRE-PROCESSING ----\n")

count_words <- function(x) sum(vapply(strsplit(trimws(x), "\\s+"), function(w) sum(nzchar(w)),
integer(1)))

custom_stop <- c("also", "may", "can", "one", "used", "use", "using", "many", "well", "often", "called",
                 "known", "two", "new", "first", "however", "based", "within", "including", "example",
                 "examples", "such", "like", "since", "among", "per", "will", "would", "could", "much",
                 "often", "various", "several", "became", "become", "due", "thus", "eg", "etc")
stop_all <- unique(c(tm::stopwords("english"), custom_stop))

# Step 1: remove non-pertinent sections (references, links, navigation)
nonpert_rx <- "^(references|external links|see also|further
reading|notes|sources|bibliography|citations|footnotes)$"
keep_rows  <- !grepl(nonpert_rx, trimws(web_df$section), ignore.case = TRUE)
clean_df   <- web_df[keep_rows, ]

# Step 2: remove citation markers, URLs, HTML entities, curly quotes, possessives, extra spaces
strip_markup <- function(x) {
  x <- gsub("\\[[^]]*\\]", " ", x)                          # [1], [citation needed], [edit]
  x <- gsub("https?://[^ ]+|www\\.[^ ]+", " ", x)           # URLs
  x <- gsub("&[a-z]+;|&#[0-9]+;", " ", x)                   # HTML entities
  x <- gsub("[\u2018\u2019\u201B`\u00B4]", "'", x, perl = TRUE)
  x <- gsub("[\u201C\u201D]", "\"", x, perl = TRUE)
  x <- gsub("[\u2013\u2014]", " ", x, perl = TRUE)          # dashes
  x <- gsub("[\r\n\t]", " ", x)
  gsub("\\s+", " ", trimws(x))
}
clean_df$text_clean <- strip_markup(clean_df$text)

# Step 3-6: tokenise -> lower case -> remove punctuation/numbers/symbols -> stop words and short words ->
stem
tokenise <- function(x) {
  x <- tolower(x)
  x <- gsub("'s\\b", "", x, perl = TRUE)
  x <- gsub("[^a-z ]", " ", x)
  w <- unlist(strsplit(x, " +"))
  w[nchar(w) >= 3 & !(w %in% stop_all)]
}
clean_df$words <- lapply(clean_df$text_clean, tokenise)

raw_words      <- count_words(web_df$text)
sec_words      <- count_words(clean_df$text)
markup_words   <- count_words(clean_df$text_clean)
tokens_before  <- unlist(lapply(clean_df$text_clean, function(x) {
  w <- unlist(strsplit(gsub("[^a-z ]", " ", tolower(x)), " +")); w[nzchar(w)] }))
token_words    <- length(unlist(clean_df$words))

tok <- data.frame(section = rep(clean_df$section, lengths(clean_df$words)),
                  word = unlist(clean_df$words), stringsAsFactors = FALSE)
tok$stem <- SnowballC::wordStem(tok$word, language = "english")
```

```
steps <- data.frame(
  step = c("0. Raw extracted text (all sections)",
           "1. Remove non-pertinent sections (references, see also, ...)",
           "2. Remove citation markers, URLs, entities, odd symbols",
           "3. Lower case, remove punctuation and numbers",
           "4. Remove stop words and words shorter than 3 letters"),
  words = c(raw_words, sec_words, markup_words, length(tokens_before), token_words)
)
steps$change <- c(NA, diff(steps$words))   # negative = words removed; step 3 can rise because hyphenated
words split in two
print(steps, row.names = FALSE)
cat("\nUnique words before stemming:", length(unique(tok$word)), "\n")
cat("Unique terms after stemming :", length(unique(tok$stem)), "\n")

ex_i <- which(nchar(clean_df$text) > 200)[1]; if (is.na(ex_i)) ex_i <- 1
cat("\nExample - raw text:\n", substr(clean_df$text[ex_i], 1, 300), "...\n", sep = "")
cat("\nExample - after cleaning:\n", paste(head(clean_df$words[[ex_i]], 35), collapse = " "), " ...\n",
sep = "")

# Stem completion: show the most common real word for each stem so charts stay readable
tb <- table(tok$stem)
freq <- data.frame(stem = names(tb), freq = as.integer(tb), stringsAsFactors = FALSE)
lab  <- tapply(tok$word, tok$stem, function(w) names(sort(table(w), decreasing = TRUE))[1])
freq$term <- as.character(lab[freq$stem])
freq <- freq[order(-freq$freq, freq$term), ]
freq$share <- round(100 * freq$freq / sum(freq$freq), 2)
rownames(freq) <- NULL

write.csv(clean_df[, c("id", "section", "type", "text_clean")], "q3_clean_text.csv", row.names = FALSE)


# 3.3  TEXT SUMMARISATION

cat("\n\n---- 3.3  TEXT SUMMARISATION ----\n")

caption_txt <- paste0("Source: Wikipedia, '", page_title, "' (CC BY-SA 4.0), accessed ", accessed)

# Technique 1: term frequency and TF-IDF keywords
cat("\n[Technique 1] Keyword summary using term frequency\n")
cat("Total terms:", nrow(freq), " | Total tokens:", sum(freq$freq), "\n\n")
print(head(freq[, c("term", "freq", "share")], top_n_terms), row.names = FALSE)

# TF-IDF with each section treated as one document
sec_sizes <- table(tok$section)
min_tokens <- 50
big_secs <- names(sec_sizes)[sec_sizes >= min_tokens]
if (length(big_secs) < 3) big_secs <- names(sort(sec_sizes, decreasing = TRUE))[seq_len(min(3,
length(sec_sizes)))]
big_secs <- names(sort(sec_sizes[big_secs], decreasing = TRUE))[seq_len(min(6, length(big_secs)))]
all_secs <- names(sec_sizes)
N <- length(all_secs)
doc_freq <- table(unlist(lapply(all_secs, function(s) unique(tok$stem[tok$section == s]))))
tfidf_list <- lapply(big_secs, function(s) {
  st <- tok$stem[tok$section == s]
  cnt <- table(st)
  tf  <- as.numeric(cnt) / length(st)
  idf <- log(N / as.numeric(doc_freq[names(cnt)]))
  d <- data.frame(section = s, stem = names(cnt), tf_idf = tf * idf, stringsAsFactors = FALSE)
  d <- d[order(-d$tf_idf), ]
  d <- head(d, 5)
  d$term <- as.character(lab[d$stem])
  d
})
tfidf_df <- do.call(rbind, tfidf_list)
cat("\nTop TF-IDF words per section (words that characterise each section):\n")
for (s in big_secs) cat("  ", s, ": ", paste(tfidf_df$term[tfidf_df$section == s], collapse = ", "), "\n",
sep = "")

# Sentences (paragraphs only) for the extractive techniques
split_sentences <- function(x) {
  x <- gsub("\\b(e\\.g|i\\.e|etc|vs|Dr|Mr|Mrs|Prof|Fig|approx|cf|al)\\.", "\\1<dot>", x)
  s <- unlist(strsplit(x, "(?<=[.!?])\\s+(?=[A-Z\"'])", perl = TRUE))
  trimws(gsub("<dot>", ".", s, fixed = TRUE))
}
para <- clean_df[clean_df$type == "paragraph", ]
sent_list <- lapply(seq_len(nrow(para)), function(i) {
  s <- split_sentences(para$text_clean[i])
  if (length(s) == 0) return(NULL)
  data.frame(para_id = para$id[i], section = para$section[i], sentence = s, stringsAsFactors = FALSE)
})
sent_df <- do.call(rbind, sent_list)
sent_df$tokens <- lapply(sent_df$sentence, function(s) SnowballC::wordStem(tokenise(s), language =
"english"))
sent_df$n_tokens <- lengths(sent_df$tokens)
```

```
sent_df <- sent_df[sent_df$n_tokens >= 6, ]
rownames(sent_df) <- NULL
sent_df$sent_id <- seq_len(nrow(sent_df))
cat("\nSentences available for extractive summaries:", nrow(sent_df), "\n")
article_words <- count_words(sent_df$sentence)

wrap_print <- function(sents) for (i in seq_along(sents)) cat(strwrap(paste0(i, ". ", sents[i]), width =
90, exdent = 3), sep = "\n")

# Technique 2: frequency-based sentence scoring
cat("\n[Technique 2] Extractive summary: frequency-based sentence scoring\n")
tf_norm <- setNames(freq$freq / max(freq$freq), freq$stem)
sent_df$freq_score <- vapply(sent_df$tokens, function(tk) sum(tf_norm[tk], na.rm = TRUE) /
sqrt(length(tk)), numeric(1))
pick_freq <- head(order(-sent_df$freq_score), n_summary_sentences)
pick_freq <- sort(pick_freq)
sum_freq <- sent_df$sentence[pick_freq]
wrap_print(sum_freq)

# Technique 3: TextRank (graph-based ranking, PageRank on a sentence similarity graph)
cat("\n[Technique 3] Extractive summary: TextRank\n")
vocab <- unique(unlist(sent_df$tokens))
M <- matrix(0, nrow = nrow(sent_df), ncol = length(vocab), dimnames = list(NULL, vocab))
for (i in seq_len(nrow(sent_df))) {
  ct <- table(sent_df$tokens[[i]]); M[i, names(ct)] <- as.numeric(ct)
}
idf_v <- log(nrow(M) / pmax(colSums(M > 0), 1))
W <- sweep(M, 2, idf_v, `*`)
norms <- sqrt(rowSums(W^2)); norms[norms == 0] <- 1
Wn <- W / norms
S <- Wn %*% t(Wn); diag(S) <- 0                          # cosine similarity between sentences
rs <- rowSums(S); rs[rs == 0] <- 1
P <- S / rs                                                # transition matrix
d <- 0.85; n <- nrow(S); r <- rep(1 / n, n)
for (it in 1:100) {
  r_new <- (1 - d) / n + d * as.numeric(t(P) %*% r)
  if (sum(abs(r_new - r)) < 1e-8) { r <- r_new; break }
  r <- r_new
}
sent_df$textrank <- r
pick_tr <- sort(head(order(-sent_df$textrank), n_summary_sentences))
sum_tr <- sent_df$sentence[pick_tr]
wrap_print(sum_tr)

# Comparison of the two summaries
overlap <- length(intersect(pick_freq, pick_tr))
cat("\nSummary comparison\n")
cat("  Article length (words in sentences):", article_words, "\n")
cat("  Frequency summary:", count_words(sum_freq), "words (", round(100 * count_words(sum_freq) /
article_words, 1), "% of article )\n")
cat("  TextRank summary :", count_words(sum_tr), "words (", round(100 * count_words(sum_tr) /
article_words, 1), "% of article )\n")
cat("  Sentences chosen by both methods:", overlap, "of", n_summary_sentences, "\n")

writeLines(c(paste("Summaries of:", page_title), caption_txt, "",
             "FREQUENCY-BASED EXTRACTIVE SUMMARY", sum_freq, "",
             "TEXTRANK EXTRACTIVE SUMMARY", sum_tr), "q3_summaries.txt")
write.csv(freq, "q3_term_frequency.csv", row.names = FALSE)
write.csv(tfidf_df[, c("section", "term", "tf_idf")], "q3_tfidf_by_section.csv", row.names = FALSE)
write.csv(sent_df[, c("sent_id", "section", "sentence", "freq_score", "textrank")],
"q3_sentence_scores.csv", row.names = FALSE)


# VISUALISATIONS (use the arrows in the Plots pane to move between plots)

cat("\n\n---- VISUALISATIONS ----\n")

# Plot 1: most frequent terms
top_df <- head(freq, top_n_terms)
top_df$term <- factor(top_df$term, levels = rev(top_df$term))
p1 <- ggplot(top_df, aes(x = term, y = freq)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = freq), hjust = -0.2, size = 3.5) +
  coord_flip() +
  expand_limits(y = max(top_df$freq) * 1.1) +
  labs(title = paste("Top", top_n_terms, "most frequent terms"),
       subtitle = paste0("Pre-processed text of the Wikipedia article '", page_title, "'"),
       x = "Term (stem completed to the most common word form)", y = "Frequency (number of occurrences)",
       caption = caption_txt) +
  theme_minimal(base_size = 12)
print(p1); ggsave("q3_plot1_term_frequency.png", p1, width = 10, height = 6, dpi = 150)

# Plot 2: word cloud
draw_cloud <- function() {
```

```
  op <- par(mar = c(0, 0, 2.5, 0)); on.exit(par(op))
  wc <- head(freq, 60)
  set.seed(seed)
  wordcloud(wc$term, wc$freq, scale = c(4, 0.8), min.freq = 1, random.order = FALSE,
            colors = brewer.pal(8, "Dark2"))
  title(main = paste0("Word cloud of the 60 most frequent terms - ", page_title), cex.main = 1.1)
  mtext("Word size is proportional to term frequency", side = 1, line = -1, cex = 0.8)
}
draw_cloud()
png("q3_plot2_wordcloud.png", width = 1600, height = 1100, res = 150); draw_cloud(); invisible(dev.off())

# Plot 3: TF-IDF keywords by section
tfidf_df$term_id <- factor(seq_len(nrow(tfidf_df)), levels = rev(seq_len(nrow(tfidf_df))))
tfidf_df$section <- factor(tfidf_df$section, levels = big_secs)
p3 <- ggplot(tfidf_df, aes(x = term_id, y = tf_idf, fill = section)) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  facet_wrap(~ section, scales = "free") +
  scale_x_discrete(labels = setNames(tfidf_df$term, tfidf_df$term_id)) +
  labs(title = "Characteristic words per section (TF-IDF)",
       subtitle = "Higher TF-IDF = word is frequent in this section but rare in the other sections",
       x = "Term", y = "TF-IDF score", caption = caption_txt) +
  theme_minimal(base_size = 11)
print(p3); ggsave("q3_plot3_tfidf_by_section.png", p3, width = 11, height = 7, dpi = 150)

# Plot 4: TextRank sentence scores
top_s <- head(sent_df[order(-sent_df$textrank), ], 10)
top_s$label <- paste0("S", top_s$sent_id, ": ", substr(top_s$sentence, 1, 60), "...")
top_s$in_summary <- ifelse(top_s$sent_id %in% sent_df$sent_id[pick_tr], "Selected for summary", "Not
selected")
top_s$label <- factor(top_s$label, levels = rev(top_s$label))
p4 <- ggplot(top_s, aes(x = label, y = textrank, fill = in_summary)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(values = c("Selected for summary" = "darkorange", "Not selected" = "grey70")) +
  labs(title = "TextRank: ten highest-ranked sentences",
       subtitle = "Sentences are ranked by how similar they are to many other sentences in the article",
       x = "Sentence (number and opening words)", y = "TextRank score", fill = NULL, caption =
caption_txt) +
  theme_minimal(base_size = 11) + theme(legend.position = "bottom")
print(p4); ggsave("q3_plot4_textrank_sentences.png", p4, width = 11, height = 6, dpi = 150)

cat("\nSaved to", getwd(), ":\n",
    " q3_raw_page.html, q3_robots.txt, q3_web_data.csv, q3_clean_text.csv, q3_summaries.txt\n",
    " q3_term_frequency.csv, q3_tfidf_by_section.csv, q3_sentence_scores.csv\n",
    " q3_plot1_term_frequency.png, q3_plot2_wordcloud.png, q3_plot3_tfidf_by_section.png,
q3_plot4_textrank_sentences.png\n")
```

Appendix C: R script for Question 4.1 and 4.2 (q4_scrape_clean.R)

The complete script for Questions 4.1 and 4.2: scraping, regular-expression analysis, cleaning and imputation.

Listing 9: `q4_scrape_clean.R`

```
# q4 scrape clean.R  -  ITBDA4-14 Project 2, Question 4 (parts 4.1 and 4.2)
# Use case: profile dataset of South Africa's public universities, scraped from Wikipedia
#   4.1 Web scraping + regex: 26 article records -> data frame (text + numeric attributes)
#   4.2 Cleaning, denoising, missing-value imputation -> cleaned CSV for loading into Postgres
#
# HOW TO RUN: set the working directory to your Question 4 folder, open this
#             file in RStudio and click "Source". Type your e-mail address when asked
#             (it goes in the User-Agent, as Wikimedia's policy requires).

# Settings
page titles <- c("University of Cape Town", "University of the Witwatersrand", "Stellenbosch University",
                 "University of Pretoria", "University of Johannesburg", "University of KwaZulu-Natal",
                 "University of the Free State", "Nelson Mandela University", "Rhodes University",
                 "North-West University", "University of Limpopo", "University of Fort Hare",
                 "University of the Western Cape", "University of Venda", "University of Zululand",
                 "Walter Sisulu University", "Durban University of Technology",
                 "Cape Peninsula University of Technology", "Tshwane University of Technology",
                 "Vaal University of Technology", "Central University of Technology",
                 "Mangosuthu University of Technology", "University of South Africa",
                 "Sol Plaatje University", "University of Mpumalanga",
                 "Sefako Makgatho Health Sciences University")
base_url    <- "https://en.wikipedia.org/wiki/"
robots_url  <- "https://en.wikipedia.org/robots.txt"
cache_dir   <- "q4_raw_pages"     # downloaded pages are kept here, so re-runs make no new requests
refresh     <- FALSE              # TRUE = download everything again
delay_sec   <- 1.5                # pause between requests (polite scraping)
min_records <- 20                 # brief: at least 20 records

# Packages (installed automatically if missing)
pkgs <- c("httr", "rvest", "xml2")
missing_pkgs <- pkgs[!(pkgs %in% rownames(installed.packages()))]
if (length(missing_pkgs) > 0) {
  message("Installing missing packages: ", paste(missing_pkgs, collapse = ", "))
  install.packages(missing_pkgs, repos = "https://cloud.r-project.org")
}
suppressPackageStartupMessages(invisible(lapply(pkgs, library, character.only = TRUE)))
cat("Working directory (outputs are saved here):\n", getwd(), "\n\n")
if (!dir.exists(cache_dir)) dir.create(cache_dir)


# 4.1  WEB SCRAPING AND REGEX ANALYSIS

cat("---- 4.1  WEB SCRAPING ----\n")
cat("Use case: profile of South Africa's public universities (source: Wikipedia, CC BY-SA 4.0)\n\n")

# (a) Identify ourselves: Wikimedia requires a descriptive User-Agent with contact details
if (interactive()) {
  contact <- trimws(readline("Enter your e-mail address (used in the User-Agent so Wikimedia can contact
you): "))
} else {
  contact <- "student@example.com"
}
if (!nzchar(contact)) stop("An e-mail address is required for the User-Agent. Source the script again and
type one.")
ua <- paste0("ITBDA4-14-Project2-Coursework/1.0 (Eduvos student research; ", length(page_titles),
             " article requests; contact: ", contact, ")")
cat("User-Agent:", ua, "\n")

# (b) robots.txt check
robots_allows <- function(txt, path) {
  lines <- trimws(sub("#.*$", "", strsplit(txt, "\r?\n")[[1]]))
  in_star <- FALSE; prev_ua <- FALSE; rules <- list()
  for (ln in lines) {
    if (!nzchar(ln)) next
    kv <- regmatches(ln, regexec("^([A-Za-z-]+)\\s*:\\s*(.*)$", ln))[[1]]
    if (length(kv) != 3) next
    key <- tolower(kv[2]); val <- kv[3]
    if (key == "user-agent") {
      if (!prev_ua) in_star <- FALSE
      if (val == "*") in_star <- TRUE
      prev_ua <- TRUE
    } else {
      prev_ua <- FALSE
      if (in_star && key %in% c("allow", "disallow") && nzchar(val)) rules[[length(rules) + 1]] <- c(key,
val)
    }
```

```
  }
  allowed <- TRUE; best <- -1
  for (r in rules) {
    if (startsWith(path, r[2]) && nchar(r[2]) > best) { best <- nchar(r[2]); allowed <- (r[1] == "allow")
}
  }
  list(allowed = allowed, n_rules = length(rules))
}

title_to_path <- function(t) paste0("/wiki/", gsub(" ", "_", t))
online <- TRUE
robots_resp <- tryCatch(GET(robots_url, user_agent(ua), timeout(30)), error = function(e) NULL)
if (!is.null(robots_resp) && status_code(robots_resp) == 200) {
  robots_txt <- content(robots_resp, as = "text", encoding = "UTF-8")
  writeLines(robots_txt, "q4_robots.txt")
  ok <- vapply(page_titles, function(t) robots_allows(robots_txt, title_to_path(t))$allowed, logical(1))
  cat("robots.txt read (", robots_allows(robots_txt, "/")$n_rules, " rules for all bots).\n", sep = "")
  cat("Article pages allowed for automated access:", sum(ok), "of", length(ok), "\n")
  if (!all(ok)) {
    cat("Skipping pages disallowed by robots.txt:", paste(page_titles[!ok], collapse = "; "), "\n")
    page_titles <- page_titles[ok]
  }
} else {
  online <- FALSE
  cat("Could not read robots.txt (no connection or access refused). Only saved pages will be used.\n")
}

# (c) Download each article once, politely, and keep a local copy
fetch_page <- function(title) {
  fn  <- file.path(cache_dir, paste0(gsub("[^A-Za-z0-9]+", "_", title), ".html"))
  url <- paste0(base_url, gsub(" ", "_", title))
  if (!refresh && file.exists(fn)) {
    return(list(html = paste(readLines(fn, warn = FALSE, encoding = "UTF-8"), collapse = "\n"),
                url = url, source = "saved copy", status = 200L,
                fetched = format(as.Date(file.info(fn)$mtime), "%Y-%m-%d")))
  }
  if (!online) return(list(html = NULL, url = url, source = "none", status = NA_integer_, fetched =
NA_character_))
  Sys.sleep(delay_sec)
  resp <- tryCatch(GET(url, user_agent(ua), timeout(60)), error = function(e) NULL)
  if (!is.null(resp) && status_code(resp) == 200) {
    writeBin(content(resp, as = "raw"), fn)
    return(list(html = content(resp, as = "text", encoding = "UTF-8"), url = url, source = "downloaded",
status = 200L,
                fetched = format(Sys.Date(), "%Y-%m-%d")))
  }
  list(html = NULL, url = url, source = "failed", status = if (is.null(resp)) NA_integer_ else
status_code(resp), fetched = NA_character_)
}

# (d) Parse one article: infobox rows (label/value) + lead paragraphs + page-level counts
get_label_value <- function(tbl) {
  rows <- html_elements(tbl, xpath = ".//tr[th and td]")
  if (length(rows) == 0) return(data.frame(label = character(0), value = character(0), stringsAsFactors =
FALSE))
  lab <- vapply(rows, function(r) trimws(html_text2(html_element(r, "th"))), character(1))
  val <- vapply(rows, function(r) trimws(html_text2(html_element(r, "td"))), character(1))
  data.frame(label = lab, value = val, stringsAsFactors = FALSE)
}
pick <- function(kv, rx) {
  i <- which(grepl(rx, kv$label, ignore.case = TRUE) & nzchar(kv$value))
  if (length(i) == 0) NA_character_ else kv$value[i[1]]
}
count_words <- function(x) { w <- strsplit(trimws(x), "\\s+")[[1]]; sum(nzchar(w)) }

parse_article <- function(html_text, title, url) {
  page <- read_html(html_text)
  bodies <- html_elements(page, "div.mw-parser-output")          # some pages also have a tiny coordinates
box
  if (length(bodies) == 0) return(NULL)
  body <- bodies[[which.max(vapply(bodies, function(b) nchar(html_text(b)), integer(1)))]]
  for (sel in c("style", "script")) {                             # hidden CSS/JS code must not leak into
the text
    nodes <- html_elements(body, sel)
    if (length(nodes) > 0) xml_remove(nodes)
  }
  h1   <- html_element(page, "h1")
  name <- if (inherits(h1, "xml_missing")) title else trimws(html_text2(h1))

  ib <- html_element(body, "table.infobox")
  kv <- if (inherits(ib, "xml_missing")) data.frame(label = character(0), value = character(0)) else
get_label_value(ib)

  paras <- html_elements(body, xpath = ".//p[not(ancestor::table)]")
```

```
  ptxt  <- vapply(paras, function(p) trimws(html_text2(p)), character(1))
  lead  <- paste(head(ptxt[nchar(ptxt) >= 60], 2), collapse = " ")
  if (!nzchar(lead)) lead <- NA_character_

  # first-pass numbers: deliberately simple regex (the first number-like token), cleaned properly in 4.2
  first_number <- function(x) {
    if (is.na(x)) return(NA_real_)
    m <- regmatches(x, regexpr("[0-9][0-9,. \u00A0]*", x, perl = TRUE))
    if (length(m) == 0) NA_real_ else suppressWarnings(as.numeric(gsub("[^0-9.]", "", m)))
  }
  first_year <- function(x) {
    if (is.na(x)) return(NA_real_)
    m <- regmatches(x, regexpr("[0-9]{4}", x))
    if (length(m) == 0) NA_real_ else as.numeric(m)
  }
  est_raw <- pick(kv, "^(established|founded|founding|opened)")
  stu_raw <- pick(kv, "^(students|total students|total enrol|enrol|student (body|population|enrol))")
  stf_raw <- pick(kv, "^(academic staff|faculty|staff|total staff|employees)")

  data.frame(
    university          = name,
    source_url          = url,
    lead_text           = lead,
    established_raw     = est_raw,
    type_raw            = pick(kv, "^type"),
    students_raw        = stu_raw,
    undergraduates_raw  = pick(kv, "^undergrad"),
    postgraduates_raw   = pick(kv, "^(postgrad|post-grad)"),
    staff_raw           = stf_raw,
    location_raw        = pick(kv, "^(location|campus|campuses|address|city)"),
    founded_first_pass  = first_year(est_raw),
    students_first_pass = first_number(stu_raw),
    staff_first_pass    = first_number(stf_raw),
    article_words       = sum(vapply(ptxt, count_words, integer(1))),
    n_citations         = length(html_elements(body, "sup.reference")),
    n_sections          = length(html_elements(body, "h2")),
    stringsAsFactors = FALSE
  )
}

cat("\nDownloading and parsing ", length(page_titles), " article pages (one request each, ", delay_sec, "
s pause) ...\n", sep = "")
rows <- list(); log_df <- data.frame()
for (i in seq_along(page_titles)) {
  pg <- fetch_page(page_titles[i])
  rec <- if (!is.null(pg$html)) tryCatch(parse_article(pg$html, page_titles[i], pg$url), error =
function(e) NULL) else NULL
  if (!is.null(rec)) { rec$scraped_on <- pg$fetched; rows[[length(rows) + 1]] <- rec }
  log_df <- rbind(log_df, data.frame(page = page_titles[i], source = pg$source,
                                     parsed = !is.null(rec), stringsAsFactors = FALSE))
  cat(sprintf("  %2d/%d  %-45s %-12s %s\n", i, length(page_titles), page_titles[i], pg$source,
              if (!is.null(rec)) "parsed" else "NOT parsed"))
}
if (length(rows) < min_records) {
  stop("Only ", length(rows), " pages could be parsed (need at least ", min_records,
       "). Check your internet connection and the messages above, then Source the script again.")
}
raw_df <- do.call(rbind, rows)
raw_df <- data.frame(id = seq_len(nrow(raw_df)), raw_df, stringsAsFactors = FALSE)

cat("\nRecords scraped:", nrow(raw_df), " | Columns:", ncol(raw_df), "\n")
cat("Request summary: ", sum(log_df$source == "downloaded"), " downloaded, ",
    sum(log_df$source == "saved copy"), " from saved copies, ",
    sum(log_df$source %in% c("failed", "none")), " failed\n", sep = "")

cat("\nStructure of the raw data frame:\n"); str(raw_df, vec.len = 2, give.attr = FALSE)
cat("\nFirst rows (selected columns):\n")
print(data.frame(id = raw_df$id, university = raw_df$university,
                 established_raw = substr(raw_df$established_raw, 1, 28),
                 students_raw = substr(raw_df$students_raw, 1, 24),
                 staff_raw = substr(raw_df$staff_raw, 1, 18),
                 article_words = raw_df$article_words)[1:min(8, nrow(raw_df)), ], row.names = FALSE)

# Regex analysis of the raw data: how messy is it?
has <- function(x, rx) sum(!is.na(x) & grepl(rx, x, perl = TRUE))
n_years <- function(x) {
  if (is.na(x)) return(0L)
  length(unique(regmatches(x, gregexpr("\\b(1[6-9][0-9]{2}|20[0-2][0-9])\\b", x))[[1]]))
}
mess <- data.frame(
  issue = c("Missing founding date text", "Missing student count text", "Missing academic staff count
text",
            "Missing location text",
            "Lead text with citation markers like [1]",
```

```
            "Lead text with pronunciation / foreign-language brackets",
            "Infobox values with citation markers",
            "Founding date text with several different years",
            "Student counts with a year or note in brackets, e.g. (2023)",
            "Student counts with a trailing + (lower bound) or approx./c.",
            "Student counts using spaces or thin spaces as thousands separator",
            "Undergraduate/postgraduate fields containing no number (e.g. 'Yes')",
            "Location text containing GPS coordinates"),
  records = c(sum(is.na(raw_df$established_raw)), sum(is.na(raw_df$students_raw)),
sum(is.na(raw_df$staff_raw)),
              sum(is.na(raw_df$location_raw)),
              has(raw_df$lead_text, "\\[[0-9a-z ]+\\]"),
              has(raw_df$lead_text,
"\\((?:[^()]*?(?:Afrikaans|isiZulu|isiXhosa|Xhosa|Zulu|Sesotho|Setswana|pronounced|IPA|/)[^()]*)\\)"),
              has(paste(raw_df$established_raw, raw_df$students_raw, raw_df$staff_raw), "\\[[0-9a-z
]+\\]"),
              sum(vapply(raw_df$established_raw, n_years, integer(1)) >= 2),
              has(raw_df$students_raw, "\\([^)]*[0-9]{4}[^)]*\\)"),
              has(raw_df$students_raw, "(?i)([0-9]\\s*\\+|\\bc\\.|~|\u2248|approx|about|circa)"),
              has(raw_df$students_raw, "[0-9][ \u00A0\u2009\u202F][0-9]{3}\\b"),
              sum(!is.na(c(raw_df$undergraduates_raw, raw_df$postgraduates_raw)) &
                  !grepl("[0-9]", c(raw_df$undergraduates_raw, raw_df$postgraduates_raw))),
              has(raw_df$location_raw, "\u00B0"))
)
cat("\nRegex analysis of the raw data (real-world inconsistencies found):\n")
print(mess, row.names = FALSE)

write.csv(raw_df, "q4_universities_raw.csv", row.names = FALSE, na = "", fileEncoding = "UTF-8")


# 4.2  DATA CLEANING, DENOISING AND MISSING-VALUE IMPUTATION

cat("\n\n---- 4.2  DATA CLEANING ----\n")
clean_df <- raw_df

# Step 1: remove duplicate records
n0 <- nrow(clean_df)
clean_df <- clean_df[!duplicated(tolower(trimws(clean_df$university))), ]
cat("Step 1  Duplicate records removed      :", n0 - nrow(clean_df), "\n")

# Step 2: denoise text (citation markers, pronunciation brackets, special spaces, entities)
clean_txt <- function(x) {
  x <- gsub("\\[[^]]*\\]", " ", x)                                                  # [1], [a],
[citation needed]
  x <-
gsub("\\((?:[^()]*?(?:Afrikaans|isiZulu|isiXhosa|Xhosa|Zulu|Sesotho|Setswana|pronounced|IPA|/)[^()]*)\\)",
" ", x, perl = TRUE)
  x <- gsub("[\u00A0\u2009\u200B\u202F]", " ", x, perl = TRUE)                       # non-breaking /
thin spaces
  x <- gsub("&[a-z]+;|&#[0-9]+;", " ", x)                                           # HTML entities
  x <- gsub("[\r\n\t]+", " ", x)
  x <- gsub("\\s+([,.;:])", "\\1", x)                                               # space before
punctuation
  trimws(gsub("\\s+", " ", x))
}
clean_df$university <- clean_txt(clean_df$university)
clean_df$lead_text  <- clean_txt(clean_df$lead_text)
for (col in c("established_raw", "students_raw", "undergraduates_raw", "postgraduates_raw", "staff_raw",
"location_raw"))
  clean_df[[col]] <- clean_txt(clean_df[[col]])
cat("Step 2  Text denoised                  : citation markers, pronunciations, special spaces removed\n")

# Step 3: regex extraction of numbers from inconsistent strings
parse_count <- function(x) {
  if (is.na(x) || !nzchar(x)) return(NA_real_)
  s <- tolower(x)
  s <- gsub("\\((?:[^)]*)\\b(19|20)[0-9]{2}\\b(?:[^)]*)\\)", " ", s, perl = TRUE)          # "(2022)" etc.
  s <- gsub("\\b(as of|in|est\\.?|estimate|census)\\s*(19|20)[0-9]{2}\\b", " ", s, perl = TRUE)
  s <- gsub("(?<=\\d)[ ,](?=\\d{3}\\b)", "", s, perl = TRUE)                         # 53,000 / 53 000
-> 53000
  tk <- regmatches(s, gregexpr("[0-9]+(?:\\.[0-9]+)?\\s*(?:million|thousand|m\\b|k\\b)?", s, perl =
TRUE))[[1]]
  if (length(tk) == 0) return(NA_real_)
  val <- vapply(tk, function(t) {
    n <- as.numeric(sub("^([0-9]+(?:\\.[0-9]+)?).*$", "\\1", t, perl = TRUE))
    if (grepl("million|m$", trimws(t))) n <- n * 1e6 else if (grepl("thousand|k$", trimws(t))) n <- n *
1e3
    n
  }, numeric(1))
  val <- val[!(val >= 1900 & val <= 2030 & val == round(val))]                       # drop stray years
  val <- val[val >= 100]
  if (length(val) == 0) return(NA_real_)
  if (length(val) == 2 && grepl("[\u2013-]|\\bto\\b", s, perl = TRUE)) return(mean(val))            #
range -> midpoint
```

```
  val[1]
}
parse_year <- function(x) {
  if (is.na(x) || !nzchar(x)) return(NA_real_)
  y <- as.numeric(regmatches(x, gregexpr("\\b(1[6-9][0-9]{2}|20[0-2][0-9])\\b", x))[[1]])
  y <- y[y >= 1800 & y <= 2026]
  if (length(y) == 0) NA_real_ else min(y)                                          # earliest =
original founding
}

clean_df$founded_year <- vapply(clean_df$established_raw, parse_year, numeric(1), USE.NAMES = FALSE)
clean_df$students     <- vapply(clean_df$students_raw,    parse_count, numeric(1), USE.NAMES = FALSE)
clean_df$staff        <- vapply(clean_df$staff_raw,        parse_count, numeric(1), USE.NAMES = FALSE)
ug <- vapply(clean_df$undergraduates_raw, parse_count, numeric(1), USE.NAMES = FALSE)
pg <- vapply(clean_df$postgraduates_raw,  parse_count, numeric(1), USE.NAMES = FALSE)
from_parts <- is.na(clean_df$students) & (!is.na(ug) | !is.na(pg))
clean_df$students[from_parts] <- rowSums(cbind(ug, pg), na.rm = TRUE)[from_parts]
cat("Step 3  Numbers extracted with regex   : founding year, students, staff (", sum(from_parts),
    " student totals rebuilt from undergraduate + postgraduate)\n", sep = "")

# Step 4: recover missing values from the lead paragraph with regex
n_before <- c(year = sum(is.na(clean_df$founded_year)), students = sum(is.na(clean_df$students)), staff =
sum(is.na(clean_df$staff)))
founded_rx  <- "(?i)(?:founded|established|created|opened|formed)\\s+(?:in|on)?\\s*(?:[0-9]{1,2}\\s+[A-Za-
z]+\\s+)?(1[6-9][0-9]{2}|20[0-2][0-9])"
students_rx <- "(?i)([0-9]{1,3}(?:[, ][0-9]{3})+|[0-9]{3,6})\\s+(?:full-
time\\s+|registered\\s+|enrolled\\s+)?students"
staff_rx    <- "(?i)([0-9]{1,3}(?:[, ][0-9]{3})+|[0-
9]{3,5})\\s+(?:academic\\s+|permanent\\s+)?(?:staff|employees)"
grab <- function(txt, rx) {
  if (is.na(txt)) return(NA_real_)
  m <- regmatches(txt, regexec(rx, txt, perl = TRUE))[[1]]
  if (length(m) < 2) NA_real_ else as.numeric(gsub("[, ]", "", m[2]))
}
for (i in seq_len(nrow(clean_df))) {
  lt <- clean_df$lead_text[i]
  if (is.na(clean_df$founded_year[i])) clean_df$founded_year[i] <- grab(lt, founded_rx)
  if (is.na(clean_df$students[i]))     clean_df$students[i]     <- grab(lt, students_rx)
  if (is.na(clean_df$staff[i]))        clean_df$staff[i]        <- grab(lt, staff_rx)
}
n_after <- c(year = sum(is.na(clean_df$founded_year)), students = sum(is.na(clean_df$students)), staff =
sum(is.na(clean_df$staff)))
cat("Step 4  Values recovered from lead text: founding year ", n_before["year"] - n_after["year"],
    ", students ", n_before["students"] - n_after["students"], ", staff ", n_before["staff"] -
n_after["staff"], "\n", sep = "")

# Step 5: derive categorical attributes with regex (institution type, province)
clean_df$institution_type <- ifelse(grepl("(?i)university of technology|technikon", clean_df$university,
perl = TRUE), "University of Technology",
                             ifelse(grepl("(?i)comprehensive", clean_df$lead_text, perl = TRUE),
"Comprehensive University",
                                    "Traditional University"))
province_map <- c("Western Cape" = "Western Cape|Cape Town|Stellenbosch|Bellville|Mowbray",
                  "Gauteng" =
"Gauteng|Johannesburg|Pretoria|Tshwane|Soweto|Vanderbijlpark|Braamfontein|Auckland Park|Ga-
Rankuwa|Medunsa",
                  "KwaZulu-Natal" = "KwaZulu|Durban|Pietermaritzburg|Richards
Bay|Empangeni|Westville|Umlazi",
                  "Eastern Cape" = "Eastern Cape|Gqeberha|Port
Elizabeth|Makhanda|Grahamstown|Mthatha|Alice|East London",
                  "Free State" = "Free State|Bloemfontein|Welkom|Qwaqwa",
                  "Limpopo" = "Limpopo|Polokwane|Thohoyandou|Mankweng|Sovenga",
                  "North West" = "North West|Potchefstroom|Mahikeng|Mafikeng|Mmabatho",
                  "Mpumalanga" = "Mpumalanga|Mbombela|Nelspruit|Siyabuswa",
                  "Northern Cape" = "Northern Cape|Kimberley")
find_province <- function(txt) {                       # the province mentioned FIRST in the text wins
  if (is.na(txt)) return(NA_character_)
  pos <- vapply(province_map, function(rx) {
    m <- regexpr(rx, txt, perl = TRUE)
    if (m[1] == -1) Inf else as.numeric(m[1])
  }, numeric(1))
  if (all(is.infinite(pos))) NA_character_ else names(province_map)[which.min(pos)]
}
prov <- vapply(clean_df$location_raw, find_province, character(1), USE.NAMES = FALSE)
miss <- is.na(prov)
prov[miss] <- vapply(clean_df$lead_text[miss], find_province, character(1), USE.NAMES = FALSE)
clean_df$province <- ifelse(is.na(prov), "Unknown", prov)
cat("Step 5  Institution type and province derived with regex (province unknown for ",
    sum(clean_df$province == "Unknown"), " records)\n", sep = "")

# Step 6: remove spurious / impossible numeric values (set to missing, then imputed)
bad_year <- !is.na(clean_df$founded_year) & (clean_df$founded_year < 1800 | clean_df$founded_year > 2026)
bad_stu  <- !is.na(clean_df$students) & (clean_df$students < 500 | clean_df$students > 500000)
bad_stf  <- !is.na(clean_df$staff) & (clean_df$staff < 20 | clean_df$staff > 20000)
```

```
ratio_now <- clean_df$students / clean_df$staff
bad_ratio <- !is.na(ratio_now) & !bad_stu & !bad_stf & (ratio_now < 5 | ratio_now > 60)     # implausible
students per academic staff
clean_df$founded_year[bad_year] <- NA; clean_df$students[bad_stu] <- NA
clean_df$staff[bad_stf | bad_ratio] <- NA
cat("Step 6  Spurious values set to missing : founding year ", sum(bad_year), ", students ", sum(bad_stu),
    ", staff ", sum(bad_stf | bad_ratio), " (", sum(bad_ratio), " because of an implausible student-to-
staff ratio)\n", sep = "")

# Step 7: missing-value imputation (flagged so the original gaps stay visible)
clean_df$founded_year_imputed <- is.na(clean_df$founded_year)
clean_df$students_imputed     <- is.na(clean_df$students)
clean_df$staff_imputed        <- is.na(clean_df$staff)
grp_median <- function(v, g) {
  m <- tapply(v, g, median, na.rm = TRUE)
  out <- m[g]; out[is.na(out)] <- median(v, na.rm = TRUE); as.numeric(out)
}
clean_df$founded_year <- ifelse(is.na(clean_df$founded_year), round(grp_median(clean_df$founded_year,
clean_df$institution_type)), clean_df$founded_year)
clean_df$students     <- ifelse(is.na(clean_df$students), round(grp_median(clean_df$students,
clean_df$institution_type)), clean_df$students)
ratio <- clean_df$staff / clean_df$students
ratio_fill <- grp_median(ifelse(clean_df$staff_imputed, NA, ratio), clean_df$institution_type)
clean_df$staff <- ifelse(is.na(clean_df$staff), round(clean_df$students * ratio_fill), clean_df$staff)
cat("Step 7  Missing values imputed         : founding year ", sum(clean_df$founded_year_imputed),
    " (median by institution type), students ", sum(clean_df$students_imputed),
    " (median by type), staff ", sum(clean_df$staff_imputed), " (staff-to-student ratio by type)\n", sep =
"")

# Step 8: derived numeric attributes and tidy column set for the database
this_year <- as.integer(format(Sys.Date(), "%Y"))
clean_df$years_since_founding <- this_year - clean_df$founded_year
clean_df$students_per_staff   <- round(clean_df$students / clean_df$staff, 1)
clean_df$lead_words           <- vapply(clean_df$lead_text, function(t) if (is.na(t)) 0L else
count_words(t), integer(1), USE.NAMES = FALSE)
clean_df$lead_text[is.na(clean_df$lead_text)] <- ""

final_df <- data.frame(
  university = clean_df$university, institution_type = clean_df$institution_type, province =
clean_df$province,
  founded_year = as.integer(clean_df$founded_year), years_since_founding =
as.integer(clean_df$years_since_founding),
  students = as.integer(clean_df$students), staff = as.integer(clean_df$staff),
  students_per_staff = clean_df$students_per_staff,
  article_words = as.integer(clean_df$article_words), n_citations = as.integer(clean_df$n_citations),
  lead_words = as.integer(clean_df$lead_words), lead_text = clean_df$lead_text,
  founded_year_imputed = clean_df$founded_year_imputed, students_imputed = clean_df$students_imputed,
  staff_imputed = clean_df$staff_imputed, source_url = clean_df$source_url, scraped_on =
clean_df$scraped_on,
  stringsAsFactors = FALSE
)
final_df <- data.frame(university_id = seq_len(nrow(final_df)), final_df, stringsAsFactors = FALSE)
cat("Step 8  Derived attributes added       : years_since_founding, students_per_staff, lead_words\n")

# Before / after quality report
before_after <- data.frame(
  check = c("Records", "Missing founding year", "Missing student count", "Missing staff count",
            "Lead texts with citation markers",
            "Student counts outside 500 - 500 000", "Province known"),
  before = c(nrow(raw_df), sum(is.na(raw_df$founded_first_pass)), sum(is.na(raw_df$students_first_pass)),
sum(is.na(raw_df$staff_first_pass)),
             has(raw_df$lead_text, "\\[[0-9a-z ]+\\]"),
             sum(!is.na(raw_df$students_first_pass) & (raw_df$students_first_pass < 500 |
raw_df$students_first_pass > 500000)),
             sum(!is.na(raw_df$location_raw))),
  after = c(nrow(final_df), sum(is.na(final_df$founded_year)), sum(is.na(final_df$students)),
sum(is.na(final_df$staff)),
            has(final_df$lead_text, "\\[[0-9a-z ]+\\]"),
            sum(final_df$students < 500 | final_df$students > 500000),
            sum(final_df$province != "Unknown"))
)
cat("\nBefore and after cleaning:\n"); print(before_after, row.names = FALSE)

cat("\nExamples of regex extraction - student counts (raw text -> cleaned value):\n")
idx <- match(raw_df$id, clean_df$id)
ex_stu <- data.frame(university = raw_df$university,
                     students_raw = gsub("\\s+", " ", substr(raw_df$students_raw, 1, 34)),
                     cleaned = clean_df$students[idx], imputed = clean_df$students_imputed[idx],
stringsAsFactors = FALSE)
ex_stu <- ex_stu[!is.na(ex_stu$students_raw) & grepl("[^0-9, ]", ex_stu$students_raw), ]
print(head(ex_stu, 10), row.names = FALSE)

cat("\nExamples of regex extraction - founding year (first 4-digit number vs rule 'earliest year
stated'):\n")
```

```
ex_yr <- data.frame(university = raw_df$university,
                    established_raw = gsub("\\s+", " ", substr(raw_df$established_raw, 1, 52)),
                    first_pass = raw_df$founded_first_pass, cleaned = clean_df$founded_year[idx],
stringsAsFactors = FALSE)
print(head(ex_yr[!is.na(ex_yr$first_pass) & ex_yr$first_pass != ex_yr$cleaned, ], 8), row.names = FALSE)

cat("\nRecords where a missing value was imputed:\n")
imp <- data.frame(university = clean_df$university, students = clean_df$students, students_imputed =
clean_df$students_imputed,
                  staff = clean_df$staff, staff_imputed = clean_df$staff_imputed, stringsAsFactors =
FALSE)
cat("  Student counts imputed:", sum(imp$students_imputed), "of", nrow(imp), " | Staff counts imputed:",
sum(imp$staff_imputed), "of", nrow(imp), "\n")
print(imp[imp$students_imputed, ], row.names = FALSE)

cat("\nSummary of the cleaned numeric columns:\n")
print(summary(final_df[, c("founded_year", "students", "staff", "students_per_staff", "article_words",
"n_citations")]))
cat("\nStructure of the cleaned data frame:\n"); str(final_df, vec.len = 2, give.attr = FALSE)
cat("\nCleaned data (first rows):\n")
print(head(final_df[, c("university_id", "university", "institution_type", "province", "founded_year",
"students", "staff", "students_per_staff")], 10), row.names = FALSE)

# Export for further analysis / loading into Postgres (UTF-8, empty string = NULL)
write.csv(final_df, "q4_universities_clean.csv", row.names = FALSE, na = "", fileEncoding = "UTF-8")
write.csv(before_after, "q4_quality_report.csv", row.names = FALSE)
cat("\nSaved to", getwd(), ":\n",
    " q4_universities_raw.csv (4.1), q4_universities_clean.csv (4.2), q4_quality_report.csv,
q4_robots.txt, folder q4_raw_pages/\n")
```

Appendix D: R script for Question 4.3 (q4_load_postgres.R)

The complete script for Question 4.3: connection to PostgreSQL, table creation, CSV import and the two SQL investigations. The database password is typed in when the script runs and is not stored in the code.

Listing 10: `q4_load_postgres.R`

```
# q4 load postgres.R  -  ITBDA4-14 Project 2, Question 4.3
# Loads the cleaned CSV into a PostgreSQL relational table and runs two SQL investigations.
#
# BEFORE RUNNING: 1) q4_universities_clean.csv must be in your Question 4 folder
#                 2) PostgreSQL must be running and the database "universities_db" must exist
# HOW TO RUN: set the working directory to your Question 4 folder, open this file in
#             RStudio and click "Source". Type your PostgreSQL password when asked.

# Settings
db_host <- "localhost"
db_port <- 5432
db_name <- "universities_db"
db_user <- "postgres"
csv_file <- "q4_universities_clean.csv"

# Packages (installed automatically if missing)
pkgs <- c("DBI", "RPostgres")
missing_pkgs <- pkgs[!(pkgs %in% rownames(installed.packages()))]
if (length(missing_pkgs) > 0) {
  message("Installing missing packages: ", paste(missing_pkgs, collapse = ", "))
  install.packages(missing_pkgs, repos = "https://cloud.r-project.org")
}
suppressPackageStartupMessages(invisible(lapply(pkgs, library, character.only = TRUE)))
cat("Working directory:\n", getwd(), "\n\n")
if (!file.exists(csv_file)) stop("Cannot find ", csv_file, " in the working directory. Run
q4_scrape_clean.R first.")


# 1. CONNECT TO POSTGRESQL (the password is typed in, never saved in the code)

cat("---- 1. CONNECTING TO POSTGRESQL ----\n")
pw <- Sys.getenv("PGPASSWORD", unset = "")
if (!nzchar(pw)) {
  if (requireNamespace("rstudioapi", quietly = TRUE) && rstudioapi::isAvailable()) {
    pw <- rstudioapi::askForPassword(paste0("PostgreSQL password for user '", db_user, "'"))
  } else {
    pw <- readline(paste0("PostgreSQL password for user '", db_user, "': "))
  }
}
if (is.null(pw) || !nzchar(pw)) stop("No password entered. Source the script again and type the
password.")

con <- tryCatch(
  dbConnect(RPostgres::Postgres(), host = db_host, port = db_port, dbname = db_name, user = db_user,
password = pw,
           bigint = "numeric"),
  error = function(e) stop("Could not connect to PostgreSQL: ", conditionMessage(e),
                           "\nCheck that PostgreSQL is running, the password is right and the database '",
db_name, "' exists."))
cat("Connected to database '", db_name, "' on ", db_host, ":", db_port, "\n", sep = "")
cat(dbGetQuery(con, "SELECT version() AS server_version")$server_version, "\n\n")


# 2. CREATE THE RELATIONAL TABLE (data types, primary key, constraints)

cat("---- 2. CREATING THE TABLE ----\n")
create_sql <- "
DROP TABLE IF EXISTS universities;

CREATE TABLE universities (
    university_id         INTEGER       PRIMARY KEY,
    university            VARCHAR(150)  NOT NULL UNIQUE,
    institution_type      VARCHAR(40)   NOT NULL
                          CHECK (institution_type IN ('Traditional University',
                                                      'Comprehensive University',
                                                      'University of Technology')),
    province              VARCHAR(30)   NOT NULL,
    founded_year          INTEGER       CHECK (founded_year BETWEEN 1800 AND 2026),
    years_since_founding  INTEGER,
    students              INTEGER       CHECK (students > 0),
    staff                 INTEGER       CHECK (staff > 0),
    students_per_staff    NUMERIC(6,1),
    article_words         INTEGER,
    n_citations           INTEGER,
    lead_words            INTEGER,
    lead_text             TEXT,
```

```
    founded_year_imputed  BOOLEAN       NOT NULL,
    students_imputed      BOOLEAN       NOT NULL,
    staff_imputed         BOOLEAN       NOT NULL,
    source_url            TEXT          NOT NULL,
    scraped_on            DATE          NOT NULL
);
"
writeLines(create_sql, "q4_create_table.sql")
for (stmt in strsplit(create_sql, ";")[[1]]) if (nzchar(trimws(stmt))) suppressMessages(dbExecute(con,
stmt))
cat("Table 'universities' created (SQL saved in q4_create_table.sql)\n\n")


# 3. IMPORT THE CLEANED CSV INTO THE TABLE

cat("---- 3. IMPORTING THE CSV ----\n")
df <- read.csv(csv_file, stringsAsFactors = FALSE, na.strings = "", encoding = "UTF-8")
df$scraped_on <- as.Date(df$scraped_on)
cat("Rows read from CSV:", nrow(df), " | Columns:", ncol(df), "\n")
dbWriteTable(con, "universities", df, append = TRUE, row.names = FALSE)
n_loaded <- dbGetQuery(con, "SELECT COUNT(*) AS n FROM universities")$n
cat("Rows now in the PostgreSQL table:", n_loaded, "\n\n")

cat("Table structure as stored in PostgreSQL:\n")
print(dbGetQuery(con, "
  SELECT column_name, data_type, is_nullable
  FROM information_schema.columns
  WHERE table_name = 'universities'
  ORDER BY ordinal_position"), row.names = FALSE)
cat("\nFirst rows of the table:\n")
print(dbGetQuery(con, "SELECT university_id, university, institution_type, province, founded_year,
students, staff
                       FROM universities ORDER BY university_id LIMIT 6"), row.names = FALSE)


# 4. TWO SQL ANALYTICAL INVESTIGATIONS

cat("\n\n---- 4. SQL INVESTIGATIONS ----\n")

# Investigation 1: how do the three institution types compare in size, staffing and age?
# (staffing figures use only universities whose staff AND student counts were actually reported)
query1 <- "
SELECT institution_type,
       COUNT(*)                                                                    AS universities,
       SUM(students)                                                               AS total_students,
       ROUND(AVG(students))                                                        AS avg_students,
       ROUND((PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY students))::numeric) AS median_students,
       ROUND(AVG(students_per_staff)
             FILTER (WHERE NOT staff_imputed AND NOT students_imputed), 1)   AS
avg_students_per_staff_reported,
       COUNT(*) FILTER (WHERE NOT staff_imputed AND NOT students_imputed)    AS unis_with_reported_ratio,
       ROUND(AVG(years_since_founding))                                            AS avg_age_years,
       COUNT(*) FILTER (WHERE students_imputed)                                    AS imputed_enrolments
FROM universities
GROUP BY institution_type
ORDER BY total_students DESC;
"

# Investigation 2: how is enrolment spread across provinces, and which university leads each one?
query2 <- "
WITH province_totals AS (
    SELECT province,
           COUNT(*)                                AS universities,
           SUM(students)                           AS total_students,
           COUNT(*) FILTER (WHERE students_imputed) AS imputed_enrolments
    FROM universities
    WHERE province <> 'Unknown'
    GROUP BY province
),
largest AS (
    SELECT DISTINCT ON (province)
           province, university AS largest_university, students AS largest_enrolment
    FROM universities
    WHERE province <> 'Unknown'
    ORDER BY province, students DESC
)
SELECT p.province,
       p.universities,
       p.total_students,
       ROUND(100.0 * p.total_students / SUM(p.total_students) OVER (), 1) AS pct_of_national_enrolment,
       RANK() OVER (ORDER BY p.total_students DESC)                       AS enrolment_rank,
       p.imputed_enrolments,
       l.largest_university,
       l.largest_enrolment
```

```
FROM province_totals p
JOIN largest l USING (province)
ORDER BY enrolment_rank;
"
writeLines(c("-- Investigation 1: institution types compared", trimws(query1), "",
             "-- Investigation 2: enrolment by province", trimws(query2)), "q4_analysis.sql")

cat("\nInvestigation 1 - institution types compared (size, staffing, age):\n")
res1 <- dbGetQuery(con, query1); print(res1, row.names = FALSE)

cat("\nInvestigation 2 - enrolment by province, share of national total and largest university:\n")
res2 <- dbGetQuery(con, query2); print(res2, row.names = FALSE)

write.csv(res1, "q4_sql_result1_institution_types.csv", row.names = FALSE)
write.csv(res2, "q4_sql_result2_provinces.csv", row.names = FALSE)

dbDisconnect(con)    # close the connection once all the work is done

cat("\nSaved to", getwd(), ":\n",
    " q4_create_table.sql, q4_analysis.sql, q4_sql_result1_institution_types.csv,
q4_sql_result2_provinces.csv\n",
    "Open q4_analysis.sql in pgAdmin (Query Tool on universities_db) to run the queries there too.\n")
```

AI Evidence Section

[AI Code 1 – 4 may necessitate evidence submission as specified in your assessment instruction, this section is reserved for evidence where relevant, please delete if not applicable.]

AI Disclosure Appendix

AI tool used	Purpose / how it was used	How I verified or adapted the output
Claude Sonnet 5.5	Helped me identify gaps and error	Cross-checked with the peer-reviewed sources, my own work, claude just helped me with identifying errors and guiding me on how to upload to GitHub using Rstudio's terminal
Grammarly	Grammer and plagiarism checker	Helped me identify gaps in spelling and grammar, fixed minor issues work is still my own, used as the plagiarism checker
Claude Sonnet 5.5	Took all of my original work and helped me format it into this document style	I made a plain doc with all my code and answers and claude formatted it for me, into this document and style, It did not generate any content it just helped create this document style

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance with institutional policy.

Student Signature(s)	Date
MIKAYLEN NAIDOO	**11 November 2026**